

# Chapter 3. Dynamic Neural Networks

Neural Networks

2022/2023

Máster Universitario en Inteligencia Artificial, Reconocimiento  
de Formas e Imagen Digital

Departamento de Sistemas Informáticos y Computación

# Index

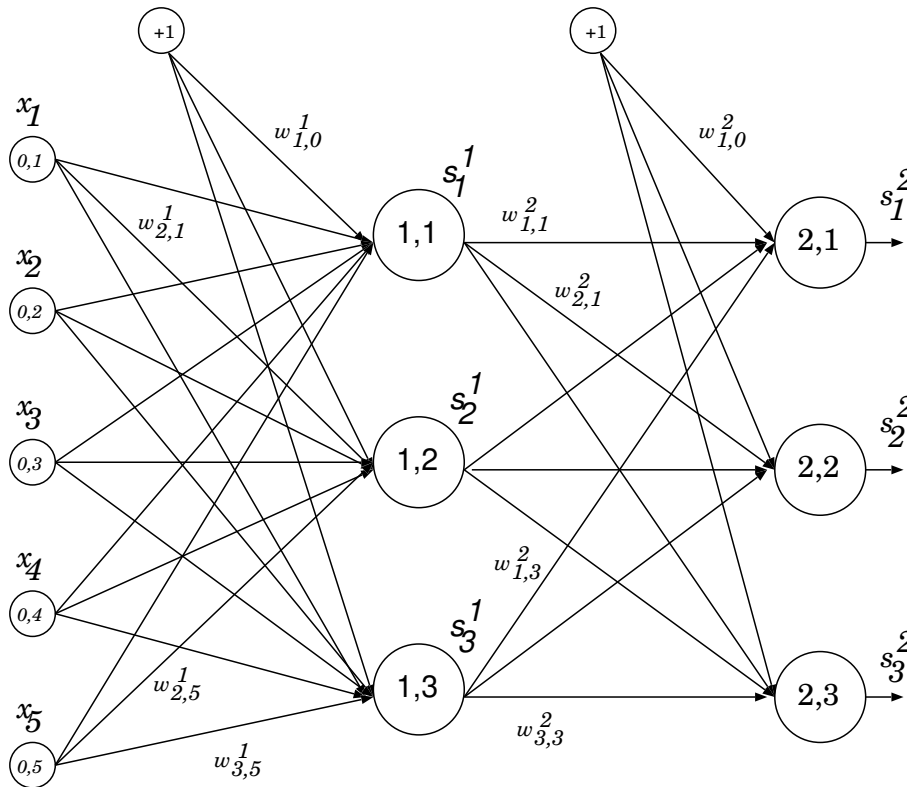
- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Index

- 1 *Introduction* ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Introduction

A feed-forward network



$$\mathbf{g} : \mathbb{R}^{M_0} \rightarrow \mathbb{R}^{M_2}$$

$$\begin{aligned} g_k(\mathbf{x}; \theta) &\equiv s_k^2(\mathbf{x}) \\ &= f\left(\sum_{j=0}^{M_1} w_{k,j}^2 s_j^1(\mathbf{x})\right) \\ &= f\left(\sum_{j=0}^{M_1} w_{k,j}^2 f\left(\sum_{j'=0}^{M_0} w_{jj'}^1 x_{j'}\right)\right), \\ &\quad 1 \leq k \leq M_2 \end{aligned}$$

A compact notation:  $\mathbf{g}(\mathbf{x}; \theta) = \mathbf{f}(\mathbf{W}^2 \mathbf{s}^1(\mathbf{x})) = \mathbf{f}(\mathbf{W}^2 \mathbf{f}(\mathbf{W}^1 \mathbf{x}))$   
 ( $\mathbf{f}$  is an extension of  $f$  from scalars to vectors)

Problem: The representation of the objects should be of fixed size

# Introduction

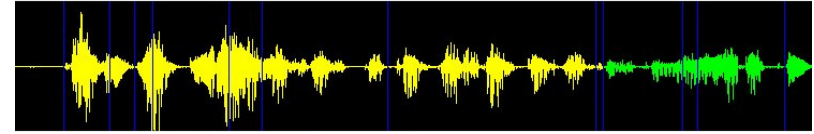
- Problem: The input must be numeric (e.g. from a vector space). What happens when the object are discrete? (for example words)
- Problem: The representation of the objects should be of fixed size. What happens when the object are of variable size? (for example sequence of subobjects)

# Index

- 1 Introduction ▷ 3
- 2 *Sequence processing* ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Sequence processing

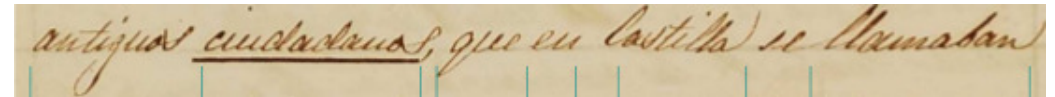
- Speech recognition and understanding



- Text translation

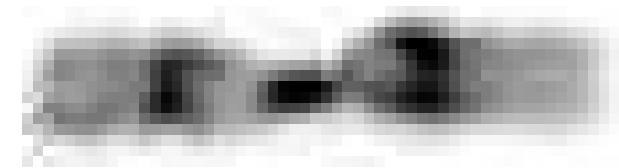
“¿ hay alguna habitación tranquila libre ?” / “is there a quiet room available” ?

- Handwritten text recognition



- Currency change

- Classification of chromosomes



- Weather forecast

- Protein sequences



- Video sequences (Video description, tracking, ...)



- Image processing (compression, description, VQA, ...)



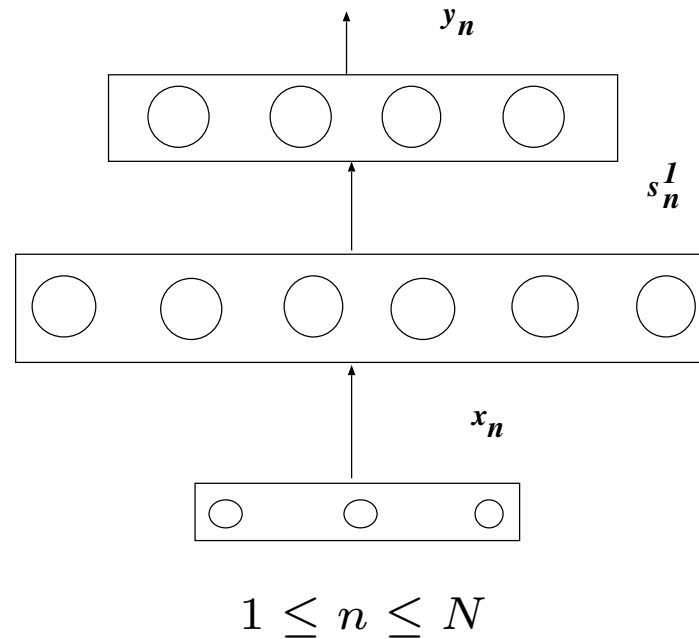
# Sequence processing

- Feed-forward networks cannot process directly sequences of variable length.
  - Naive solution:
    1. Define a maximum length (for example  $N$  vectors in  $\mathbb{R}^D$ )
    2. Transform each sequence  $\mathbf{x}$  into a vector in  $\mathbb{R}^{N \times D}$  by linear or non-linear interpolation.
  - Alternative solutions:
    - \* Hybrid approaches: hidden Markov models and multilayer perceptrons.
    - \* **Dynamic networks: bounded-memory networks and recurrent neural networks**



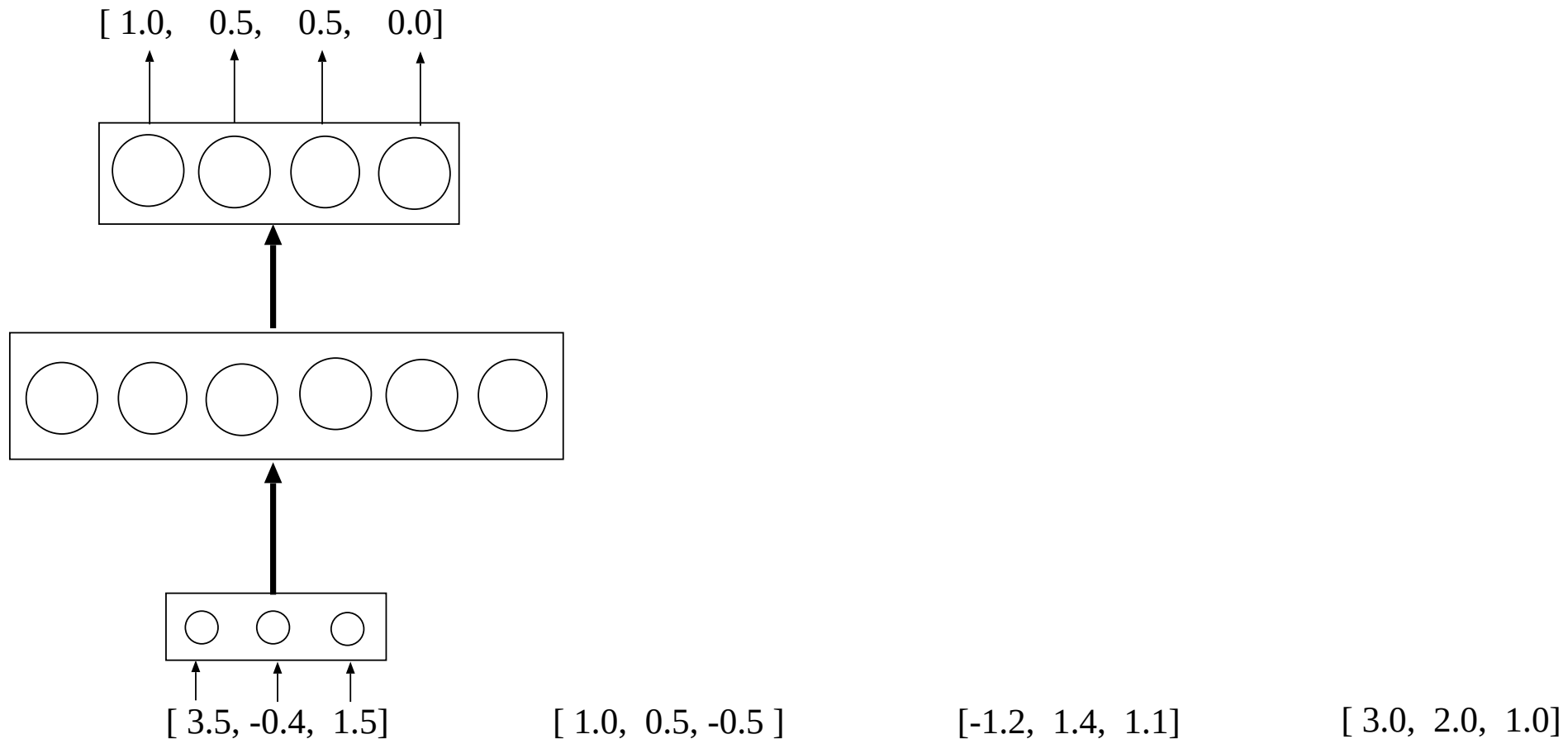
# Sequence processing: dynamic networks

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N \rightarrow \mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N$$

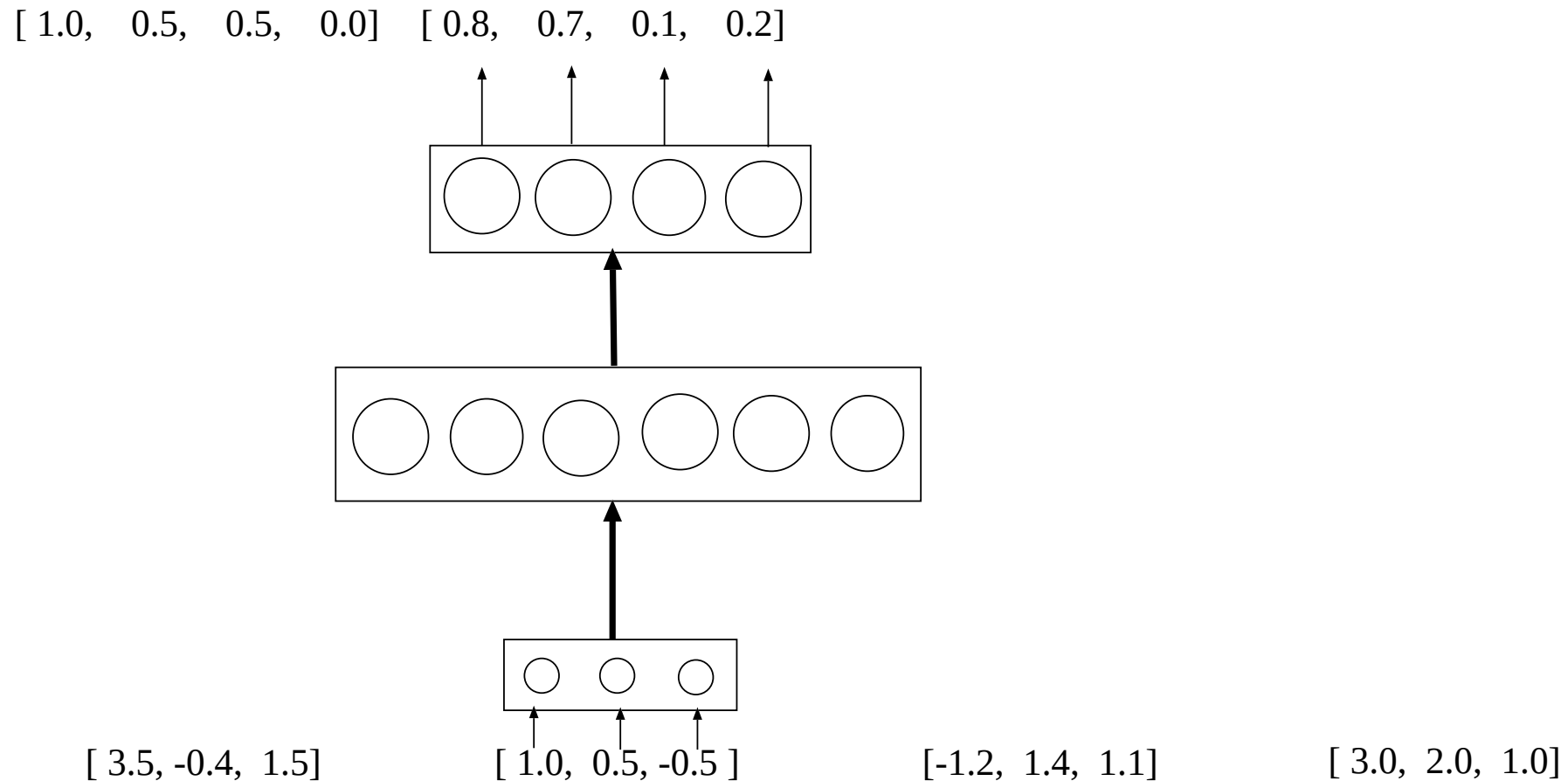


| $t$      | input layer    |               | hidden layer     |               | output layer   |
|----------|----------------|---------------|------------------|---------------|----------------|
| 1        | $\mathbf{x}_1$ | $\Rightarrow$ | $\mathbf{s}_1^1$ | $\Rightarrow$ | $\mathbf{y}_1$ |
| 2        | $\mathbf{x}_2$ | $\Rightarrow$ | $\mathbf{s}_2^1$ | $\Rightarrow$ | $\mathbf{y}_2$ |
| $\vdots$ | $\vdots$       |               | $\vdots$         |               | $\vdots$       |
| $N$      | $\mathbf{x}_N$ | $\Rightarrow$ | $\mathbf{s}_N^1$ | $\Rightarrow$ | $\mathbf{y}_N$ |

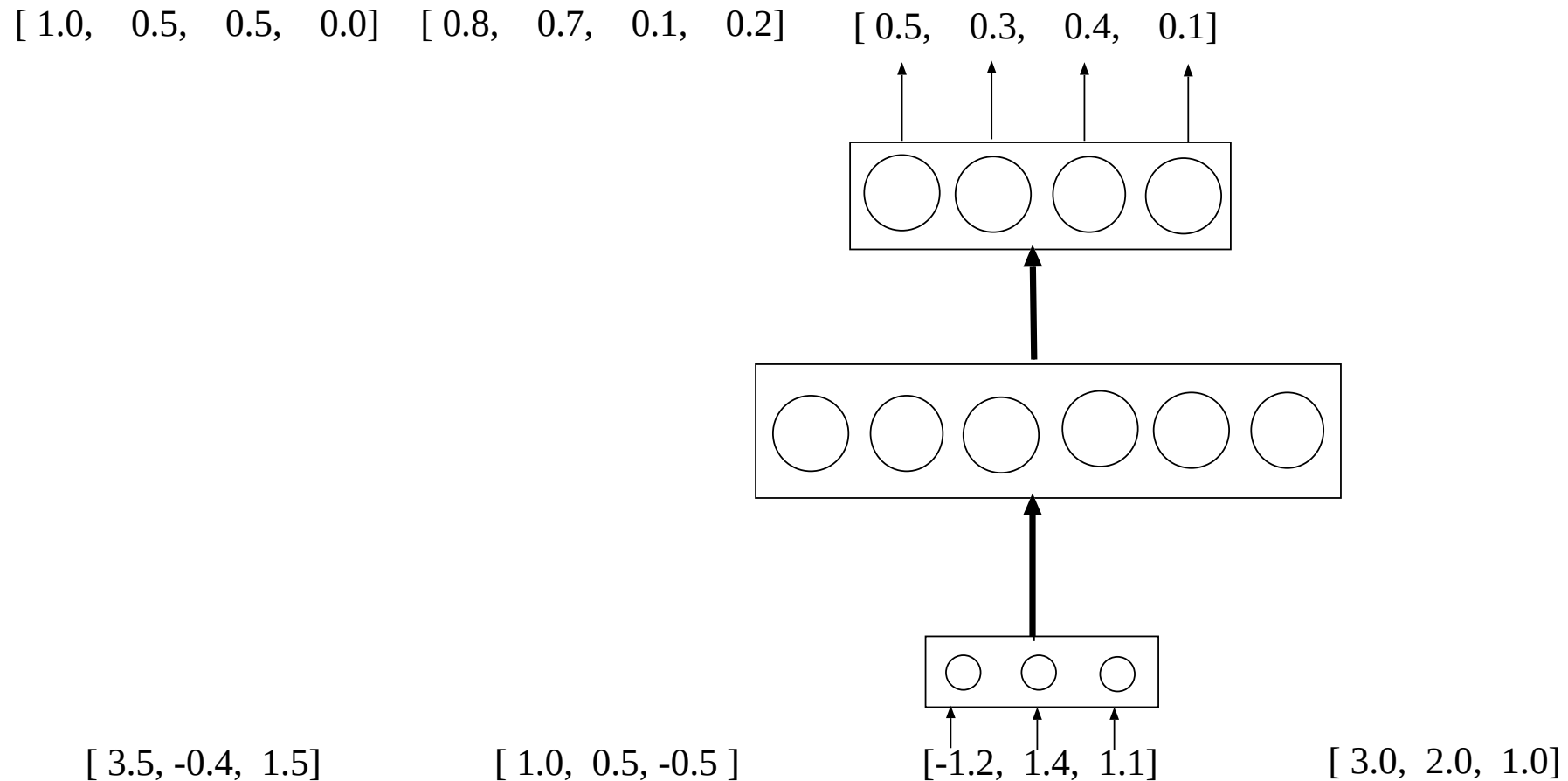
# Sequence processing: dynamic networks



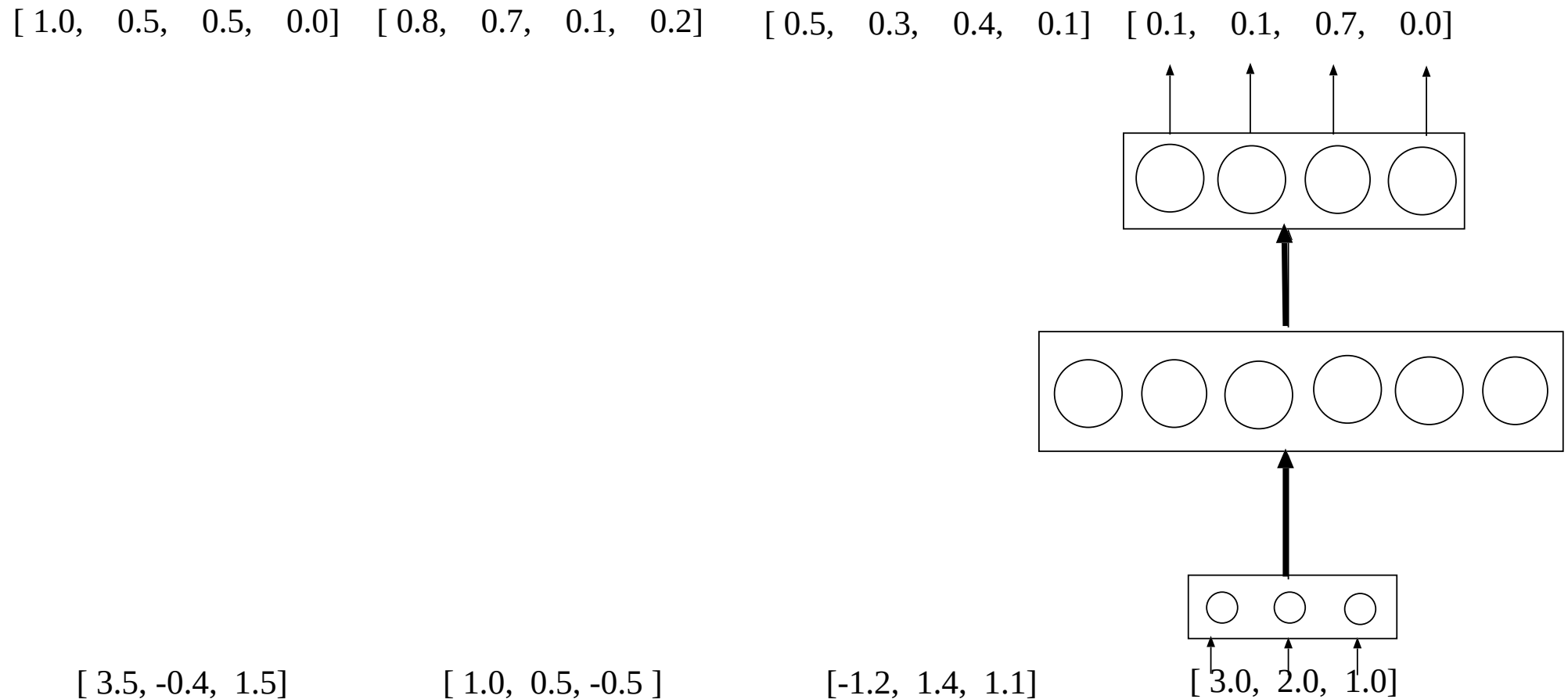
# Sequence processing: dynamic networks



# Sequence processing: dynamic networks

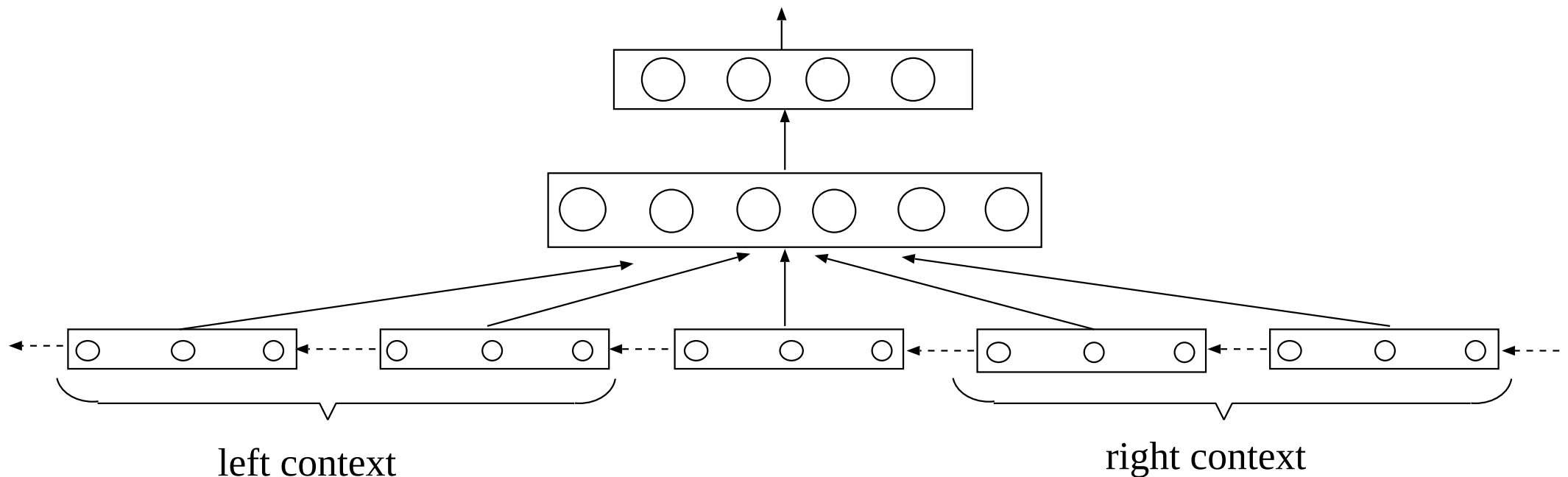


# Sequence processing: dynamic networks

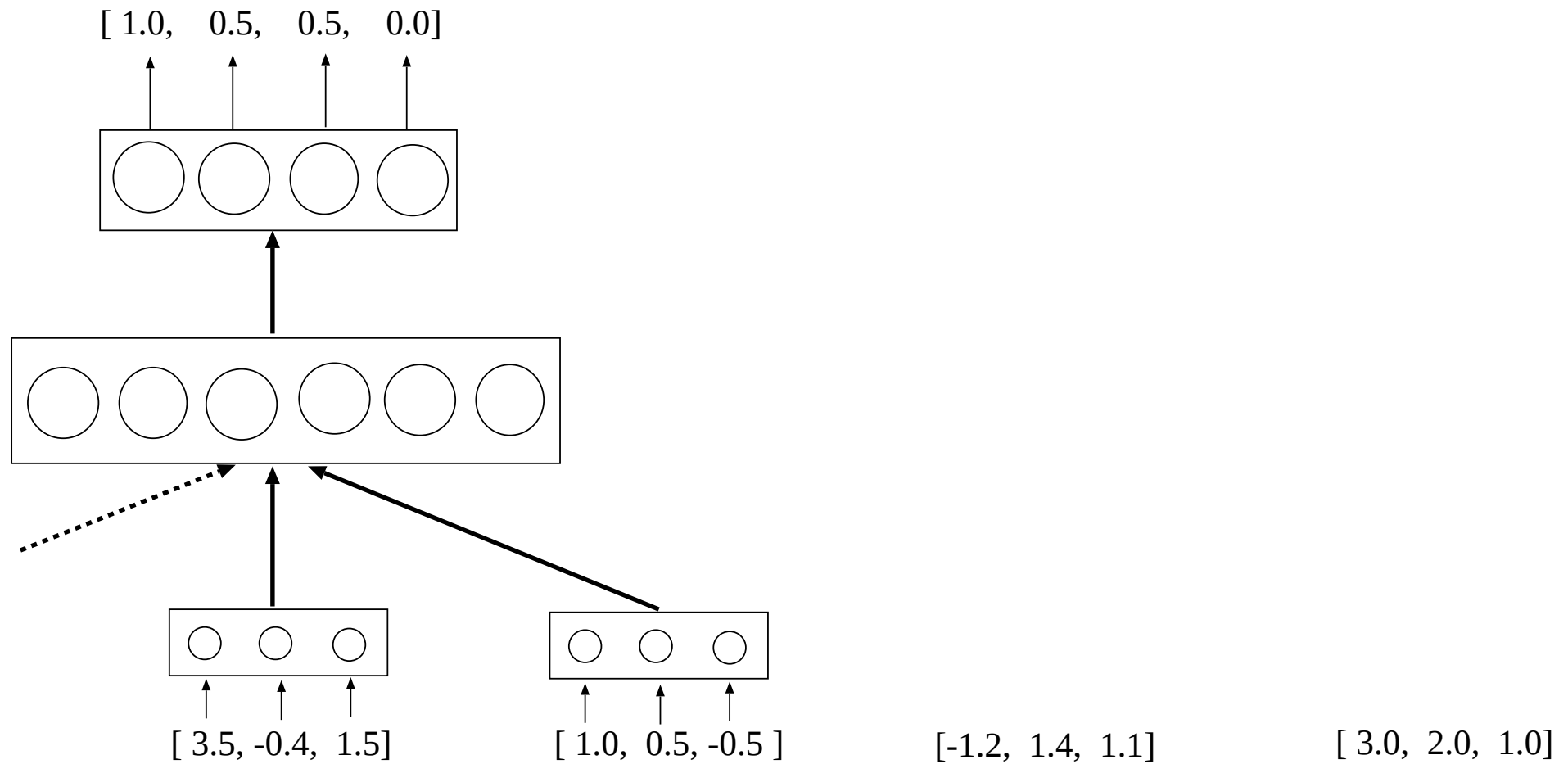


# Sequence processing: dynamic networks

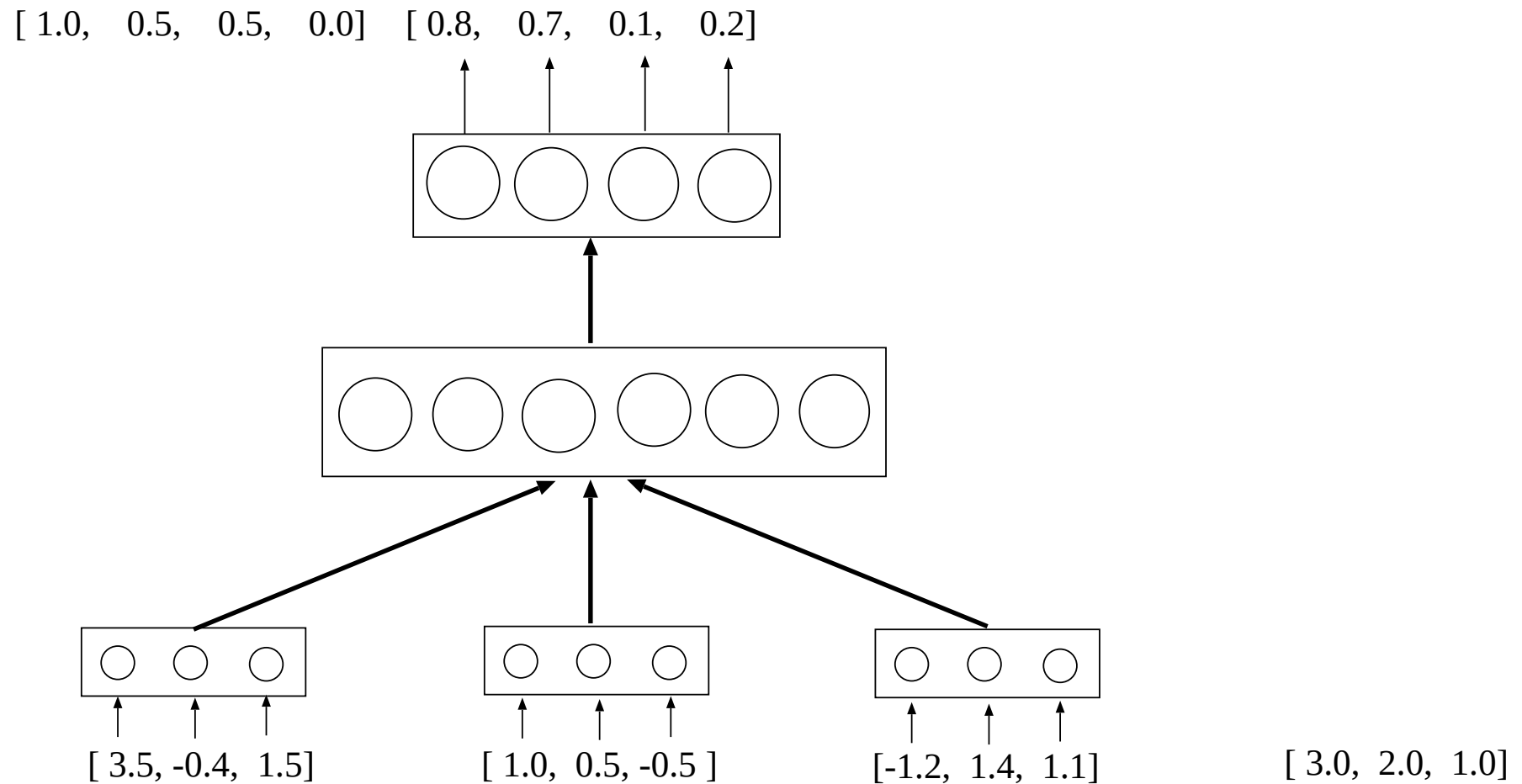
*NETtalk* (a precedent of the modern convolutional neural networks -CNN-) that learns to pronounce written English text (text  $\rightarrow$  phonetic transcription) (Sejnowski & Rosenberg. 1986)



# Sequence processing: dynamic networks

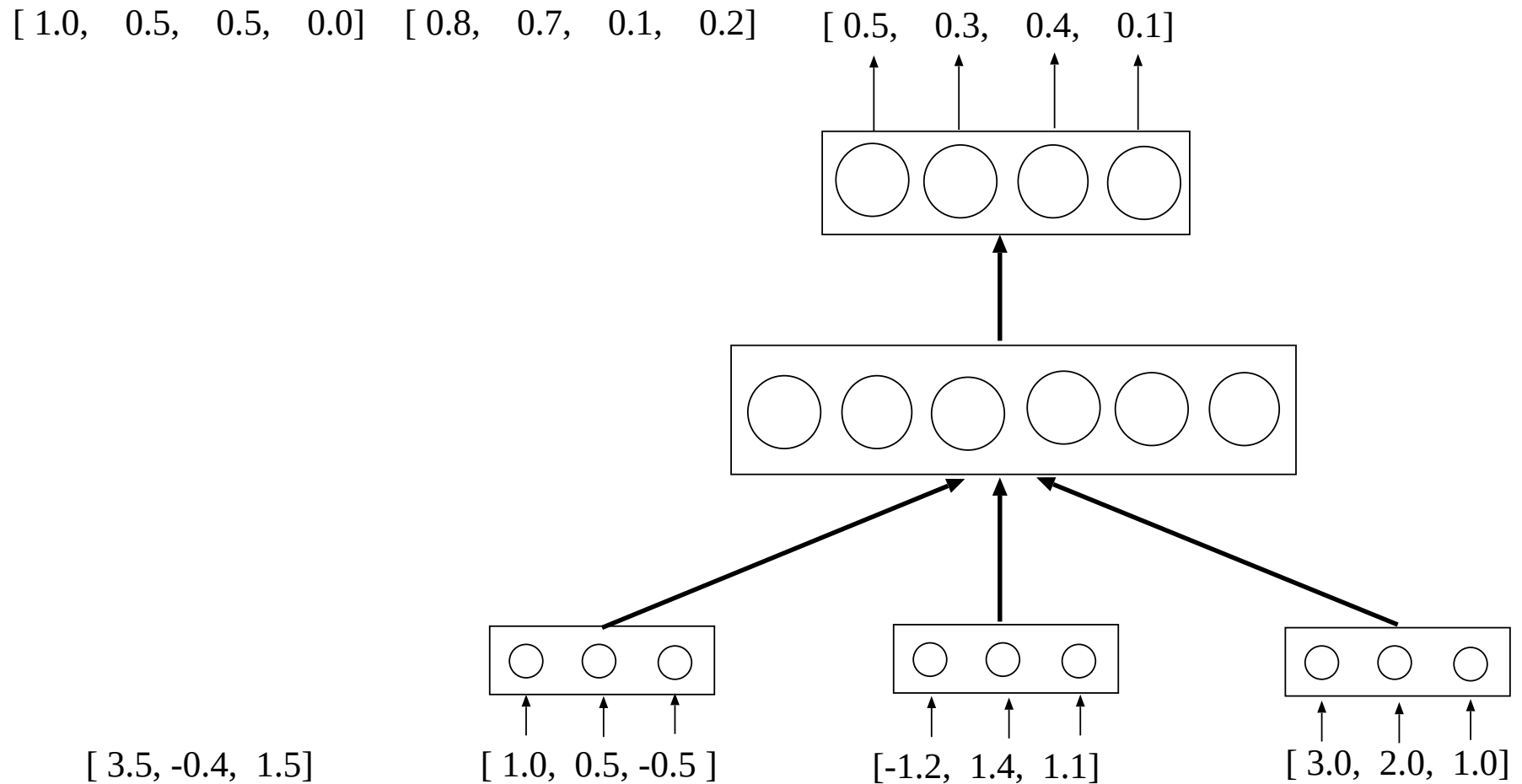


# Sequence processing: dynamic networks



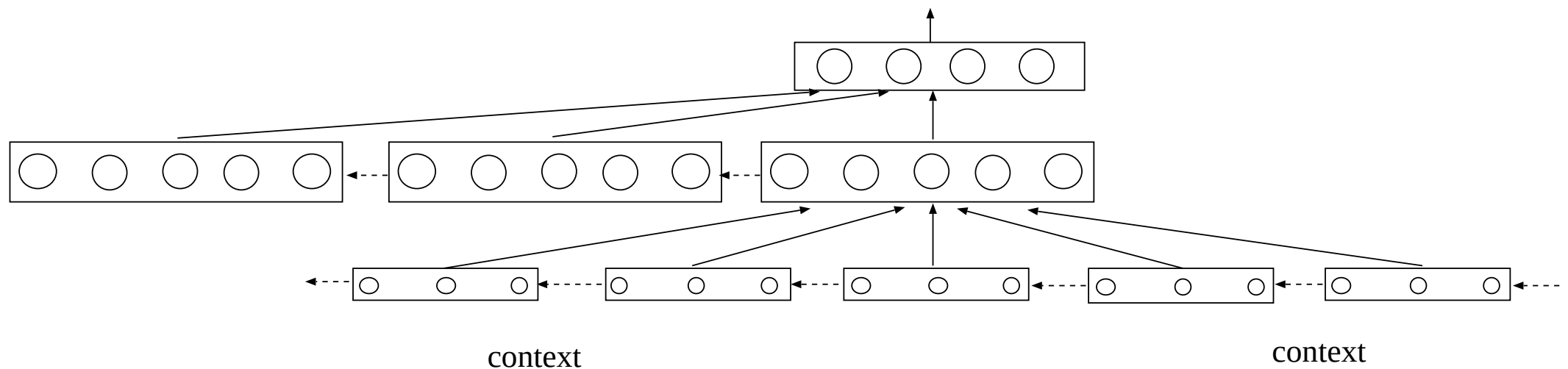


# Sequence processing: dynamic networks

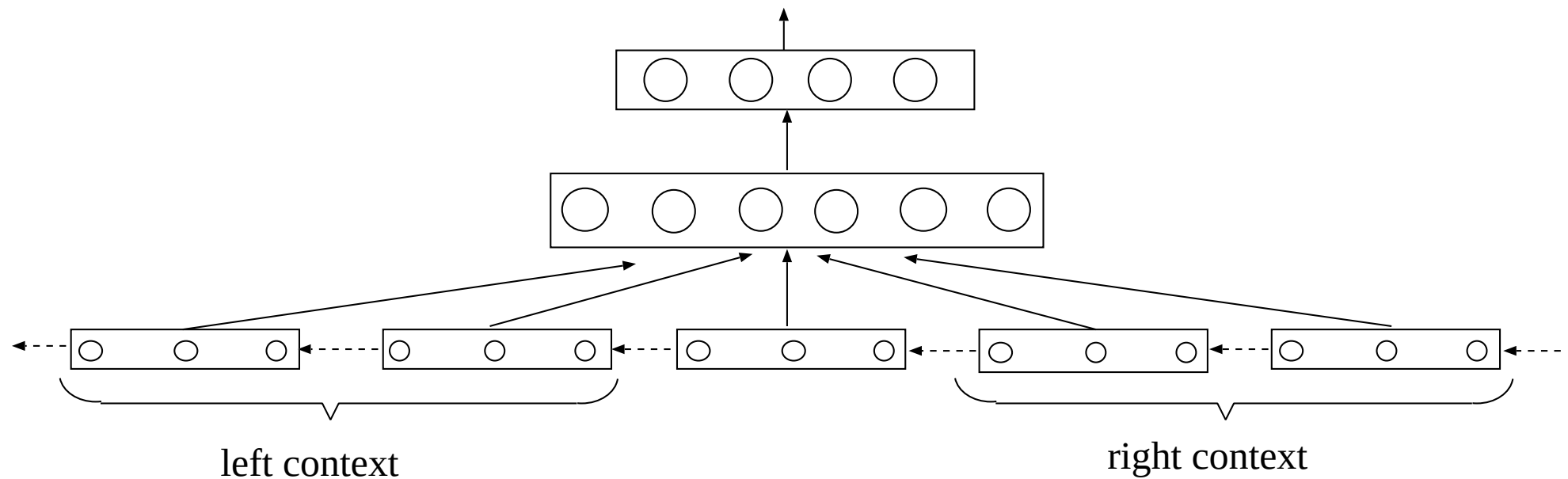


# Sequence processing: dynamic networks

*Time-delayed neural networks* (TDNN) that learn to classify pattern with shift-invariance for ASR, reading lip movement, HTR, video analysis, ... (Waibel et al. 1989)



# Training dynamic networks



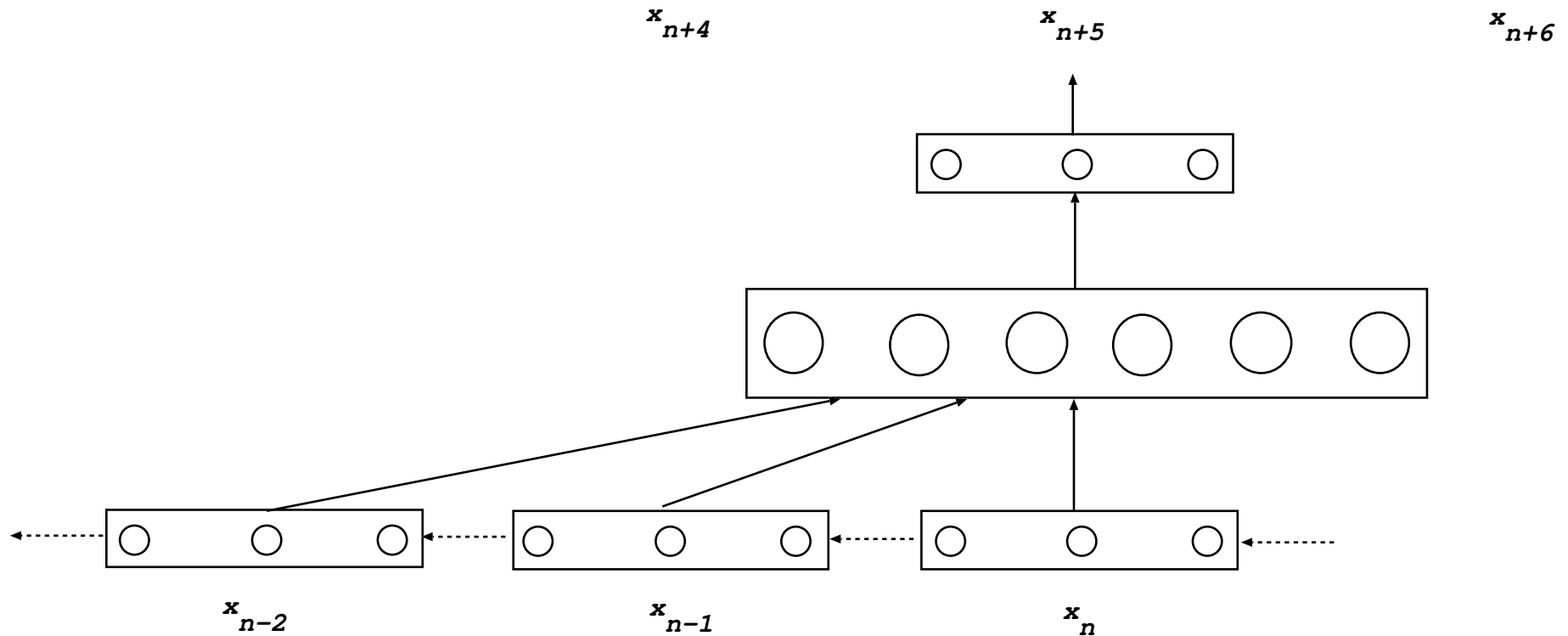
A training pair

$$(\mathbf{x}_1 \mathbf{x}_2 \dots \mathbf{x}_N, \mathbf{t}_1 \mathbf{t}_2 \dots \mathbf{t}_N) \in \mathbb{R}^{N_1 N} \times \mathbb{R}^{N_2 N}$$

is equivalent to a conventional training set

$$\{(\mathbf{00} \mathbf{x}_1 \mathbf{x}_2 \mathbf{x}_3, \mathbf{t}_1), \dots, (\mathbf{x}_{n-2} \mathbf{x}_{n-1} \mathbf{x}_n \mathbf{x}_{n+1} \mathbf{x}_{n+2}, \mathbf{t}_n), \dots, (\mathbf{x}_{N-2} \mathbf{x}_{N-1} \mathbf{x}_N \mathbf{00}, \mathbf{t}_N)\}$$

# Sequence processing: prediction of temporal series



A training sequence is:

$$\{(\mathbf{000}, \mathbf{x}_5), (\mathbf{00x}_1, \mathbf{x}_6), (\mathbf{0x}_1\mathbf{x}_2, \mathbf{x}_7), (\mathbf{x}_1\mathbf{x}_2\mathbf{x}_3, \mathbf{x}_8), \dots, (\mathbf{x}_{N-7}\mathbf{x}_{N-6}\mathbf{x}_{N-5}, \mathbf{x}_N)\}$$

Example

# A taxonomy of dynamic networks

## 1. Memory dependency

- Networks with bounded-memory
  - NETtalk.
  - TDNN.
  - CNN.
- Recurrent neural networks
  - Synchronous recurrent neural networks
  - Asynchronous recurrent neural networks: encoder-decoder architecture.
- Transformer.

## 2. Recurrent networks

- Time-synchronous recurrent neural networks
  - Regular recurrent neural networks.
- Fixed-point recurrent networks
  - Hopfield networks, BAM.
  - Boltzmann machines.

## 3. Time representation

- Continuous-time networks (models: differential equations)
- Discrete-time networks (models: difference equations)

# Index

- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 *Embeddings* ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Input/output coding of discrete variables

- *Local coding*: One input/output unit per symbol (maybe too many units).
- *Distributed coding*: Using compact codes
- *A learned word embedding*\*: by projecting the word into  $\mathbb{R}^D$  (Mikolov NIPS 2013)

| Symbol | local |   |   |   |   |   | distributed |   |   | embedding |       |
|--------|-------|---|---|---|---|---|-------------|---|---|-----------|-------|
| "a"    | 0     | 0 | 0 | 0 | 0 | 1 | 0           | 0 | 0 | 0.15      | 0.66  |
| "b"    | 0     | 0 | 0 | 0 | 1 | 0 | 0           | 0 | 1 | -0.66     | 0.02  |
| "c"    | 0     | 0 | 0 | 1 | 0 | 0 | 0           | 1 | 0 | 0.32      | -0.49 |
| "d"    | 0     | 0 | 1 | 0 | 0 | 0 | 0           | 1 | 1 | 0.89      | 0.78  |
| "e"    | 0     | 1 | 0 | 0 | 0 | 0 | 1           | 0 | 0 | -0.17     | -0.12 |
| "f"    | 1     | 0 | 0 | 0 | 0 | 0 | 1           | 0 | 1 | 0.33      | 0.55  |

# Word embeddings

- Neural language model (Bengio 2003)
  - Bag-of-words neural networks (Mikolov 2013)
  - Continuous skip-grams (Mikolov 2013)
  - Pre-trained neural networks: BERT (Devlin 2019)
  - ...
- 
- Extension to images: for example, patch embeddings [Dosovitskiy 2020]



# Word embeddings

- A word  $x$  from a ordered vocabulary  $V_X$  with index  $i(x)$
- $\mathbf{W}_E(x) \equiv [\mathbf{W}_E]_{i(x)} = \mathbf{x} \in \mathbb{R}^{D_W}$ : a row of  $\mathbf{W}_E$  that is the word embedding of  $x$
- A sentence  $x_1^N$  is a finite-length sequence of words from  $V_X$
- $\mathbf{x}_1^N = \mathbf{W}_E(x_1) \dots \mathbf{W}_E(x_N)$  is a finite-length sequence of word embeddings of the words in  $x_1^N$
- $\mathbf{S}_E(x_1^N) \equiv \mathbf{X} \in \mathbb{R}^{D_S}$  is the sentence embedding (vector) of the sentence  $x_1^N$
- word2vec is one of the most popular toolkit  
<https://code.google.com/archive/p/word2vec/>
- Curretly, pre-trained models as BERT and similars are also used.  
<https://github.com/google-research/bert>

# Word embeddings (Bengio et al. 2003)

Trigram language model:

$$p(X_n = x_n \mid x_{n-1}, x_{n-2}) =$$

$$[\mathbf{f}_{sm}(\mathbf{W}^3[\mathbf{W}(x_{n-1})\mathbf{W}(x_{n-2})] +$$

$$\mathbf{W}^1\mathbf{f}_{th}(\mathbf{W}^2[\mathbf{W}(x_{n-1})\mathbf{W}(x_{n-2})]))]_{i(x_n)}$$

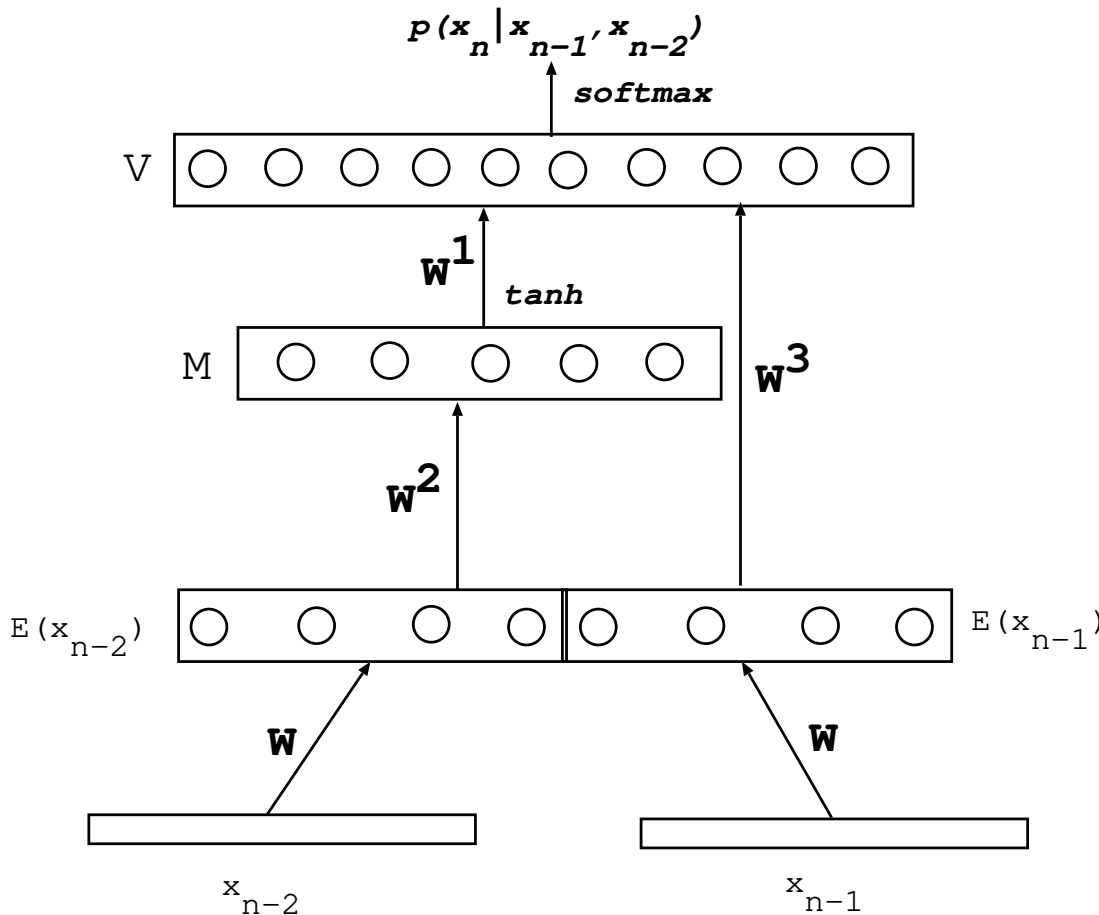
Goal: given a sequence of words  $x_1^N$ ,

$$\mathbf{W}_E =$$

$$\arg\max_{\mathbf{W}} \max_{\mathbf{W}^1, \mathbf{W}^2, \mathbf{W}^3} \left( \sum_{n=1}^N \log p(x_n \mid x_{n-2}x_{n-1}) + \right.$$

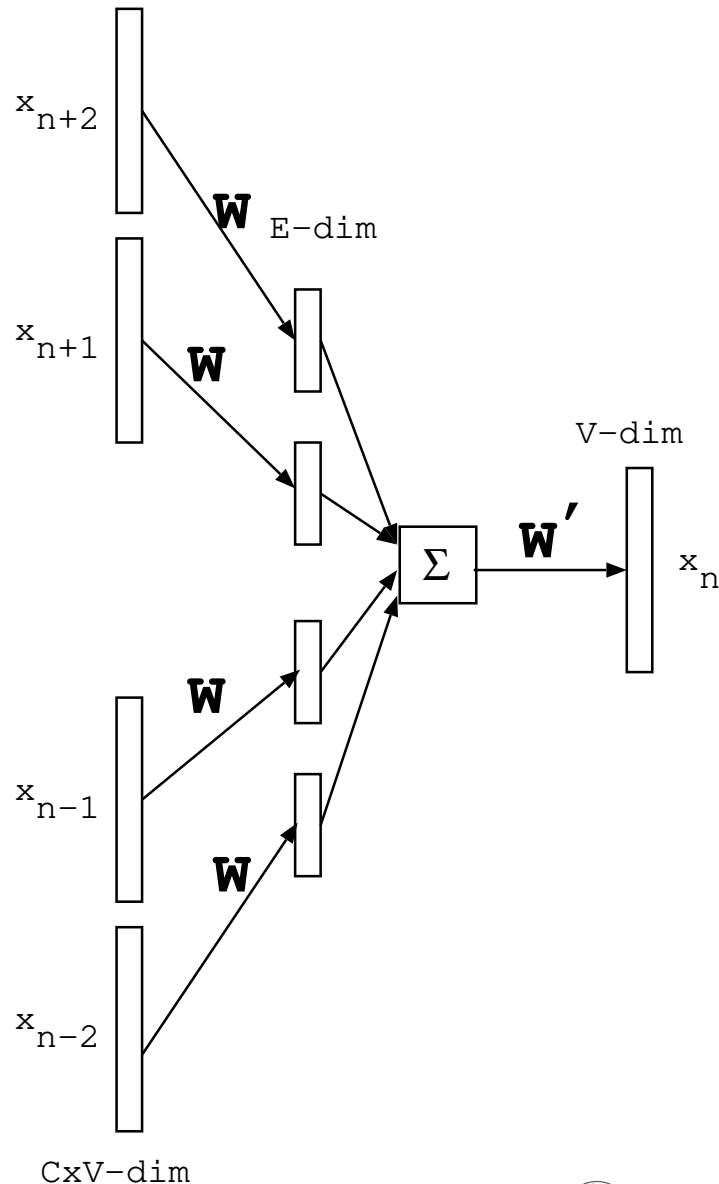
$$\left. R(\mathbf{W}^1, \mathbf{W}^2, \mathbf{W}^3, \mathbf{W}) \right)$$

Solution: back error propagation



# Encoder-decoders for word embeddings (Mikolov et al, 2013)

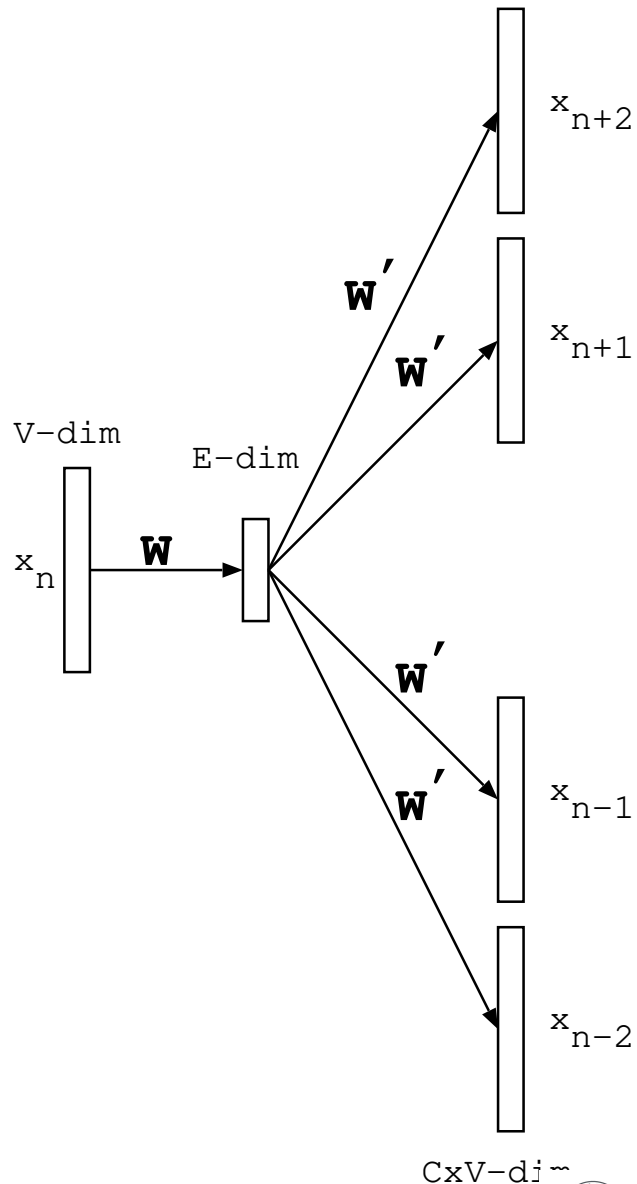
## Bag-of-words neural networks



- $C(x_n) = \{x_{n-c}, \dots, x_{n-1}, x_{n+1}, \dots, x_{n+c}\}$
- $p(x_n | C(x_n)) = [\mathbf{f}_{sm}(\mathbf{W}' \sum_{x \in C(x_n)} \mathbf{W}(x))]_{i(x_n)}$
- Goal: given a sequence of words  $x_1^N$ ,  
$$\mathbf{W}_E = \operatorname{argmax}_{\mathbf{W}} \max_{\mathbf{W}'} \sum_{n=1}^N \log p(x_n | C(x_n))$$
- Solution: stochastic gradient ascent
- Toolkit: <https://code.google.com/archive/p/word2vec/>

# Encoder-decoders for word embeddings (Mikolov et al, 2013)

## Continuous skip-gram neural networks



- $C(x_n) = \{x_{n-c}, \dots, x_{n-1}, x_{n+1}, \dots, x_{n+c}\}$
- $p(C(x_n) \mid x_n) = \prod_{x \in C(x_n)} [\mathbf{f}_{sm}(\mathbf{W}' \mathbf{W}(x_n))]_{i(x)}$
- Goal: given a sequence of words  $x_1^N$ ,
$$\mathbf{W}_E = \operatorname{argmax}_{\mathbf{W}} \max_{\mathbf{W}'} \sum_{n=1}^N \log p(C(x_n) \mid x_n)$$
- Solution: stochastic gradient ascent
- Toolkit: <https://code.google.com/archive/p/word2vec/>

# Continuous representation of sentences

Given a sequence of words  $x_1^N$ ,

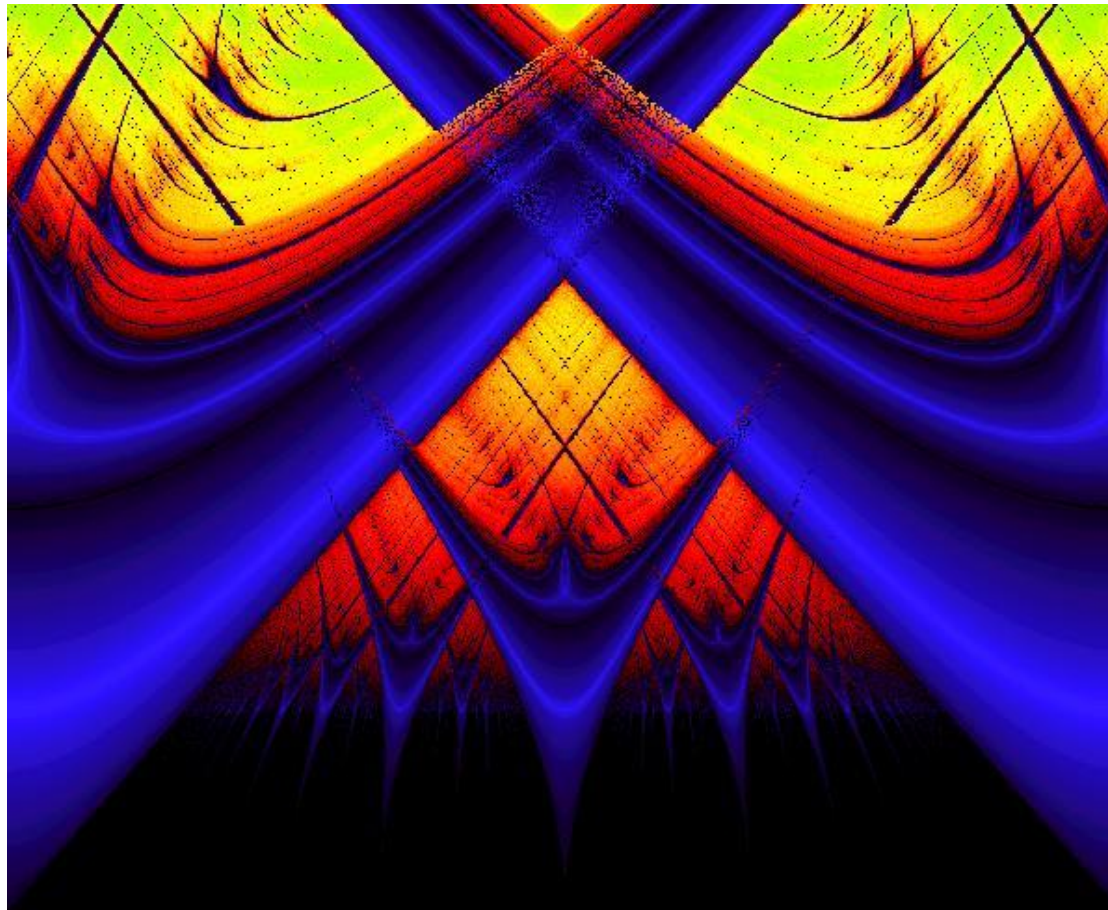
- Sum:  $\mathbf{S_E}(x_1^N) = \sum_{n=1}^N \mathbf{W_E}(x_n)$  or  $\mathbf{S_E}(x_1^N) = \frac{\sum_{n=1}^N \mathbf{W_E}(x_n)}{N}$
- Product (element-wise):  $\mathbf{S_E}(x_1^N) = \prod_{n=1}^N \mathbf{W_E}(x_n)$  or  $\mathbf{S_E}(x_1^N) = \sqrt[N]{\prod_{n=1}^N \mathbf{W_E}(x_n)}$
- doc2vec (<https://github.com/piskvorky/gensim/>): A variation of the skip-gram neural network or the n-gram based approach. For each new sentence, a new row is added to the matrix of sentence embedding and trained.
- From the state of a recurrent neural network at the end of processing a sentence.
- From pre-trained models as BERT or SentenceBERT (siameses network), Universal Sentence Encoder, ...

# Index

- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 *Simple recurrent neural networks* ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Simple recurrent networks

“Image of the week” SDSC: A National Laboratory for Computational Science & Engineering  
“Simple recurrent neural networks” Michael Casey, (SDSC). 1997.



# Simple recurrent networks

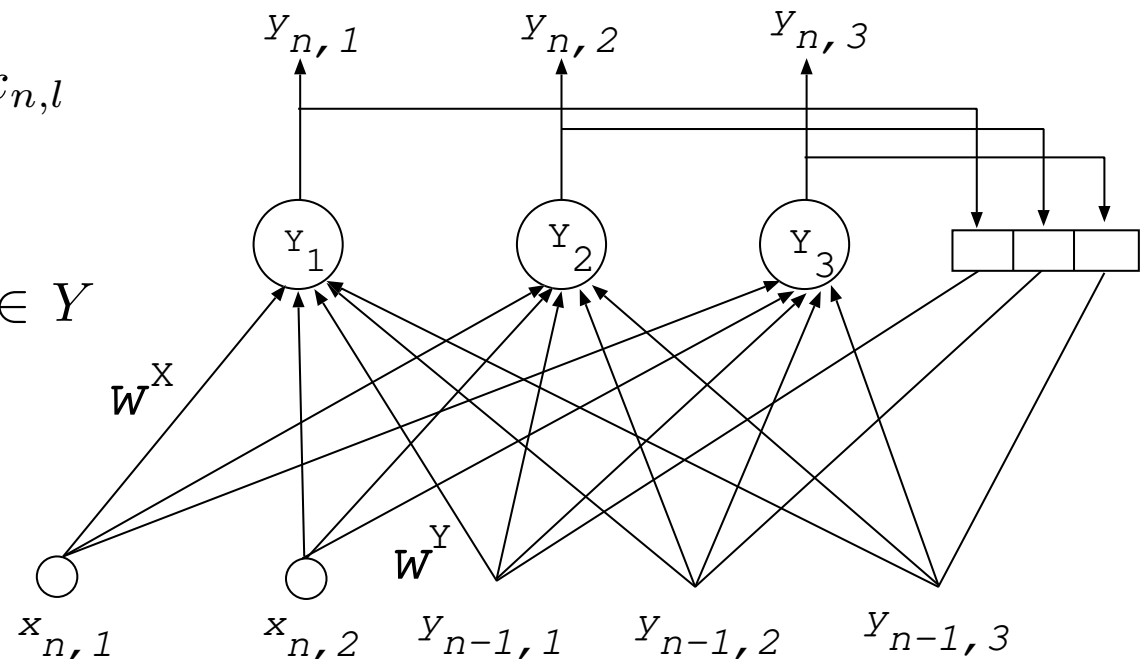
- A set  $Y$  of  $d_Y$  units and a set  $X$  of  $d_X$  inputs.
- Input:  $\mathbf{x}_n \in \mathbb{R}^{d_X}$  and output:  $\mathbf{y}_n \in \mathbb{R}^{d_Y}$ ,  $n = 1, 2, \dots$
- **Total input** of the unit  $k \in Y$  in  $n$ :

$$h_{n,k} = \sum_{l \in Y} \omega_{k,l}^Y y_{n-1,l} + \sum_{l \in X} \omega_{k,l}^X x_{n,l}$$

- The **state** (and the output) of  $k \in Y$  in  $n$ :

$$y_{n,k} = \begin{cases} f(h_{n,k}) & n \geq 1 \\ 0 & n = 0 \end{cases}$$

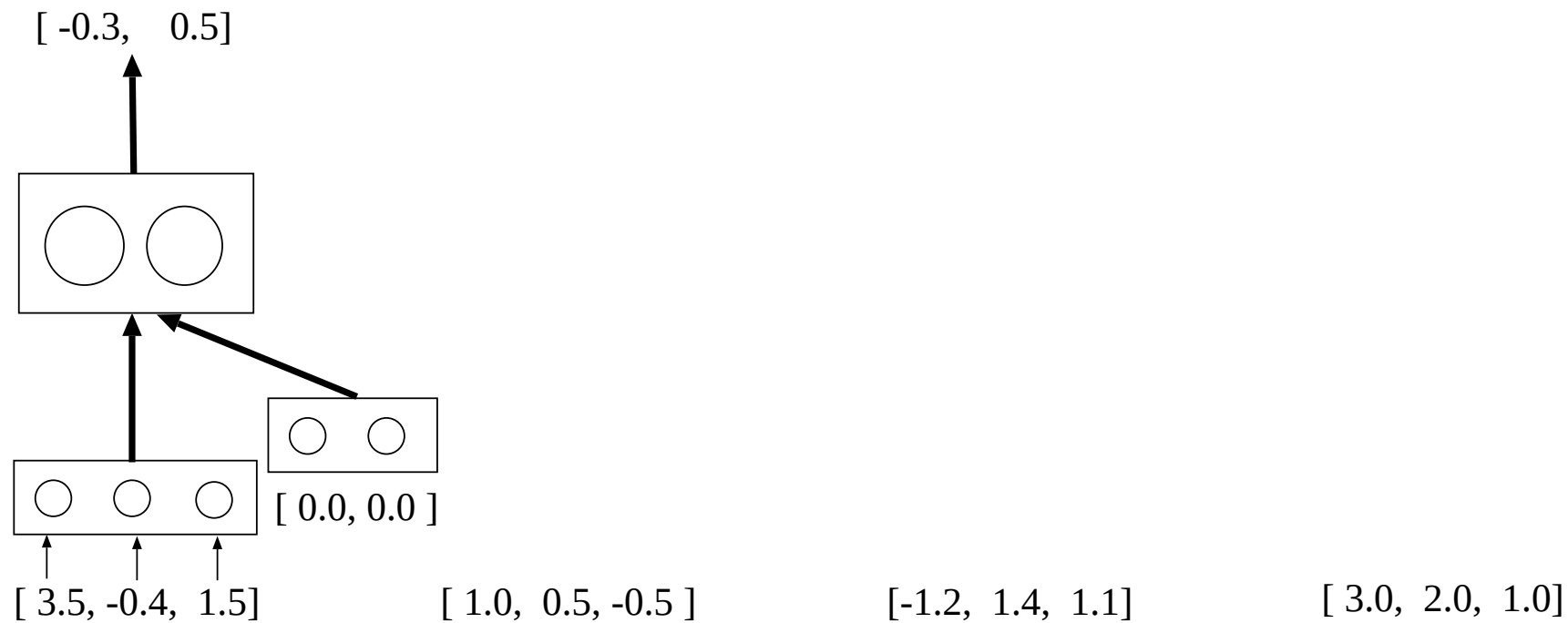
$f$  is an activation function



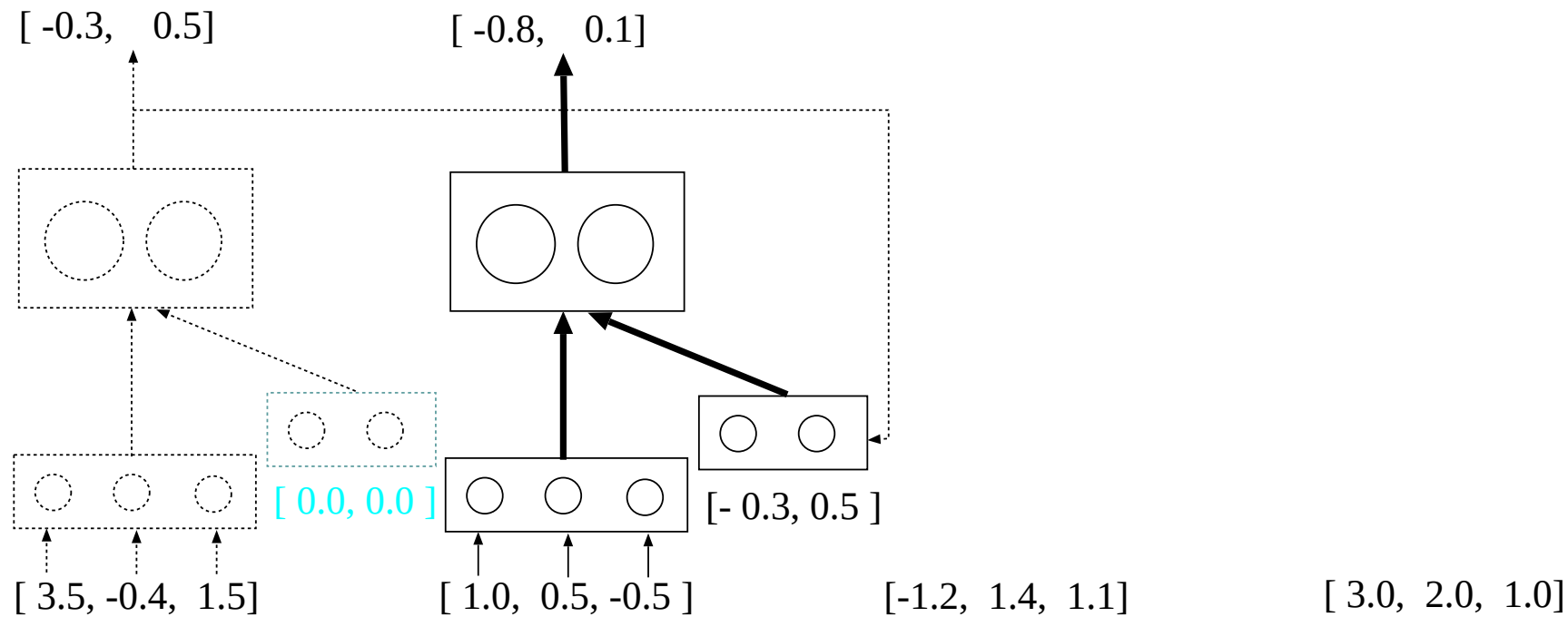
- In a compact notation:  $\mathbf{y}_n = \mathbf{f}(W^Y \mathbf{y}_{n-1} + W^X \mathbf{x}_n)$



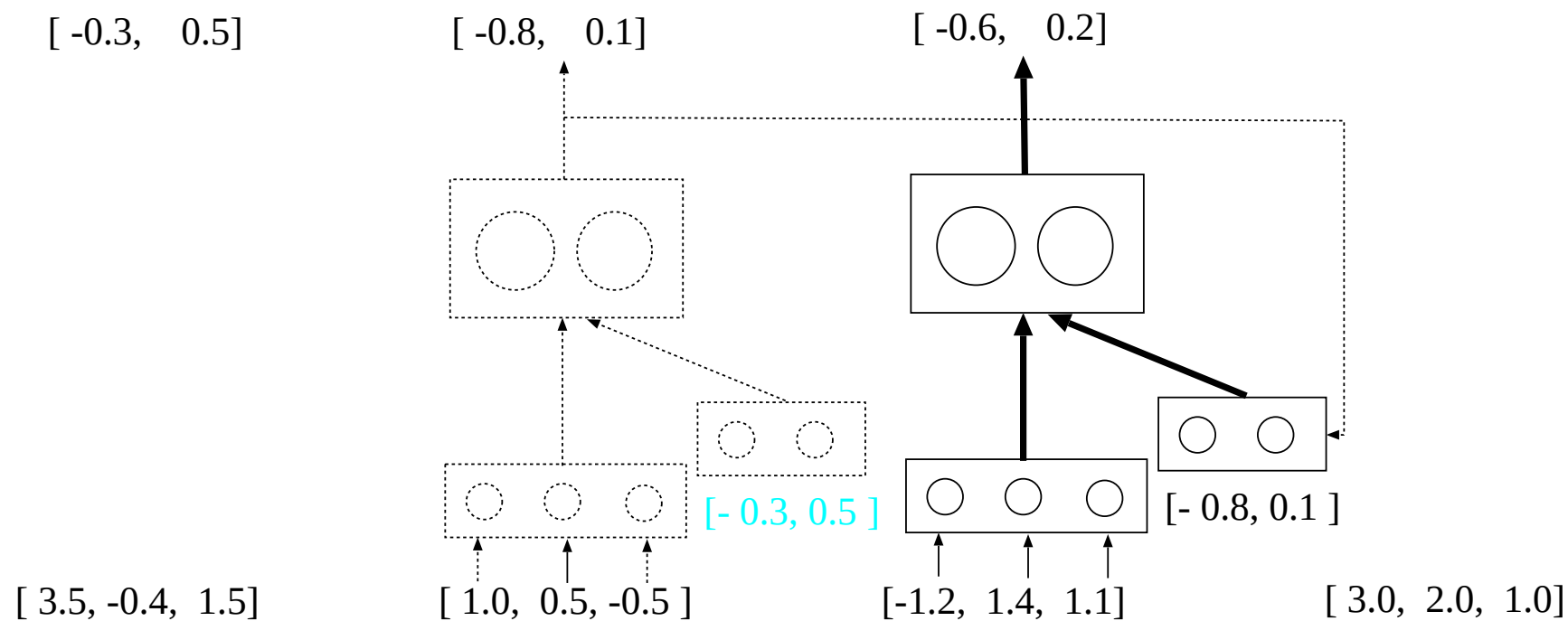
# Sequence processing: simple recurrent networks



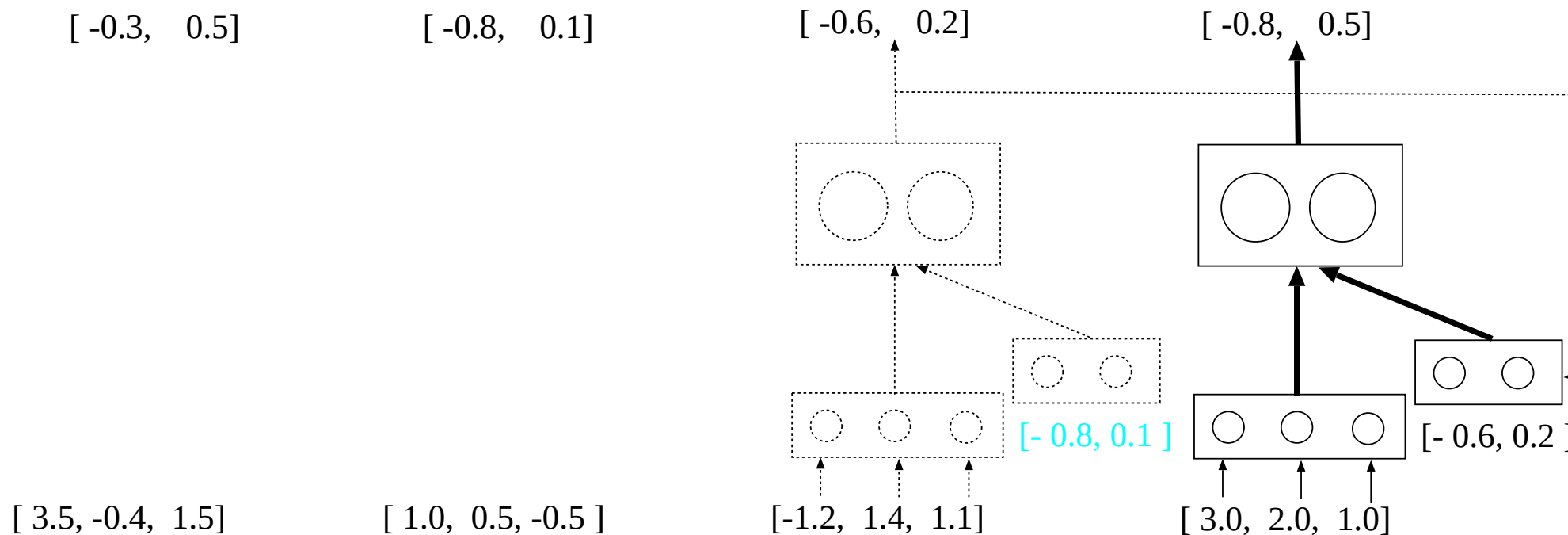
# Sequence processing: simple recurrent networks



# Sequence processing: simple recurrent networks



# Sequence processing: simple recurrent networks



# Training simple recurrent networks (regression)

- A *training sequence*  $A = (\mathbf{x}_n, \mathbf{t}_n)_{n=1,\dots,N} : \mathbf{x}_n \in \mathbb{R}^{d_X}$  and  $\mathbf{t}_n \in \mathbb{R}^{d_Y}$
- The *error in unit  $k$  in  $n$*  is:  $e_{n,k} = t_{n,k} - y_{n,k}(\mathbf{w})$  for  $1 \leq n \leq N$  and  $k \in Y$
- The *total error* (objective function) is: 
$$\mathcal{E}_A(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \sum_{k \in Y} (e_{n,k})^2$$
- Computing a local minimum of  $\mathcal{E}_A$  with respect to  $\omega_{ij}$ : GRADIENT DESCENT

$$\Delta \omega_{i,j} = -\rho \frac{\partial \mathcal{E}_A(\mathbf{w})}{\partial \omega_{i,j}} \quad i \in Y, \quad j \in Y \cup X$$

- Approaches:
  - Forward gradient algorithm.
  - Back-propagation through time algorithm.

# Training simple recurrent networks (classification)

- A *training sequence*  $A = (\mathbf{x}_n, \mathbf{t}_n)_{n=1, \dots, N} : \mathbf{x}_n \in \mathbb{R}^{d_X}$  and  $\mathbf{t}_n \in \{0, 1\}^{d_Y}$
- The objective function (cross-entropy): In this case the activation function to produce  $y_{n,k}(\mathbf{w})$  is a softmax function:

$$C_A(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N \sum_{k=1}^{N_2} t_{n,k} \log y_{n,k}(\mathbf{w})$$

- Computing a local minimum of  $C_A$  with respect to  $\omega_{ij}$ : GRADIENT DESCENT

$$\Delta \omega_{i,j} = -\rho \frac{\partial C_A(\mathbf{w})}{\partial \omega_{i,j}} \quad i \in Y, \quad j \in Y \cup X$$

- Approaches:
  - Forward gradient algorithm
  - Back-propagation through time algorithm

# Training simple recurrent networks (regression)

## Forward gradient algorithm

[Williams & Zipser, 1995]

## Forward gradient algorithm: Recurrent connections

$$\mathcal{E}_A = \frac{1}{2} \sum_{n=1}^N \sum_{k \in Y} (e_{n,k})^2 = \frac{1}{2} \sum_{n=1}^N \sum_{k \in Y} (t_{n,k} - y_{n,k})^2$$

$$\Delta \omega_{i,j}^Y = -\rho \frac{\partial \mathcal{E}_A}{\partial \omega_{i,j}^Y} = \rho \sum_{n=1}^N \sum_{k \in Y} e_{n,k} \frac{\partial y_{n,k}}{\partial \omega_{i,j}^Y}$$

$$y_{n,k} = f(h_{n,k}) = f \left( \sum_{l \in Y} \omega_{k,l}^Y y_{n-1,l} + \sum_{l \in X} \omega_{k,l}^X x_{n,l} \right)$$

$$\begin{aligned} \frac{\partial y_{n,k}}{\partial \omega_{i,j}^Y} &= f'(h_{n,k}) \frac{\partial h_{n,k}}{\partial \omega_{i,j}^Y} = f'(h_{n,k}) \sum_{l \in Y} \left( \delta_{k,i} \delta_{l,j} y_{n-1,l} + \omega_{kl}^Y \frac{\partial y_{n-1,l}}{\partial \omega_{i,j}^Y} \right) \\ &= f'(h_{n,k}) \left( \delta_{k,i} y_{n-1,j} + \sum_{l \in Y} \omega_{kl}^Y \frac{\partial y_{n-1,l}}{\partial \omega_{i,j}^Y} \right) \end{aligned}$$

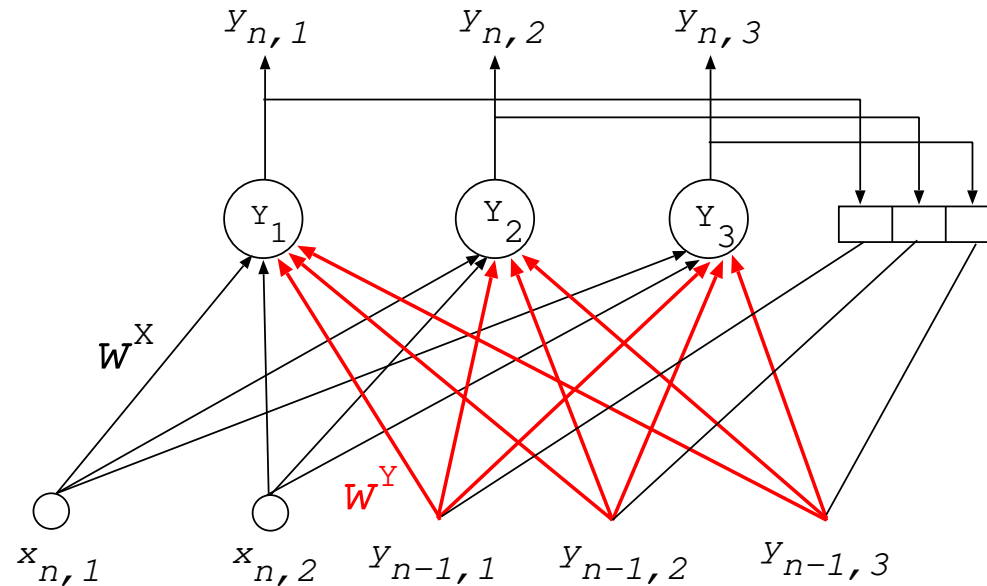


# Forward gradient algorithm: Recurrent connections

$$\Delta \omega_{ij}^Y = \rho \sum_{n=1}^N \sum_{k \in Y} e_{n,k} \frac{\partial y_{n,k}}{\partial \omega_{i,j}^Y}$$

$$\frac{\partial y_{n,k}}{\partial \omega_{i,j}^Y} = f'(h_{n,k}) \left( \delta_{k,i} y_{n-1,j} + \sum_{l \in Y} \omega_{k,l}^Y \frac{\partial y_{n-1,l}}{\partial \omega_{i,j}^Y} \right)$$

$$\frac{\partial y_{0,k}}{\partial \omega_{i,j}^Y} = 0$$



## Forward gradient algorithm: Input connections

$$\mathcal{E}_A = \frac{1}{2} \sum_{n=1}^N \sum_{k \in Y} (e_{n,k})^2 = \frac{1}{2} \sum_{n=1}^N \sum_{k \in Y} (t_{n,k} - y_{n,k})^2$$

$$\Delta \omega_{i,j}^X = -\rho \frac{\partial \mathcal{E}_A}{\partial \omega_{i,j}^X} = \rho \sum_{n=1}^N \sum_{k \in Y} e_{n,k} \frac{\partial y_{n,k}}{\partial \omega_{i,j}^X}$$

$$y_{n,k} = f(h_{n,k}) = f \left( \sum_{l \in Y} \omega_{k,l}^Y y_{n-1,l} + \sum_{l \in X} \omega_{k,l}^X x_{n,l} \right)$$

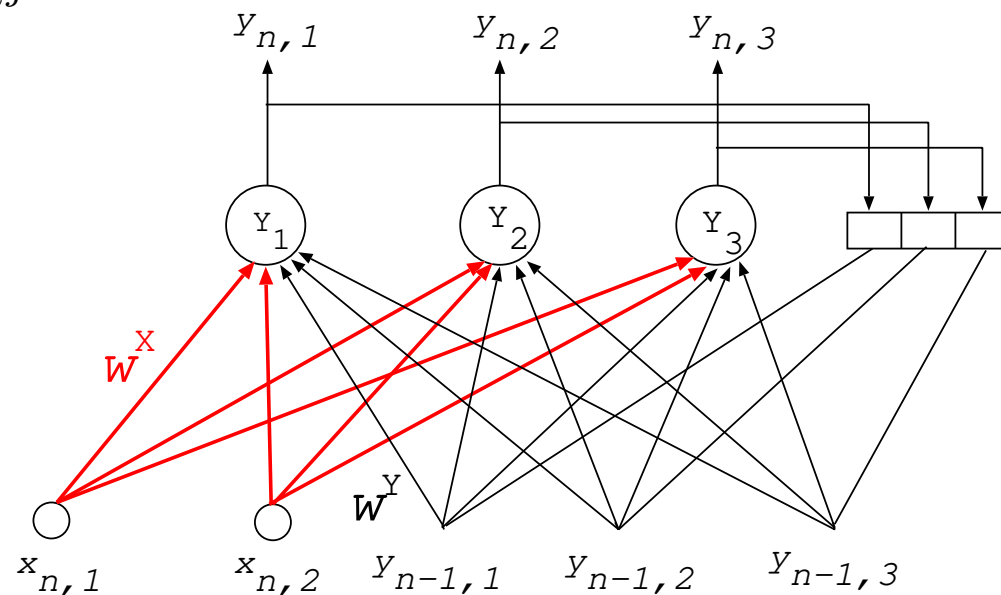
$$\begin{aligned} \frac{\partial y_{n,k}}{\partial \omega_{i,j}^X} &= f'(h_{n,k}) \frac{\partial h_{n,k}}{\partial \omega_{i,j}^X} = f'(h_{n,k}) \left( \sum_{l \in Y} \omega_{k,l}^Y \frac{\partial y_{n-1,l}}{\partial \omega_{i,j}^X} + \sum_{l \in X} \delta_{i,k} \delta_{j,l} x_{n,l} \right) \\ &= f'(h_{n,k}) \left( \sum_{l \in Y} \omega_{k,l}^Y \frac{\partial y_{n-1,l}}{\partial \omega_{i,j}^X} + \delta_{i,k} x_{n,j} \right) \end{aligned}$$

# Forward gradient algorithm: Input connections

$$\Delta\omega_{i,j}^X = \rho \sum_{n=1}^N \sum_{k \in Y} e_{n,k} \frac{\partial y_{n,k}}{\partial \omega_{i,j}^X}$$

$$\frac{\partial y_{n,k}}{\partial \omega_{i,j}^X} = f'(h_{n,k}) \left( \delta_{i,k} x_{n,j} + \sum_{l \in Y} \omega_{k,l}^Y \frac{\partial y_{n-1,l}}{\partial \omega_{i,j}^X} \right)$$

$$\frac{\partial y_{0,k}}{\partial \omega_{i,j}^X} = 0$$



# Forward gradient algorithm

- **Initialization:**  $\frac{\partial y_{0,k}}{\partial \omega_{i,j}^Y} = 0$ ;  $\frac{\partial y_{0,k}}{\partial \omega_{i,j}^X} = 0$ ;  $y_{0,k} = 0$ ;  $\Delta \omega_{i,j}^Y = 0$ ;  $\Delta \omega_{i,j}^X = 0$

- **Iterate until convergence:**

– For  $n = 1, \dots, N$ ,  $k \in Y$ ,  $i \in Y$  and  $j \in X \cup Y$

$$h_{n,k} = \sum_{l \in Y} \omega_{k,l}^Y y_{n-1,l} + \sum_{l \in X} \omega_{k,l}^X x_{n,l}$$

$$y_{n,k} = f(h_{n,k}); \quad e_{n,k} = t_{n,k} - y_{n,k}$$

$$\frac{\partial y_{n,k}}{\partial \omega_{i,j}^Y} = f'(h_{n,k}) \left( \delta_{k,i} y_{n-1,j} + \sum_{l \in Y} \omega_{k,l}^Y \frac{\partial y_{n-1,l}}{\partial \omega_{i,j}^Y} \right)$$

$$\frac{\partial y_{n,k}}{\partial \omega_{i,j}^X} = f'(h_{n,k}) \left( \delta_{i,k} x_{n,j} + \sum_{l \in Y} \omega_{k,l}^Y \frac{\partial y_{n-1,l}}{\partial \omega_{i,j}^X} \right)$$

$$\Delta \omega_{i,j}^Y += \sum_{k \in D_n} e_{n,k} \frac{\partial y_{n,k}}{\partial \omega_{i,j}^Y}; \quad \Delta \omega_{i,j}^X += \sum_{k \in D_n} e_{n,k} \frac{\partial y_{n,k}}{\partial \omega_{i,j}^X}$$

– **Updating weights with:**  $\omega_{i,j}^Y += \rho \Delta \omega_{i,j}^Y$   $i, j \in Y$ ;  $\omega_{i,j}^X += \rho \Delta \omega_{i,j}^X$   $i \in Y, j \in X$

# Forward gradient algorithm: Computational costs

$C$  = number of connections ( $C = C_Y + C_X$ ),

$C_Y$  = number of recurrent connections,  $C_X$  = number of input connections,

$d_Y$  = number of units and  $N$  = the size of the training sample

Temporal cost:  $\left\{ \begin{array}{ll} \text{Recurrent connections} & O(d_Y N C_Y d_Y) \\ \text{Input connections} & O(d_Y N C_X d_Y) \end{array} \right\}$  Total:  $O(C d_Y^2 N)$

Spatial cost:  $\left\{ \begin{array}{ll} \text{Recurrent connections} & O(d_Y C_Y) \\ \text{Input connections} & O(d_Y C_X) \end{array} \right\}$  Total:  $O(C d_Y)$

# Truncate-gradient algorithm

- **Initialization:**  $\frac{\partial y_{0,k}}{\partial \omega_{i,j}^Y} = 0; \frac{\partial y_{0,k}}{\partial \omega_{i,j}^X} = 0; y_{0,k} = 0; \Delta \omega_{i,j}^Y = 0; \Delta \omega_{i,j}^X = 0$
- **Iterate until convergence:**
  - For  $n = 1, \dots, N, k \in Y, i \in Y$  and  $j \in X \cup Y$

$$h_{n,k} = \sum_{l \in Y} \omega_{k,l}^Y y_{n-1,l} + \sum_{l \in X} \omega_{k,l}^X x_{n,l}$$

$$y_{n,k} = f(h_{n,k}); \quad e_{n,k} = t_{n,k} - y_{n,k}$$

$$\frac{\partial y_{n,k}}{\partial \omega_{i,j}^Y} = f'(h_{n,k}) \delta_{k,i} y_{n-1,j}$$

$$\frac{\partial y_{n,k}}{\partial \omega_{i,j}^X} = f'(h_{n,k}) \delta_{i,k} x_{n,j}$$

$$\Delta \omega_{i,j}^Y += \sum_{k \in D_n} e_{n,k} \frac{\partial y_{n,k}}{\partial \omega_{ij}^Y}; \quad \Delta \omega_{i,j}^X += \sum_{k \in D_n} e_{n,k} \frac{\partial y_{n,k}}{\partial \omega_{i,j}^X}$$

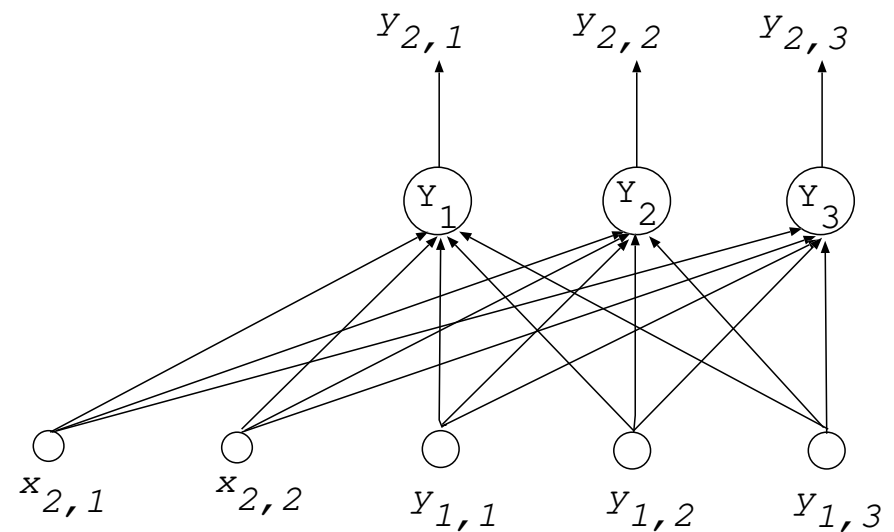
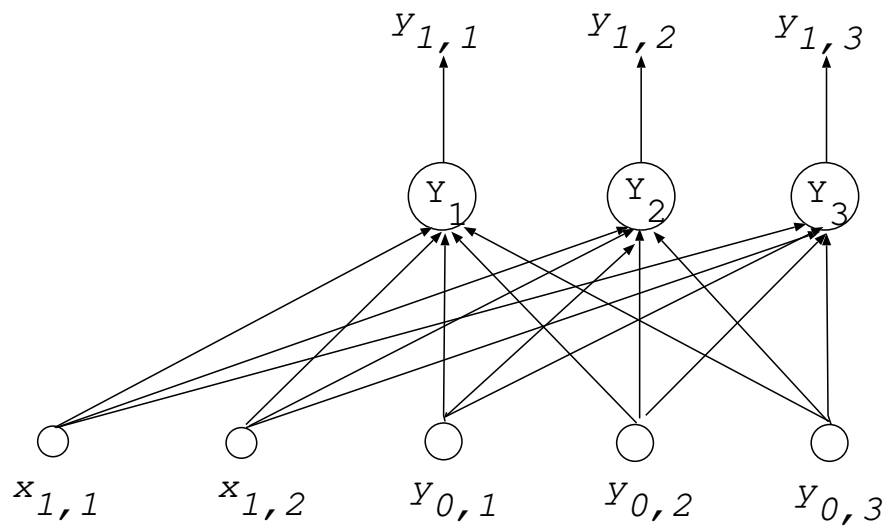
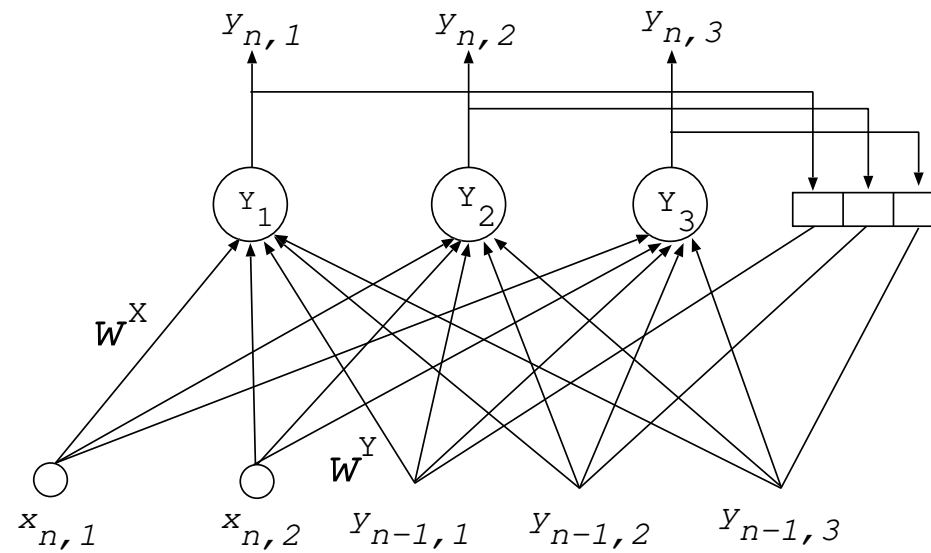
- **Updating weights with:**  $\omega_{i,j}^Y += \rho \Delta \omega_{i,j}^Y \quad i, j \in Y; \quad \omega_{i,j}^X += \rho \Delta \omega_{i,j}^X \quad i \in Y, j \in X$

Temporal cost:  $O(C d_Y N)$ ; Spatial cost:  $O(C d_Y)$

# Training simple recurrent networks (regression)

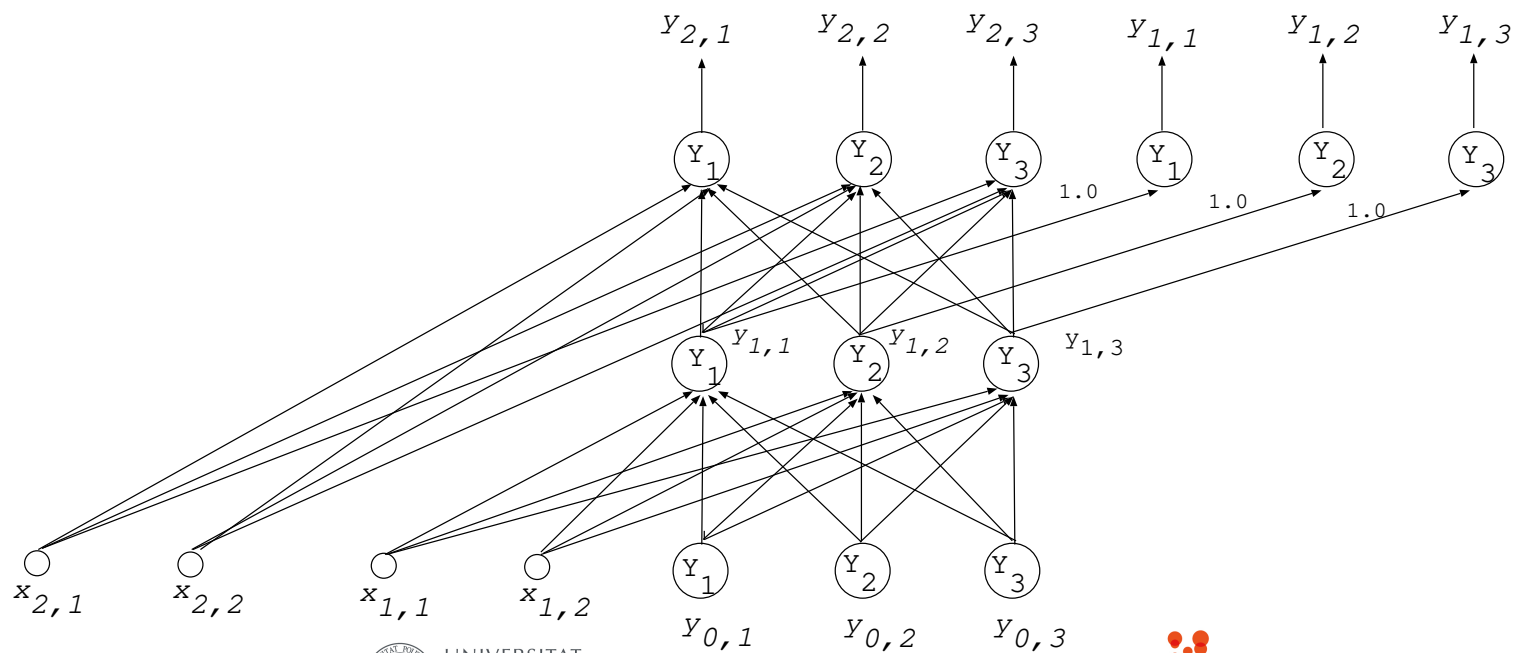
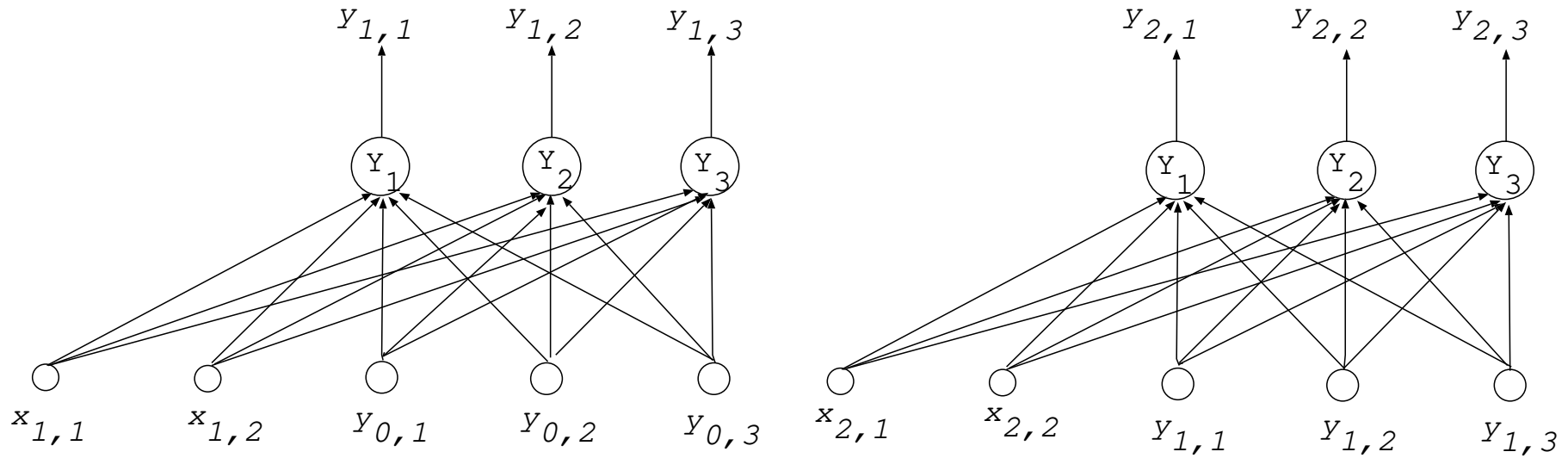
## Back-propagation through time algorithm

# Back-propagation through time algorithm (BPTT)

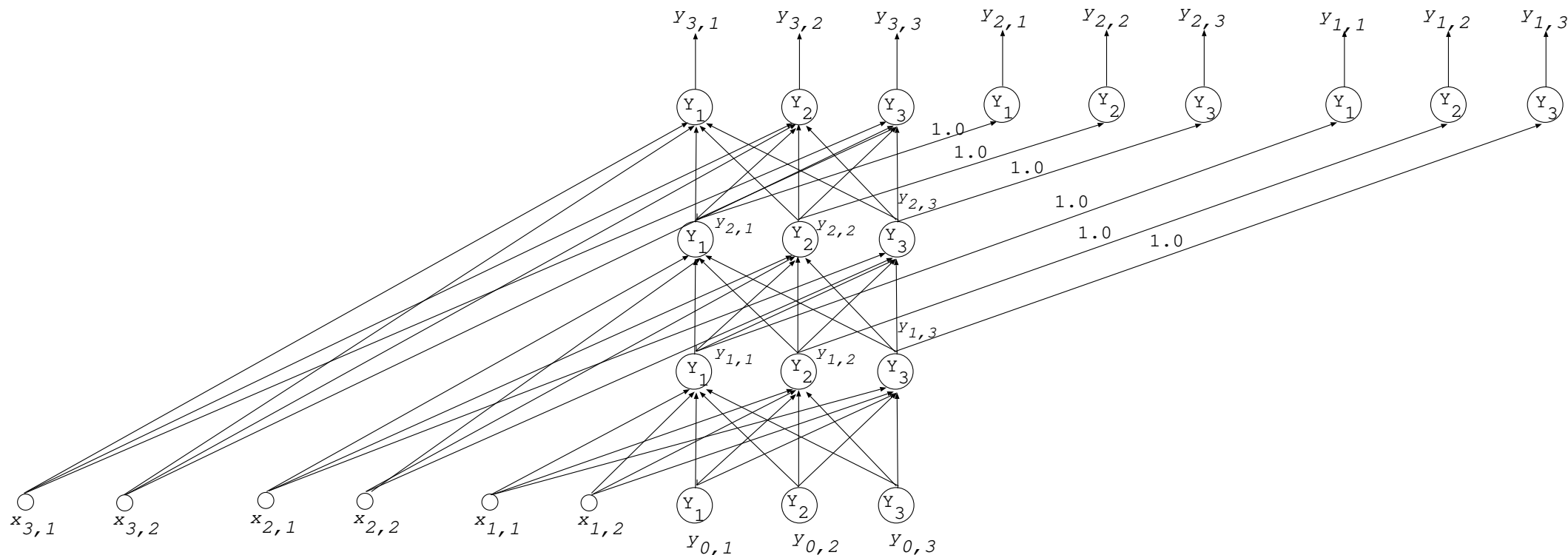




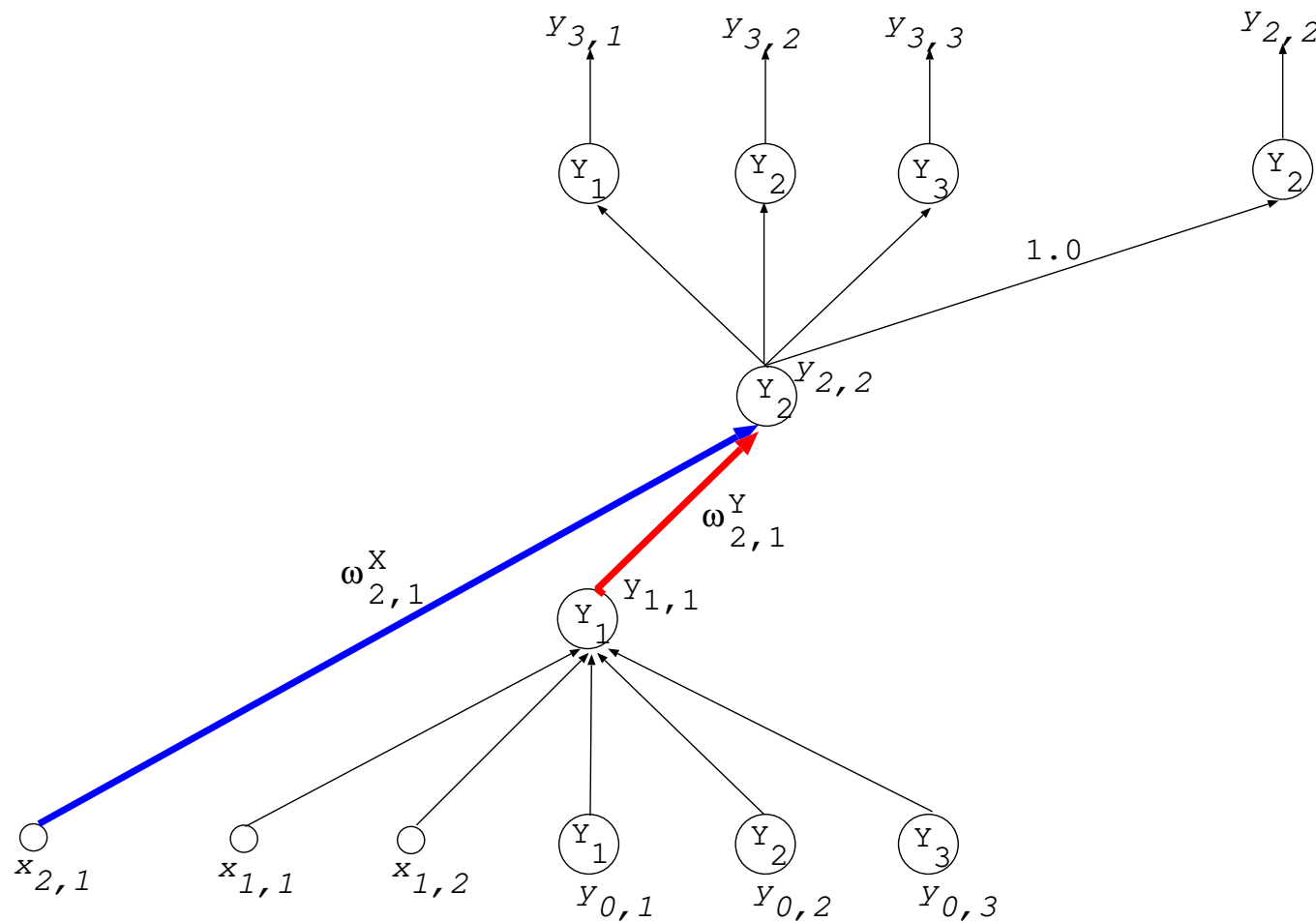
# Back-propagation through time algorithm (BPTT)



# Back-propagation through time algorithm (BPTT)



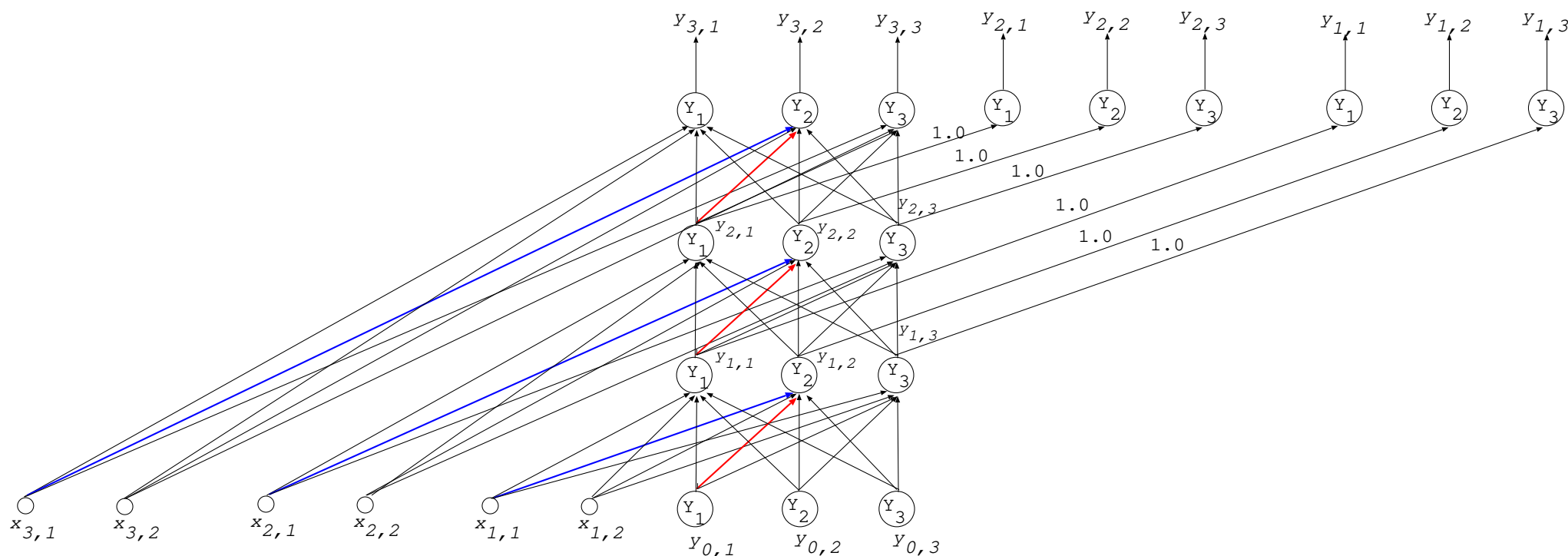
# Back-propagation through time algorithm (BPTT)



$$\Delta_2 \omega_{2,1}^X = \rho e_{2,2}^b x_{2,1}$$

$$\Delta_2 \omega_{2,1}^Y = \rho e_{2,2}^b y_{1,1}$$

# Back-propagation through time algorithm (BPTT)



$$\Delta \omega_{2,1}^X = \Delta_1 \omega_{2,1}^X + \Delta_2 \omega_{2,1}^X + \Delta_3 \omega_{2,1}^X$$

$$\Delta \omega_{21}^Y = \Delta_1 \omega_{2,1}^Y + \Delta_2 \omega_{2,1}^Y + \Delta_3 \omega_{2,1}^Y$$

# Back-propagation through time algorithm (BPTT)

- From a *training sequence of vector pairs*  $(\mathbf{x}_n, \mathbf{t}_n)_{n=1,\dots,N} : \mathbf{x}_n \in \mathbb{R}^{d_X}$  and  $\mathbf{t}_n \in \mathbb{R}^{d_Y}$  to a *training pair*:  $([\mathbf{x}_1; \dots; \mathbf{x}_N][\mathbf{t}_1; \dots; \mathbf{t}_N])$
- *Feed-forward neural networks with tied weights* with
  - For  $n = 1, \dots, N$  and  $k \in Y$ :
    - \*  $y_{n,k} = f\left(\sum_{l \in Y} \omega_{k,l}^Y y_{n-1,l} + \sum_{l \in X} \omega_{k,l}^X x_{n,l}\right)$
    - \*  $e_{n,k} = t_{n,k} - y_{n,k}$

# Back-propagation through time algorithm (BPTT)

- **Weights between two units:**  $\Delta \omega_{i,j}^Y = \sum_{n=1}^N \Delta_n \omega_{i,j}^Y = \sum_{n=1}^N \rho e_{n,i}^b y_{n-1,j}$
- **Weights from inputs to units:**  $\Delta \omega_{i,j}^X = \sum_{n=1}^N \Delta \omega_{i,j}^X(n) = \sum_{n=1}^N \rho e_{n,i}^b x_{n,j}$
- **Error propagation**

$$e_{n,i}^b = \begin{cases} \left[ \left( \sum_l e_{n+1,l}^b \omega_{l,i}^Y \right) + e_{n,i} \right] f'(h_{n,i}) & \text{for } 1 \leq n < N \\ e_{N,i} f'(h_{N,i}) & \text{for } n = N \end{cases}$$

- **Computational cost:** Temporal:  $O(C N)$  and spatial:  $O(N d_Y)$

## Other algorithms

- *Batch training or epochwise*: weights are updated at the end of the training sequence.
- *Truncated gradient algorithm*: heuristic simplification of the exact gradient algorithm.
- *Truncated algorithm or minibatch-based*: weights are updated at the end of a fixed number of time steps.
- *Schimidhuber algorithm*: combination of the truncated BPTT and exact gradient algorithm.
- *Incremental (On-line) training or real-time recurrent learning*: weights are updated at each  $n$ .
- *Momentum-based learning*.
- *Backward-forward algorithm*.

## Other algorithms (learning rate control)

- **SDG** (stochastic gradient descent).
  - **SGD with momentum**
  - **Adagrad** (Adaptive Gradient)
  - **Adadelta** (an extension of Adagrad)
  - **Adam** (Adaptive Moment Estimation)
  - NAG, RMSProp, AdaMax, Nadam, ...
- 
- Implementation through computational graphs in TensorFlow, PyTorch, ... (Koehn 2020)



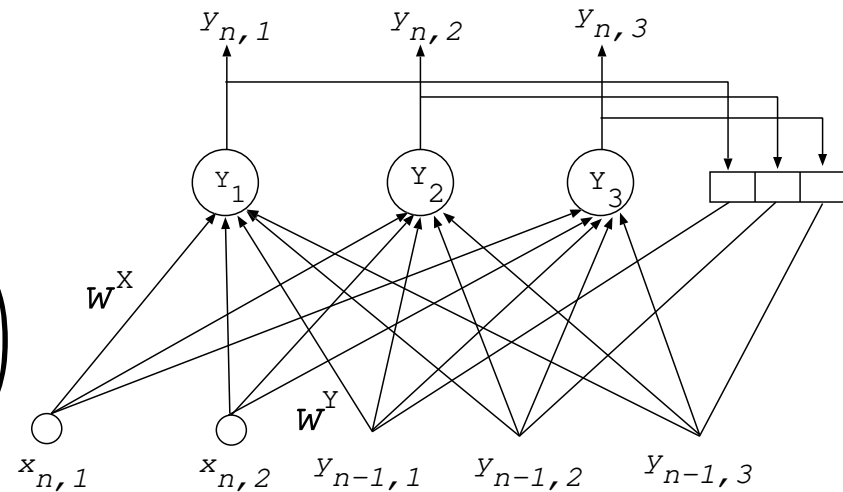
# Other recurrent neural networks

- Second-order recurrent neural networks
- Nonlinear AutoRegressive models with eXogenous inputs (NARX)
- Long Short-Term Memory (LSTM)
- Gated Recurrent Units (GRU)

## Second-order recurrent neural networks

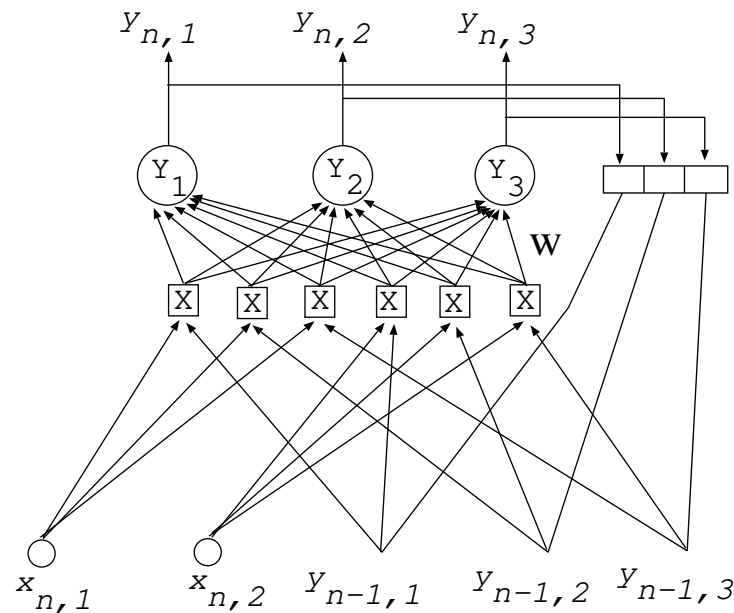
$$y_{0,k} = 0$$

$$y_{n,k} = f \left( \sum_{l \in Y} \omega_{k,l}^Y y_{n-1,l} + \sum_{l \in X} \omega_{k,l}^X x_{n,l} \right)$$

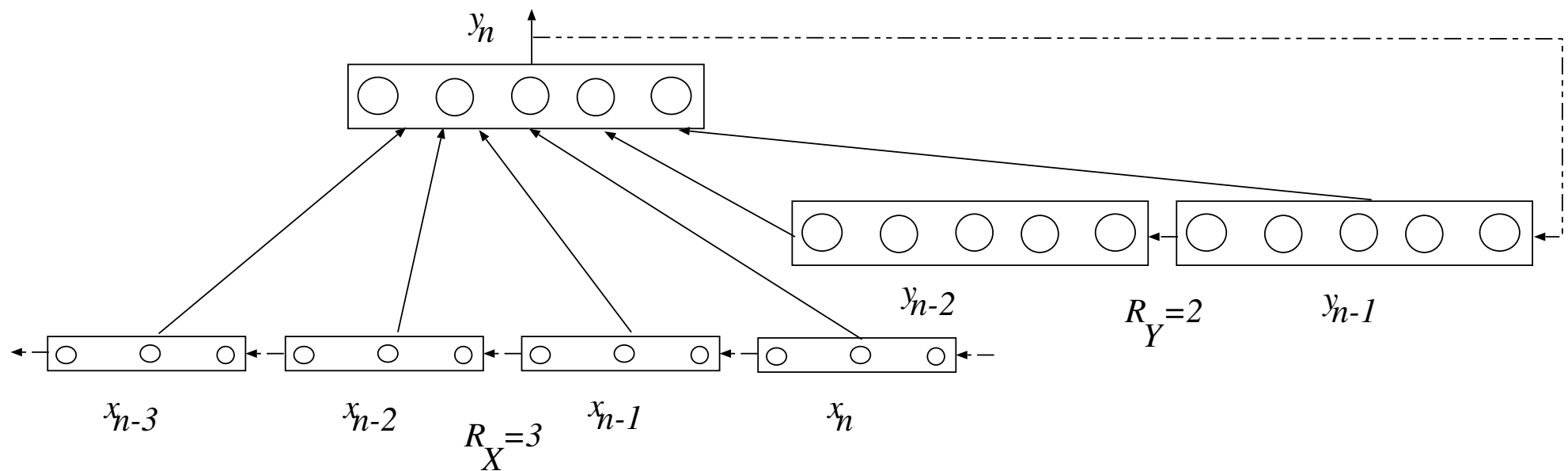


$$y_{0,k} = 0$$

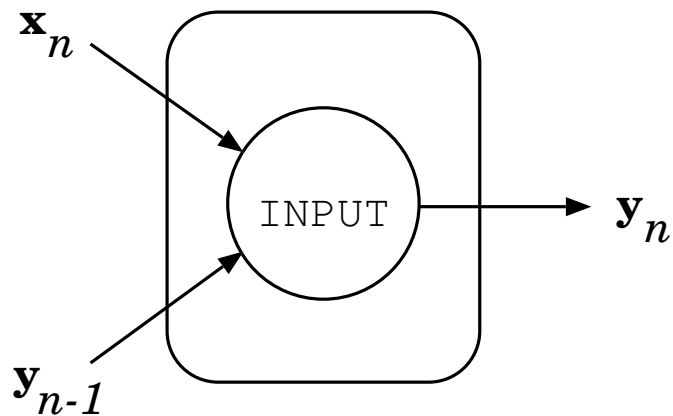
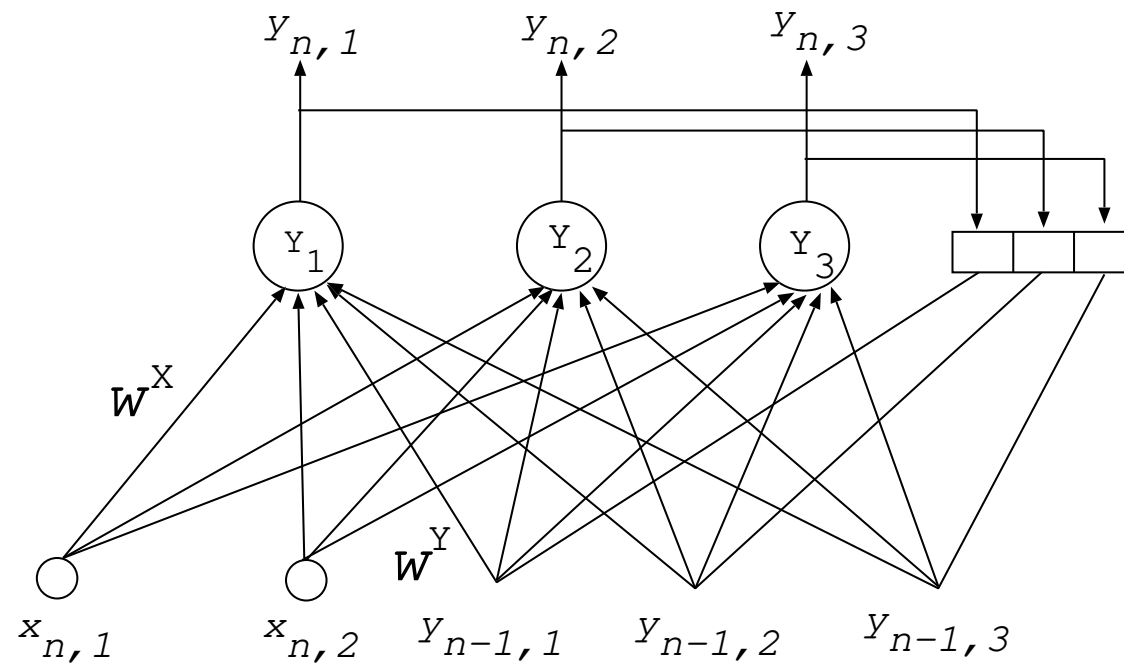
$$y_{n,k} = f \left( \sum_{l \in X} \sum_{l \in Y} \omega_{k,l,j} \ y_{n-1,l} \ x_{n,j} \right)$$



## Other simple recurrent network: NARX



# Simple units



- $y_n = f(\mathbf{W}_X \mathbf{x}_n + \mathbf{W}_Y \mathbf{y}_{n-1})$

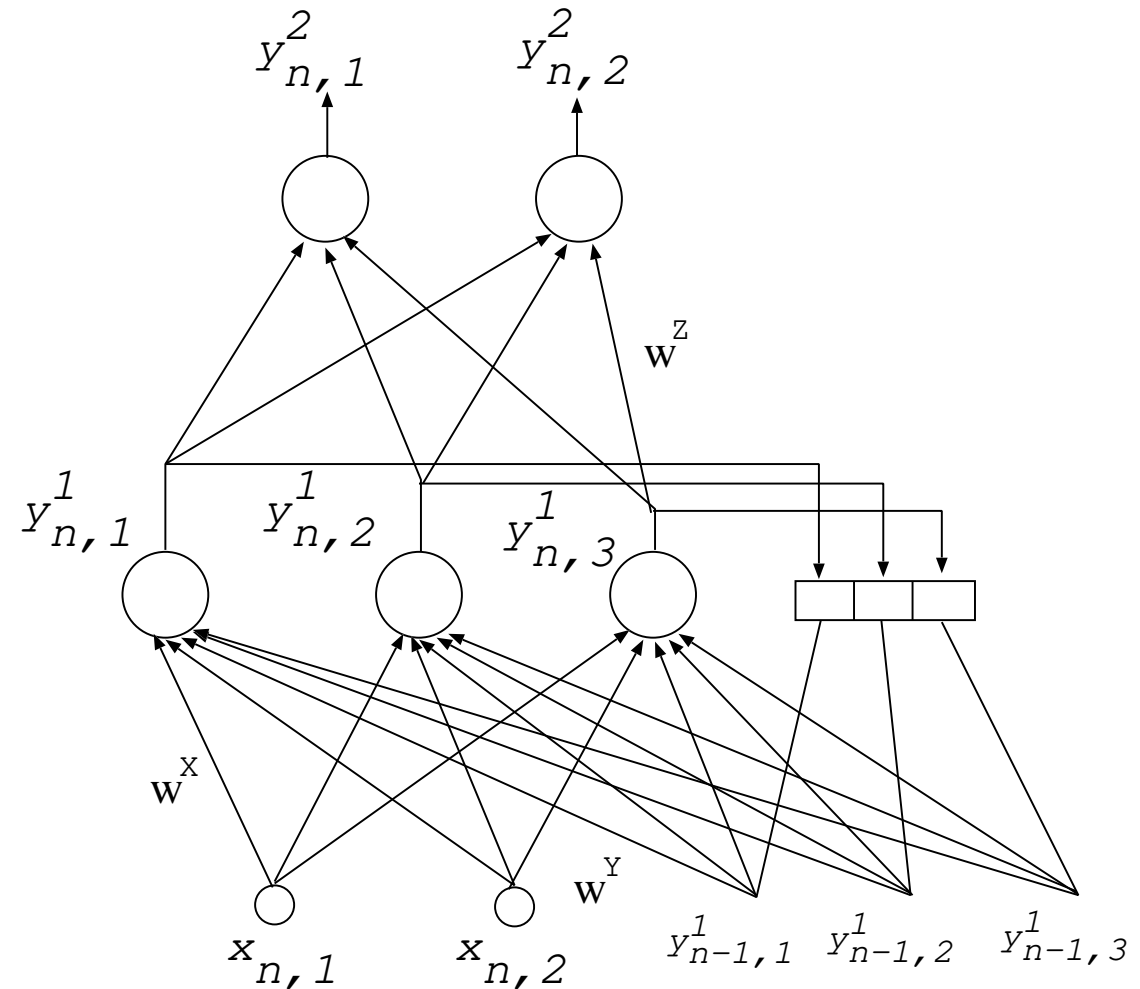
# Index

- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 *Augmented recurrent neural networks* ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Augmented recurrent neural networks

An augmented recurrent-neural network is a composition of a simple recurrent-neural network and a feed-forward neural network.

An **Elman network**: the feed-forward neural network has only one layer:  $d_X$  input units,  $d_{Y1}$  hidden units and  $d_{Y2}$  output units.



# Augmented recurrent neural networks

- For the *hidden layer* ( $1 \leq k \leq d_{Y1}$ )

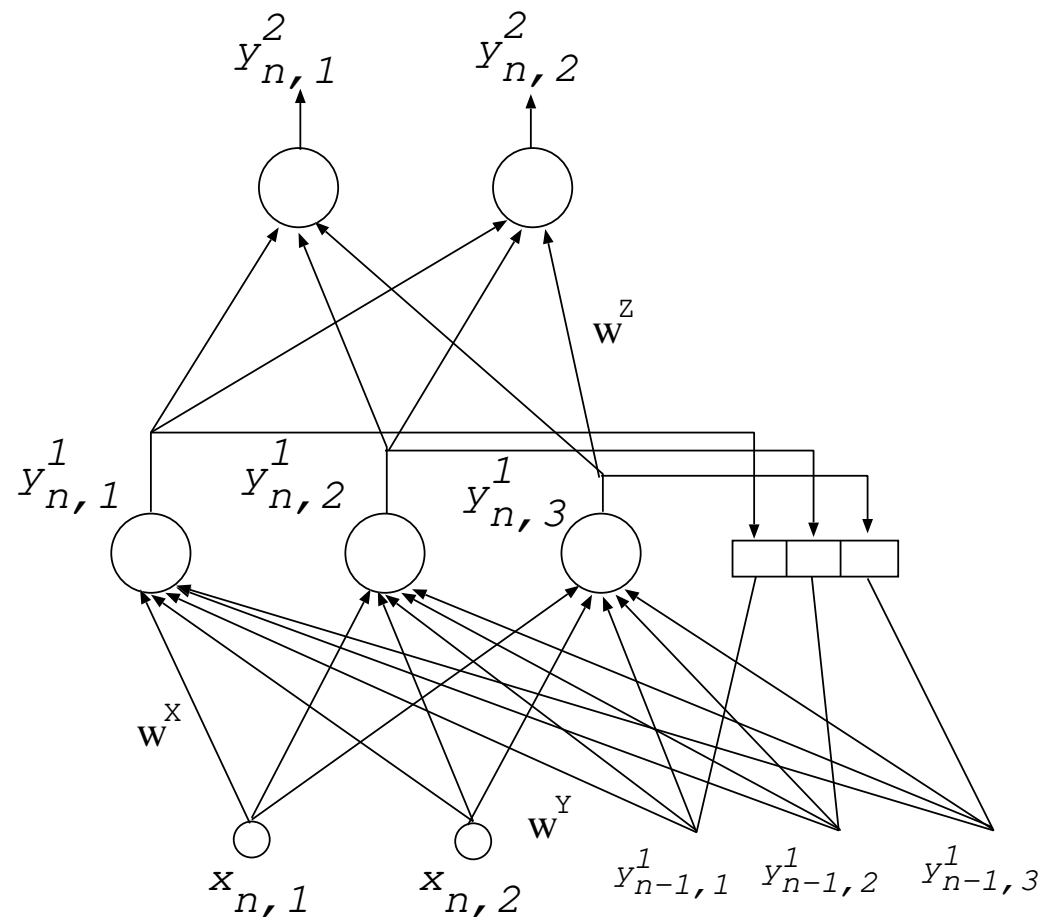
$$y_{0,k}^1 = 0$$

$$y_{n,k}^1 = f(h_{n,k}^1) =$$

$$f \left( \sum_{l=1}^{d_X} \omega_{k,l}^X x_{n,l} + \sum_{l=1}^{d_{Y1}} \omega_{k,l}^Y y_{n-1,l}^1 \right)$$

- For the *output layer* ( $1 \leq p \leq d_{Y2}$ )

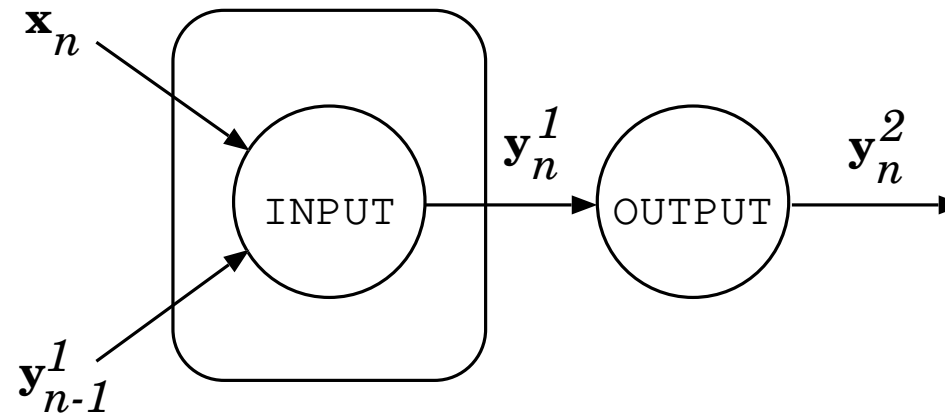
$$y_{n,p}^2 = f(h_{n,p}^2) = f \left( \sum_{l=1}^{d_{Y1}} \omega_{p,l}^Z y_{n,l}^1 \right)$$



# Augmented recurrent neural networks

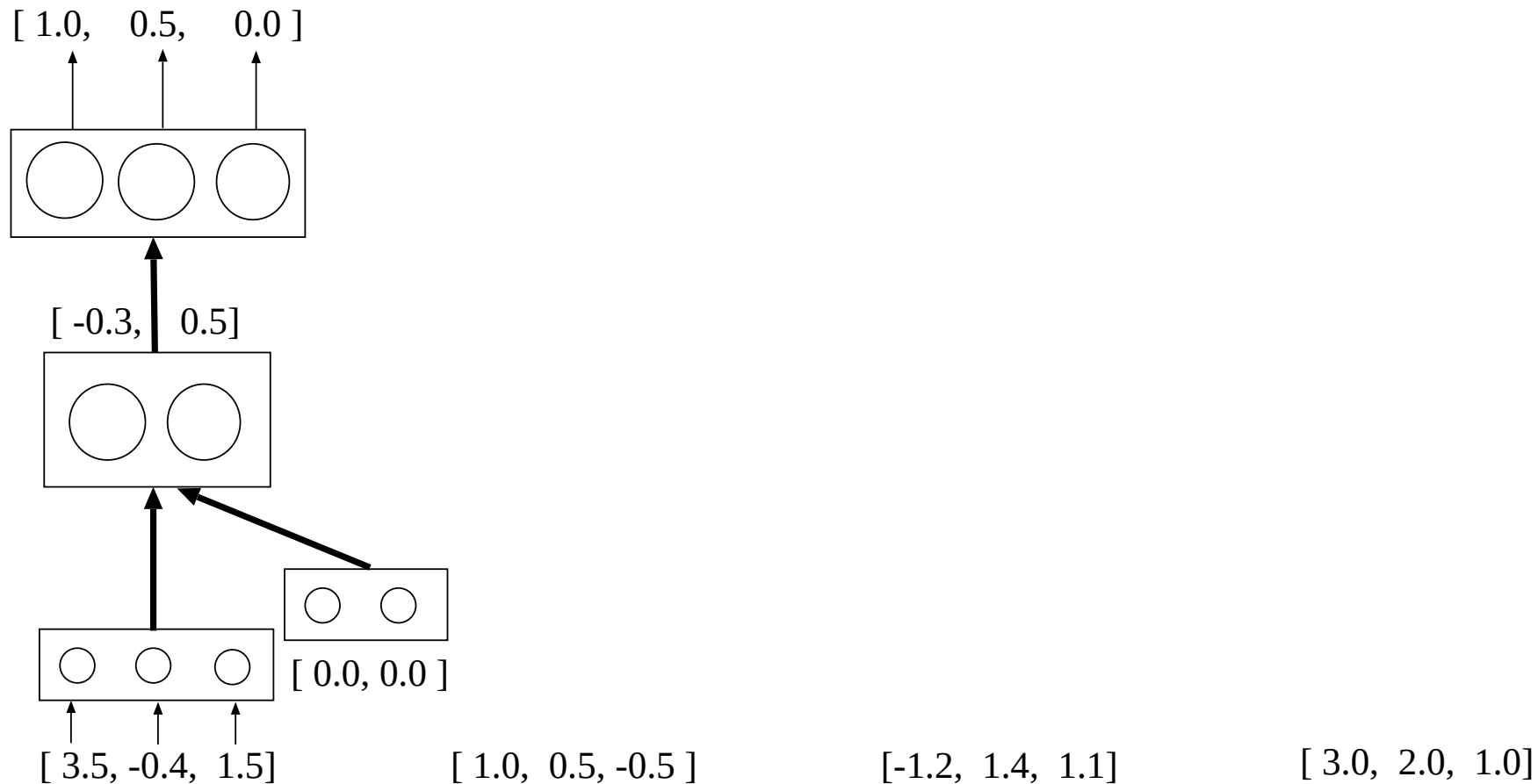
In a compact notation:

$$\mathbf{y}_n^2 = \mathbf{f}(\mathbf{W}_Z \mathbf{y}_n^1) = \mathbf{f}(\mathbf{W}_Z \mathbf{f}(\mathbf{W}_X \mathbf{x}_n + \mathbf{W}_Y \mathbf{y}_{n-1}^1))$$

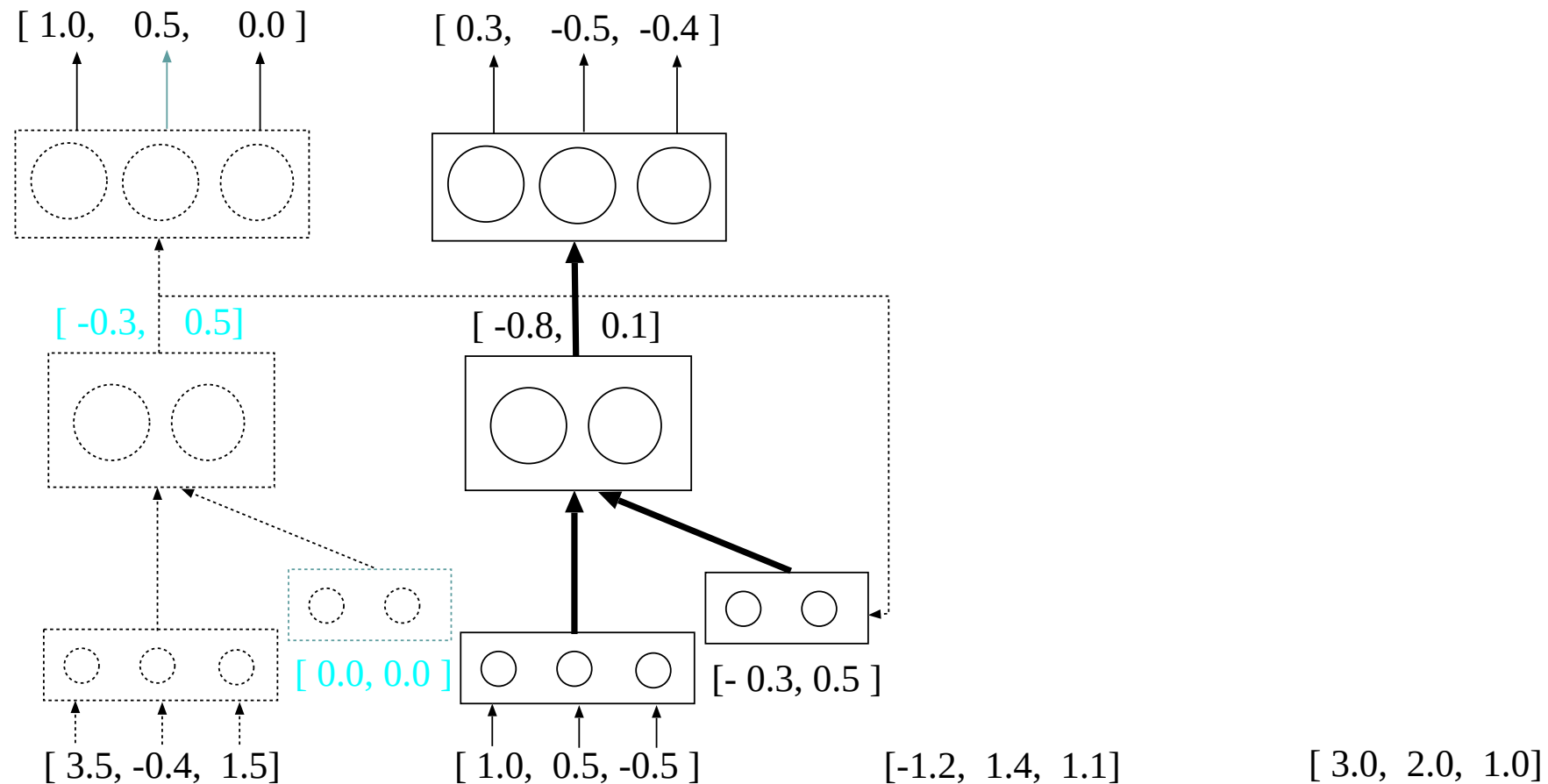




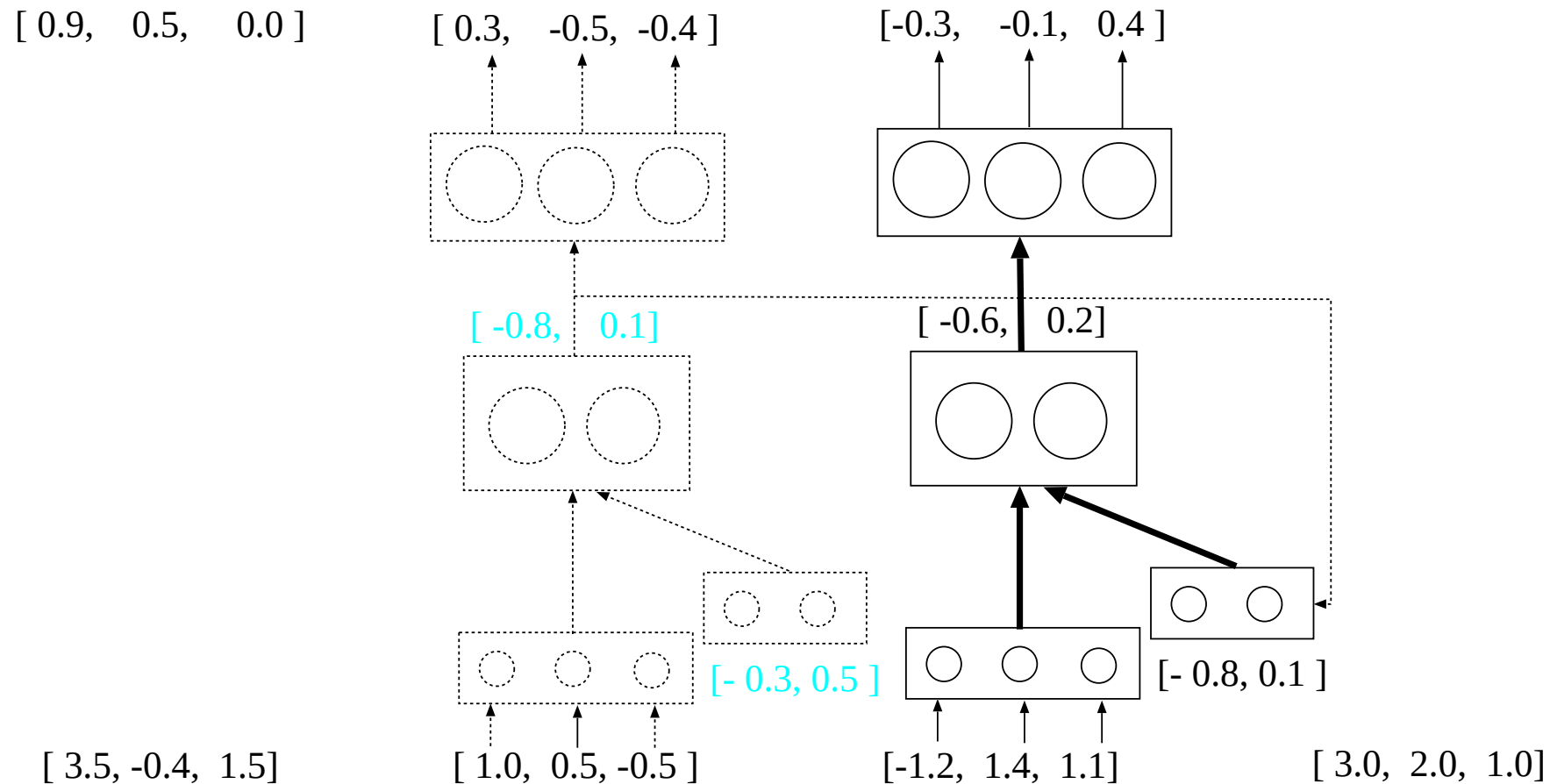
# Sequence processing: augmented recurrent neural networks



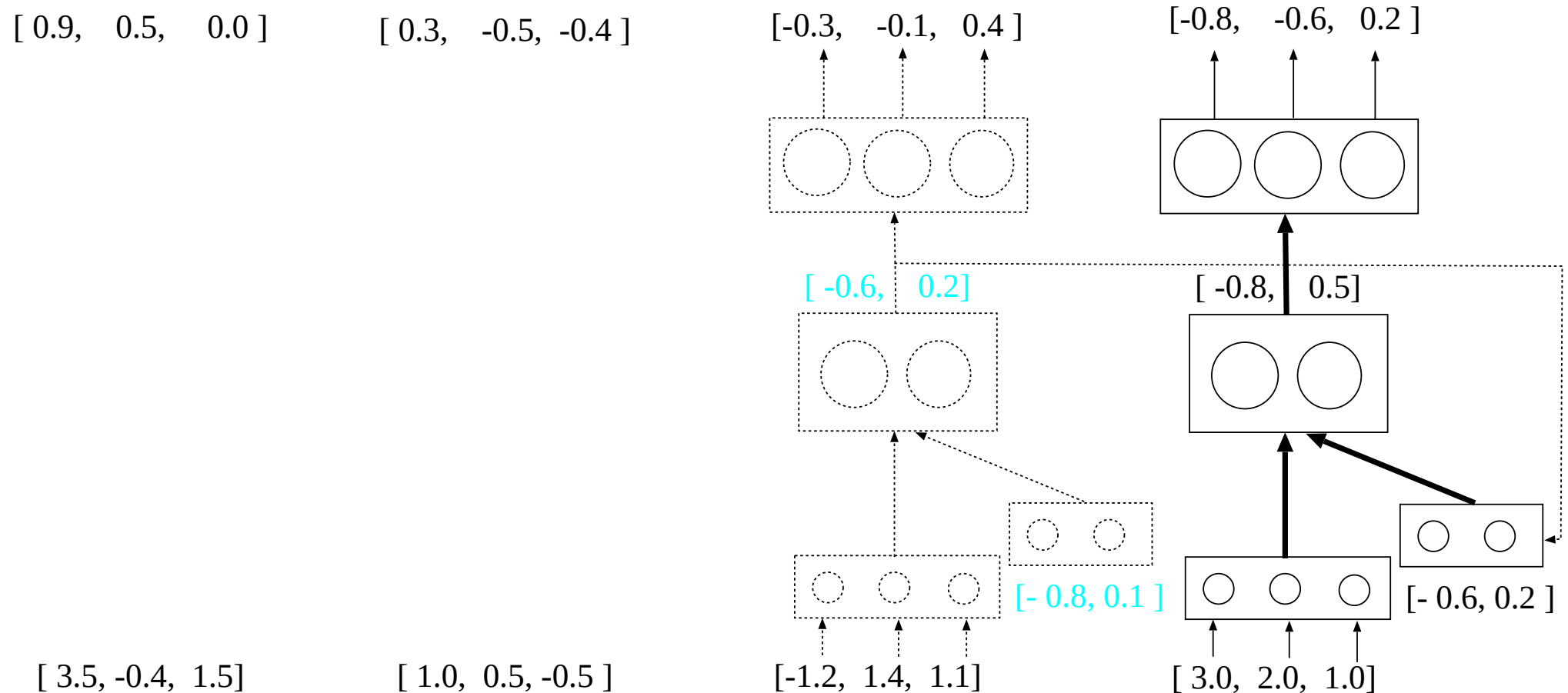
# Sequence processing: augmented recurrent neural networks



# Sequence processing: augmented recurrent neural networks



# Sequence processing: augmented recurrent neural networks



# Training augmented recurrent neural networks (regression)

- *A training sequence*  $A = (\mathbf{x}_n, \mathbf{t}_n)_{n=1,\dots,N} : \mathbf{x}_n \in \mathbb{R}^{d_X} \text{ y } \mathbf{t}_n \in \mathbb{R}^{d_{Y^2}}$
- The *error in unit  $k$  in  $n$*  is:  $e_{n,k}^2 = t_{n,k} - y_{n,k}^2$  for  $1 \leq n \leq N$  and  $k \in Y^2$
- The *error between 1 and  $N$*  is

$$\mathcal{E}_A(\mathbf{w}) = \sum_{n=1}^N \frac{1}{2} \sum_{k \in Y} (e_{n,k}^2)^2$$

- Search for a (local) minimum of  $\mathcal{E}_A$ : GRADIENT DESCENT

$$\Delta \omega_{i,j} = -\rho \frac{\partial \mathcal{E}_A(\mathbf{w})}{\partial \omega_{i,j}} \quad i \in Y^1, j \in X \cup Y^1 \text{ and } i \in Y^2, j \in Y^1$$

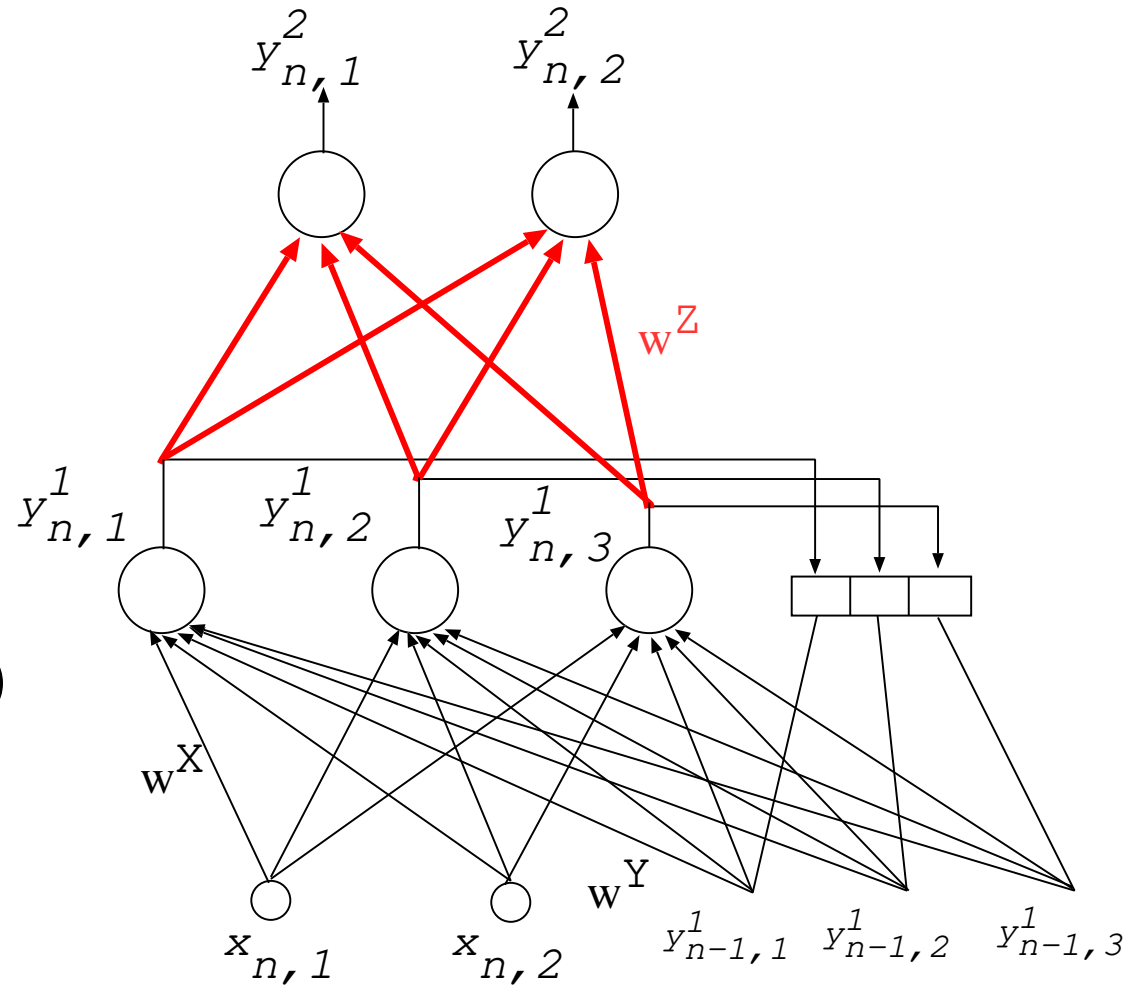
# Forward-gradient based algorithm

- Weights for the output layer**

$$1 \leq p \leq d_{Y^2} \wedge 1 \leq l \leq d_{Y^1}$$

$$\Delta \omega_{p,l}^Z = \rho \sum_{n=1}^N e_{n,p}^2 y_{n,l}^1$$

$$e_{n,p}^2 = (t_{n,p} - y_{n,p}^2) f'(h_{n,p}^2)$$



# Forward-gradient based algorithm

- Weights of recurrent connections**

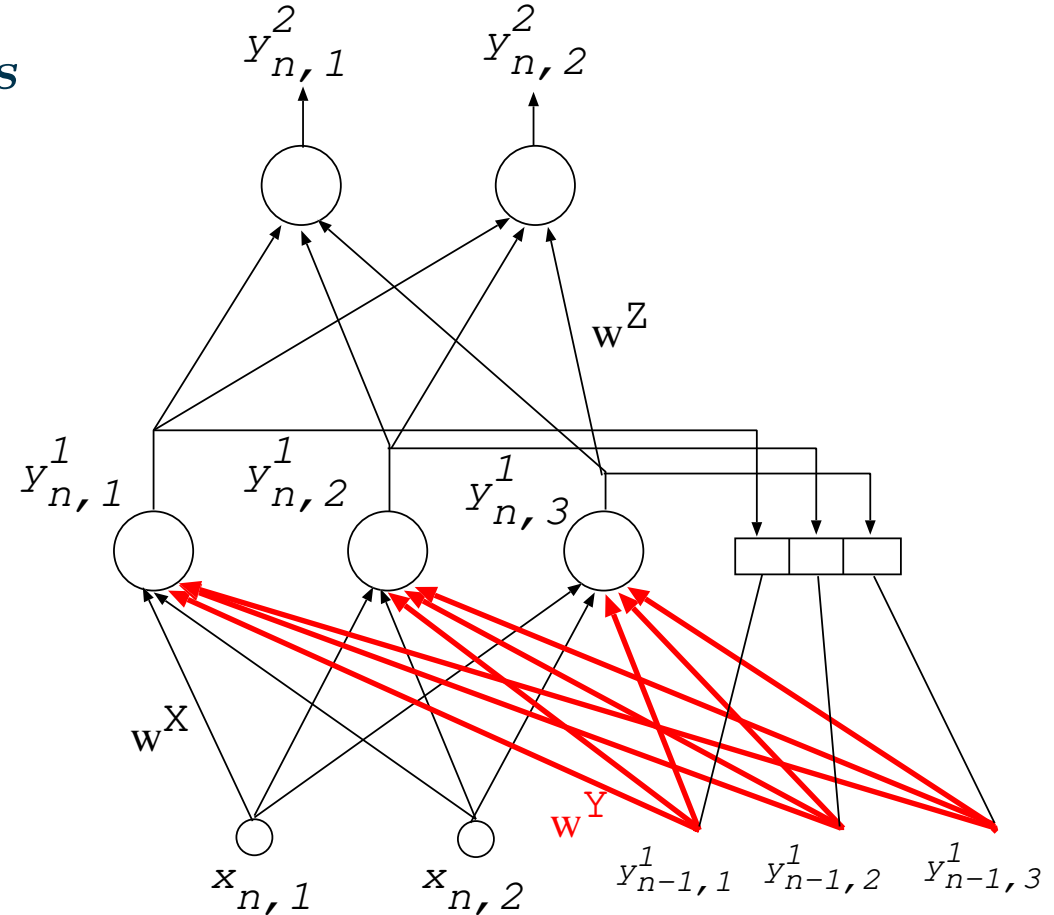
$$1 \leq k, l \leq d_{Y1}$$

$$\Delta \omega_{k,l}^Y = \rho \sum_{n=1}^N \sum_{q=1}^{d_{Y1}} e_{n,q}^1 \frac{\partial y_{n,q}^1}{\partial \omega_{k,l}^Y}$$

$$e_{n,q}^1 = \sum_p e_{n,p}^2 \omega_{p,q}^Z$$

$$\frac{\partial y_{0,q}^1}{\partial \omega_{k,l}^Y} = 0$$

$$\frac{\partial y_{n,q}^1}{\partial \omega_{k,l}^Y} = f'(h_{n,q}^1) \left( \delta_{k,q} y_{n-1,l}^1 + \sum_{r=1}^{d_{Y1}} \omega_{q,r}^Y \frac{\partial y_{n-1,r}^1}{\partial \omega_{k,l}^Y} \right)$$



# Forward-gradient based algorithm

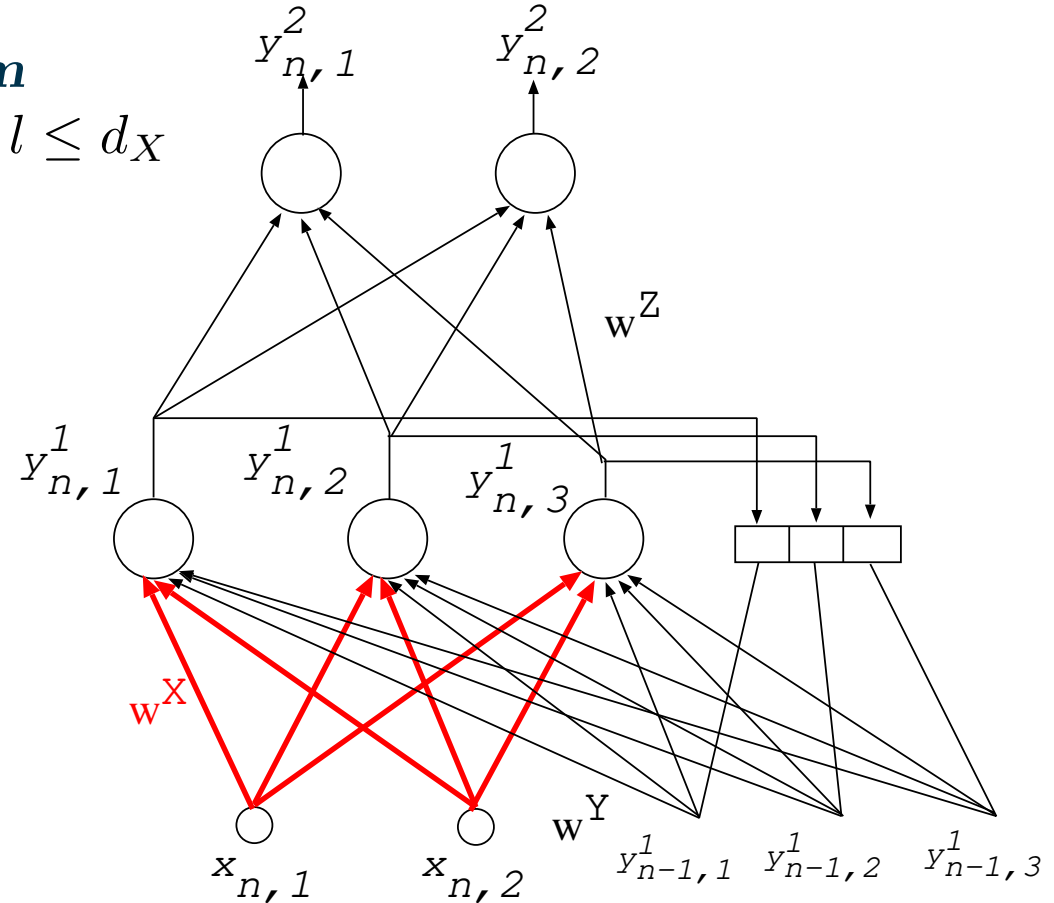
- **Weights for the connections from the input layer.**  $1 \leq k \leq d_{Y^1} \wedge 1 \leq l \leq d_X$

$$\Delta \omega_{k,l}^X = \rho \sum_{n=1}^N \sum_{r=1}^{d_{Y^1}} e_{n,r}^1 \frac{\partial y_{n,r}^1}{\partial \omega_{k,l}^X}$$

$$e_{n,r}^1 = \sum_p e_{n,p}^2 \omega_{p,r}^Z$$

$$\frac{\partial y_{0,r}^1}{\partial \omega_{k,l}^X} = 0$$

$$\frac{\partial y_{n,r}^1}{\partial \omega_{k,l}^X} = f'(h_{n,r}^1) \left( \delta_{k,r} x_{n,l} + \sum_{q=1}^{d_{Y^1}} \omega_{q,r}^X \frac{\partial y_{n-1,q}^1}{\partial \omega_{k,l}^X} \right)$$

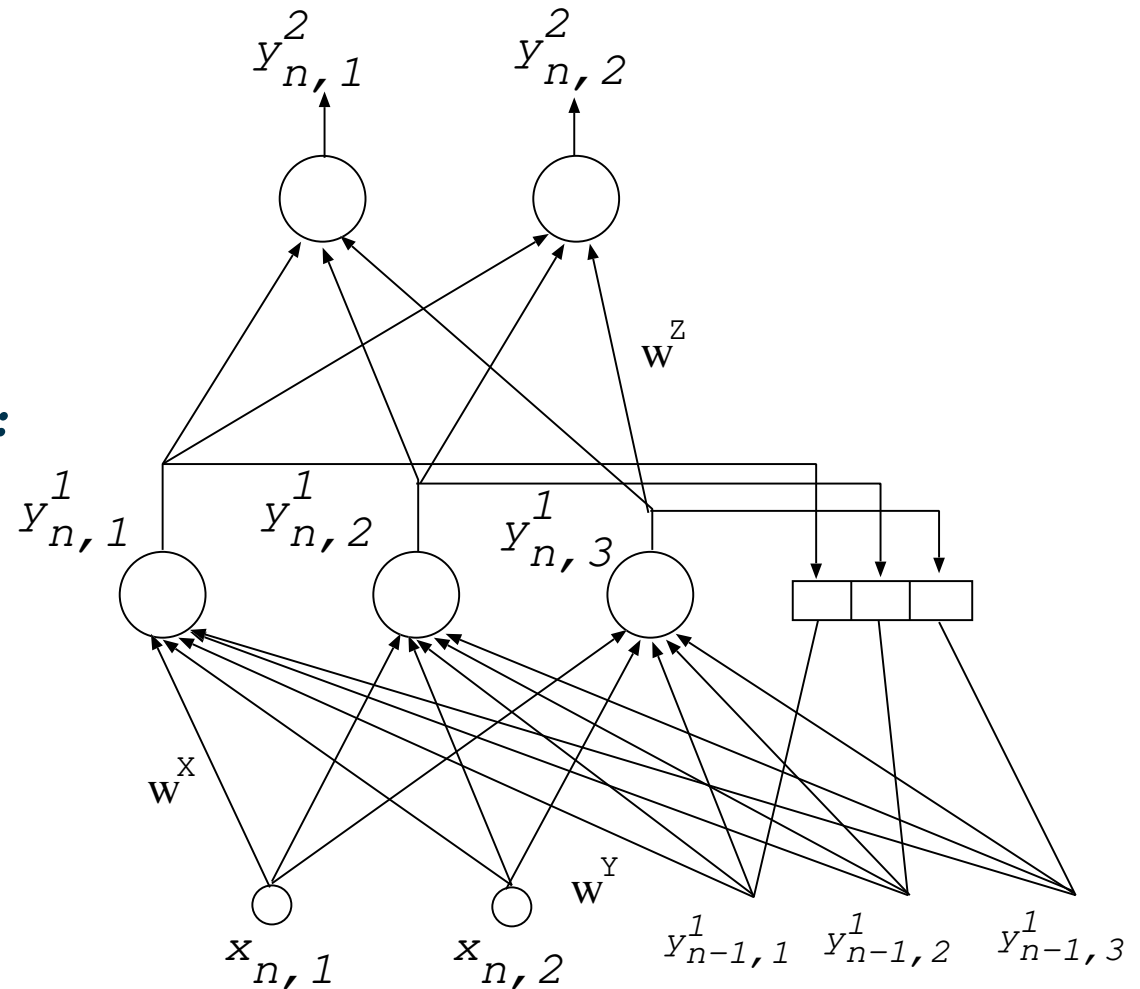




# Forward-gradient based algorithm

*Temporal computational cost:*

$$O(((C_{Y^1} + C_X) d_H^2 + C_{Y^2}) L)$$

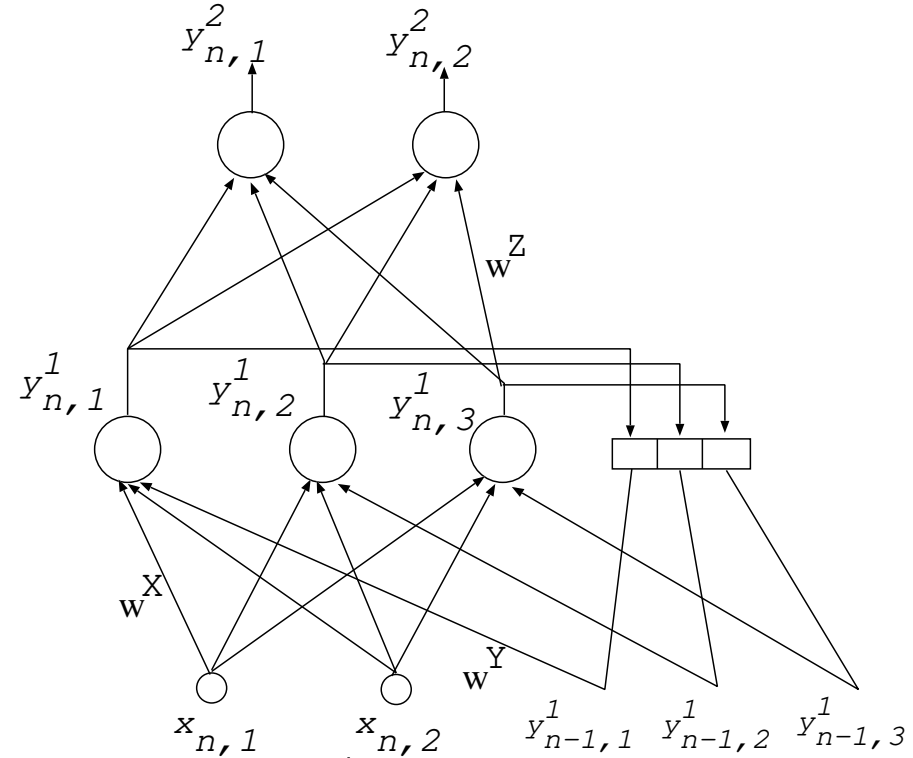


# Other algorithms

- Truncate-gradient algorithm
- Back-propagation through time
- Incremental training
- ...

# Simplified augmented recurrent neural networks

An augmented recurrent network such that each unit has a self-recurrent connections.



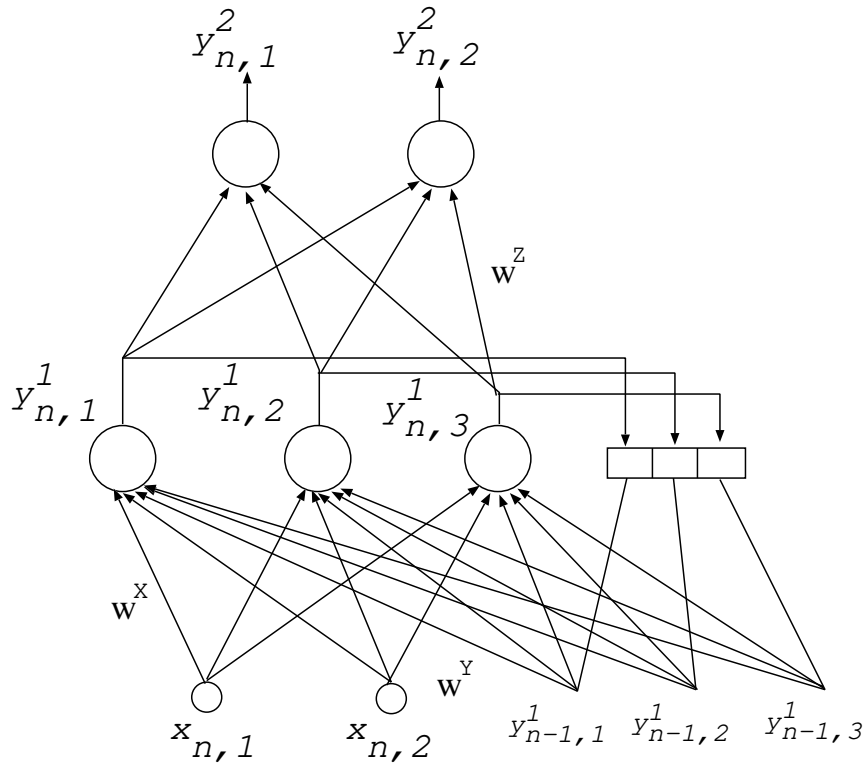
- The *hidden layer*

$$y_{n,k}^1 = f(h_{n,k}^1) = f \left( \sum_{l=1}^{d_X} \omega_{k,l}^X x_{n,l} + \omega_{k,k}^Y y_{n-1,k}^1 \right) \text{ with } 1 \leq k \leq d_{Y1}$$

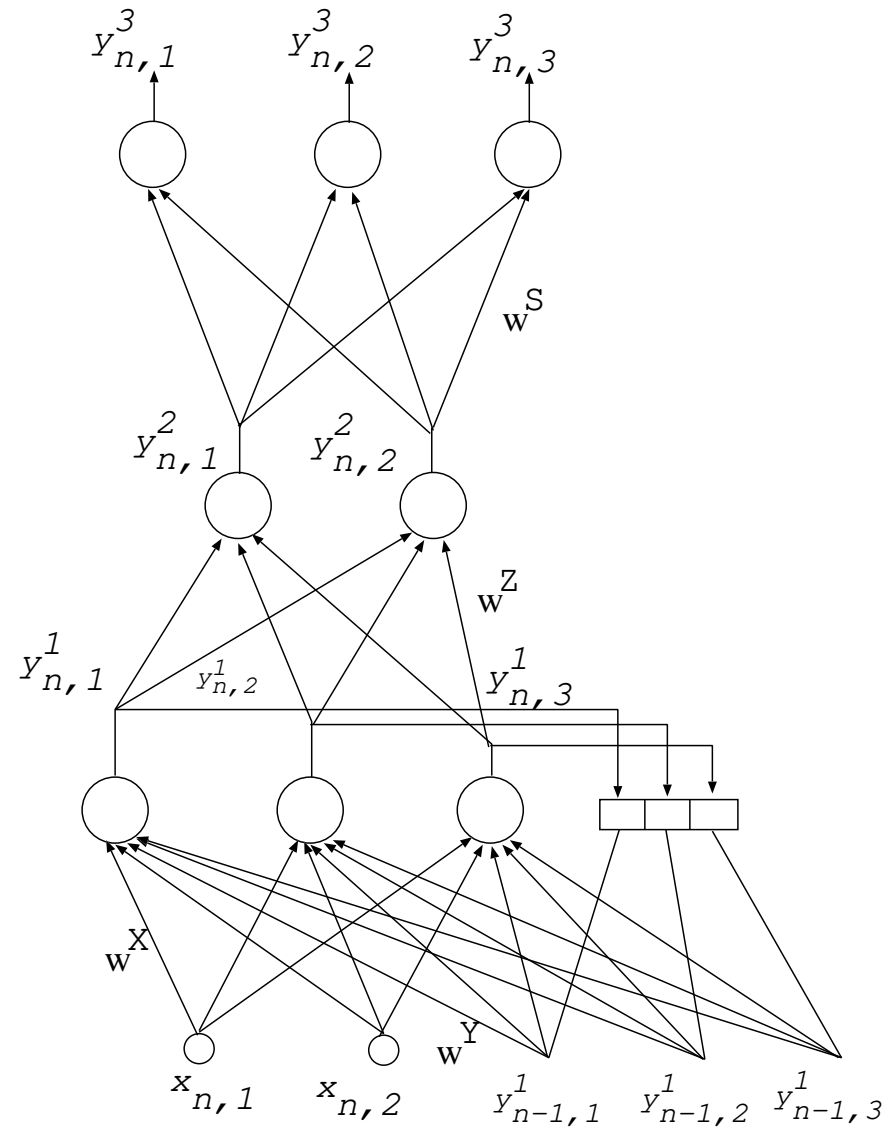
- The *output layer*

$$y_{n,p}^2 = f(h_{n,p}^2) = f \left( \sum_{l=1}^{d_{Y1}} \omega_{p,l}^Z y_{n,l}^1 \right) \text{ with } 1 \leq p \leq d_{Y2}$$

# Generalization

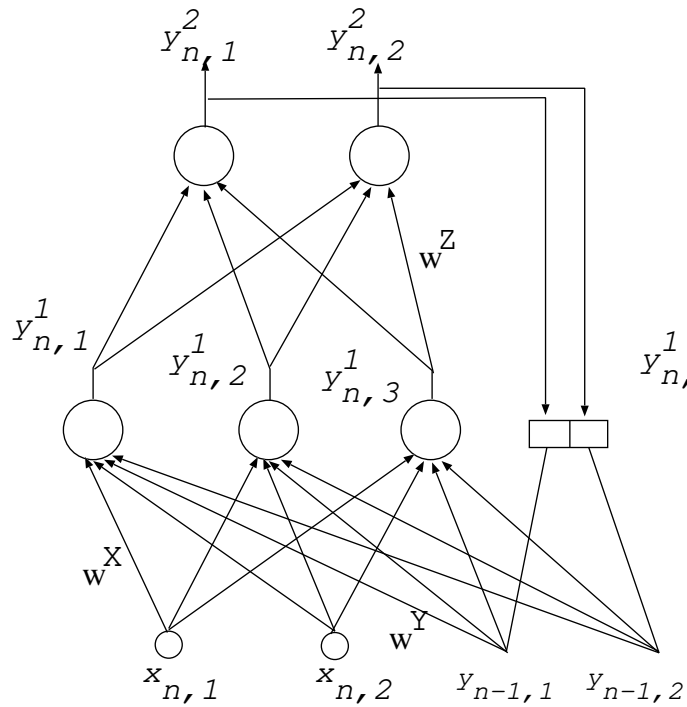


Two-layer recurrent neural network  
(Elman network)

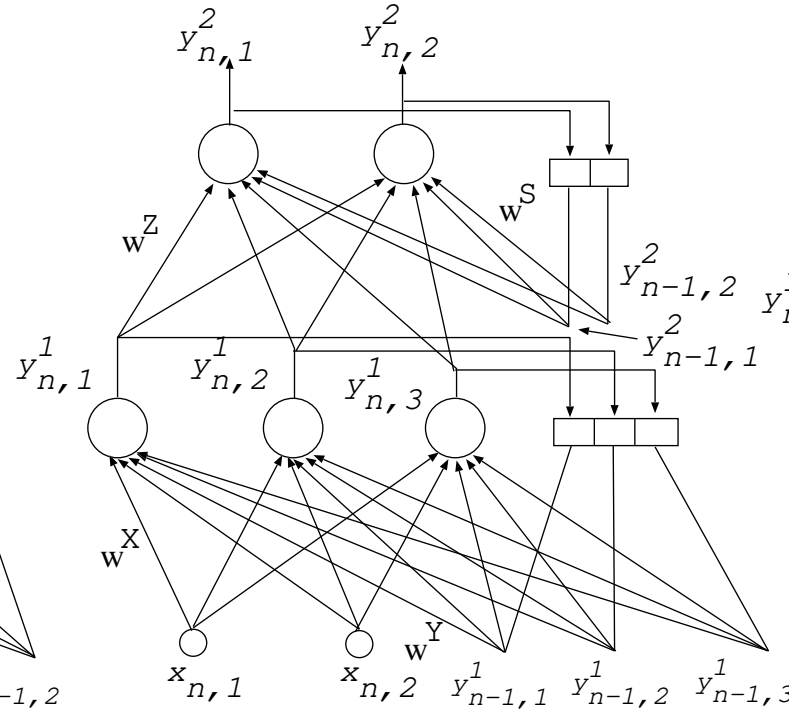


Three-layer recurrent neural network

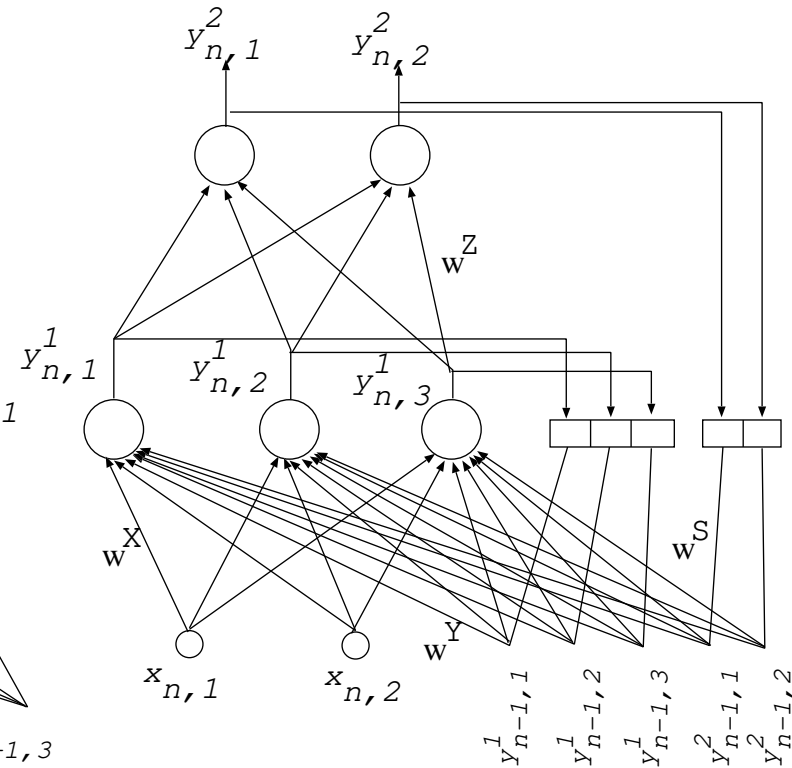
# Generalization



Jordan network



Hybrid network



Hybrid network

## Some computational results

- A first-order simple recurrent neural network cannot implement any finite-state machine (Goudreau et al., 1995).
- Any finite-state machine can be simulated using an Elman network.
- Every Turing machine can be simulated by a second-order simple recurrent neural network.

# Index

- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 *Long short-term memory* ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Long Short-Term Memory (LSTM) [Hochreiter 1997]

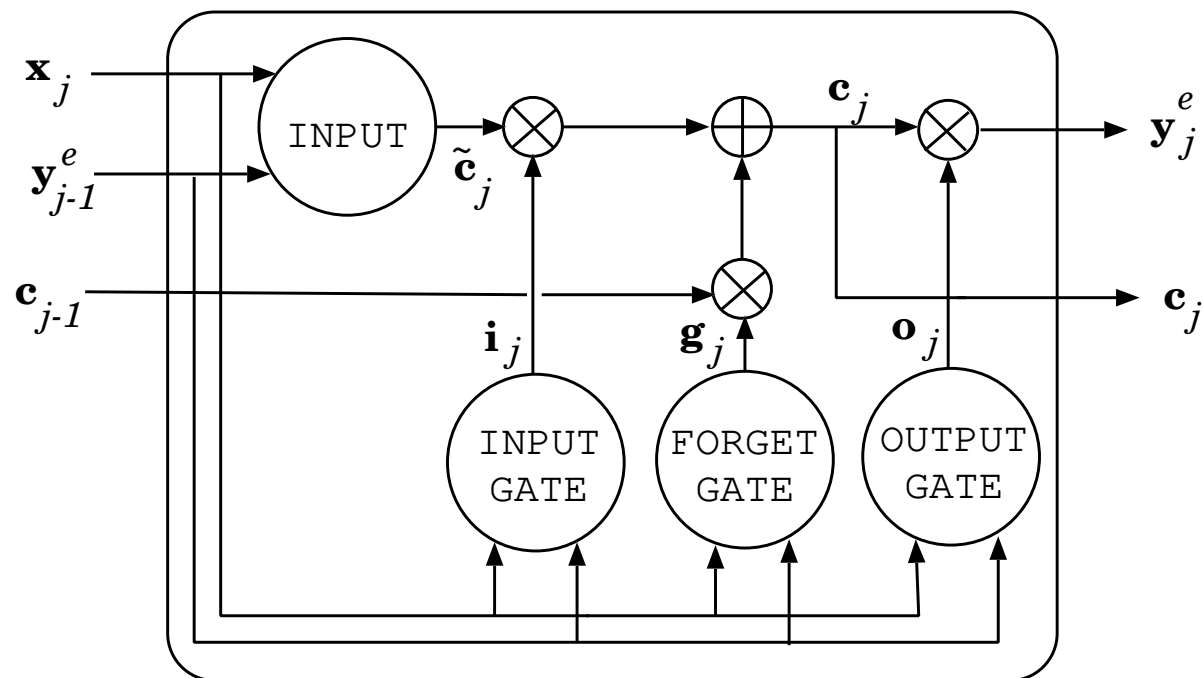
Problems with recurrent neural networks:

- The output depends on the complete past information and sometimes only recent information is needed and sometimes more far context is needed
- In back-propagation through time algorithm and exact gradient or real-time recurrent learning algorithms the errors that propagated backwards in time tend to vanish or to oscillate.

A solution: Long Short-Term Memory (LSTM) or Gated Recurrent Units (GRU)



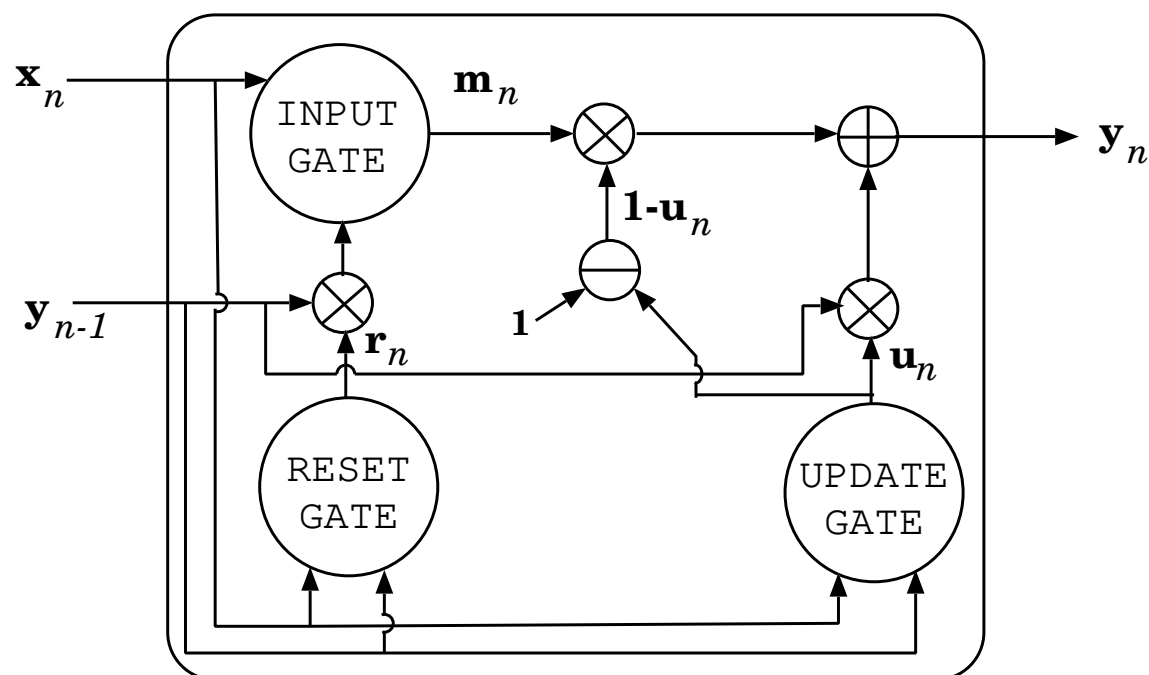
# Long Short-Term Memory (LSTM)



- $\mathbf{i}_n = \mathbf{f}_s(\mathbf{W}_Y^I \mathbf{y}_{n-1} + \mathbf{W}_X^I \mathbf{x}_n)$
- $\mathbf{g}_n = \mathbf{f}_s(\mathbf{W}_Y^F \mathbf{y}_{n-1} + \mathbf{W}_X^F \mathbf{x}_n)$
- $\mathbf{o}_n = \mathbf{f}_s(\mathbf{W}_Y^O \mathbf{y}_{n-1} + \mathbf{W}_X^O \mathbf{x}_n)$
- $\tilde{\mathbf{c}}_n = \mathbf{f}_{th}(\mathbf{W}_Y^C \mathbf{y}_{n-1} + \mathbf{W}_X^C \mathbf{x}_n)$
- $\mathbf{c}_n = \mathbf{g}_n \times \mathbf{c}_{n-1} + \mathbf{i}_n \times \tilde{\mathbf{c}}_n$
- $\mathbf{y}_n = \mathbf{o}_n \times \mathbf{f}_{th}(\mathbf{c}_n)$

$$\mathbf{y}_n = \mathbf{F}(\mathbf{x}_n, \mathbf{y}_{n-1})$$

# Gated Recurrent Units (GRU) (Cho et al. 2014)



- $\mathbf{r}_n = \mathbf{f}_s(\mathbf{W}_X^R \mathbf{x}_n + \mathbf{W}_H^R \mathbf{y}_{n-1})$
- $\mathbf{u}_n = \mathbf{f}_s(\mathbf{W}_X^U \mathbf{x}_n + \mathbf{W}_H^U \mathbf{y}_{n-1})$
- $\mathbf{m}_n = \mathbf{f}_{th}(\mathbf{W}_X^M \mathbf{x}_n + \mathbf{W}_H^M (\mathbf{r}_n \times \mathbf{y}_{n-1}))$
- $\mathbf{y}_n = \mathbf{u}_n \times \mathbf{y}_{n-1} + (1 - \mathbf{u}_n) \times \mathbf{m}_n = \mathbf{F}(\mathbf{x}_n, \mathbf{y}_{n-1})$

# Index

- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 *Connectionist temporal classification* ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Connectionist Temporal Classification (Graves 2006)

- A simple recurrent neural network with a set  $Y'$  of  $d_Y + 1$  units (each output unit  $k$  has associated a label  $l(k) \in \Sigma$  and the  $d_Y + 1$  unit is called *no label* or *blank*) and a set  $X$  of  $d_X$  inputs.
- Input:  $\mathbf{x}_n \in \mathbb{R}^{d_X}$  and output:  $\mathbf{y}_n \in \mathbb{R}^{d_Y}$  for  $1 \leq n \leq N$
- **Total input** of the unit  $k \in Y$  in “time”  $n$ :

$$h_{n,k} = \sum_{l \in U} \omega_{k,l}^Y y_{n-1,l} + \sum_{l \in I} \omega_{k,l}^X x_{n,l}$$

- The **state** of  $k \in U$  in  $n$ :

$$y_{n,k} = \begin{cases} f(h_{n,k}) & n \geq 1 \\ 0 & n = 0 \end{cases}$$

$f$  is the softmax activation function

# Connectionist Temporal Classification (Graves 2006)

- Let  $\Sigma'^N = (\Sigma \cup \{blank\})^N$  be the set of  $N$ -length sequences of labels, and  $l'_1{}^N \in \Sigma'^N$
- The state of  $k \in U$  in  $n$  is interpreted as the probability that the label of  $\mathbf{x}_n$  is  $l'(k) \in \Sigma'^N$ :  $p(l'(k) \mid \mathbf{x}_n) \equiv y_{n,k}$
- $p(l'_1{}^N \mid \mathbf{x}_1^N) = \prod_{n=1}^N p(l'_n \mid l'_1{}^{n-1}, \mathbf{x}_n) \approx \prod_{n=1}^N y_{n,\pi_n}$
- Let  $\mathcal{G} : \Sigma'^N \rightarrow \Sigma^M$  with  $M \leq N$  for removing the no-labels or the blanks. Therefore, for  $l_1^M \in \Sigma^M$

$$p(l_1^M \mid \mathbf{x}_1^N) = \sum_{l'_1{}^N : \mathcal{G}(l'_1{}^N) = l_1^M} p(l'_1{}^N \mid \mathbf{x}_1^N) \approx \max_{l'_1{}^N : \mathcal{G}(l'_1{}^N) = l_1^M} p(l'_1{}^N \mid \mathbf{x}_1^N)$$

# Connectionist Temporal Classification (Graves 2006)

- Decoding: Given a input sequence  $\mathbf{x}_1^N$ , search for a sequence of labels such that:

$$\widehat{l_1^M} = \operatorname{argmax}_{M, l_1^M, M \leq N} p(l_1^M | \mathbf{x}_1^N)$$

## Approximations

- Viterbi decoding:  $\approx \mathcal{G}(\operatorname{argmax}_{l_1^N} p(l_1^N | \mathbf{x}_1^N))$
  - Prefix search decoding: adaptation of the forward-backward algorithm
- Training: Given a **training sequence**  $A = (\mathbf{x}_1^N, t_1^M)$ , the goal function is the log-likelihood (equivalent to cross-entropy in this case):

$$\mathcal{L}_A(\mathbf{w}) = -\log(p(t_1^M | \mathbf{x}_1^N))$$

- Forward-Backward algorithm

# Index

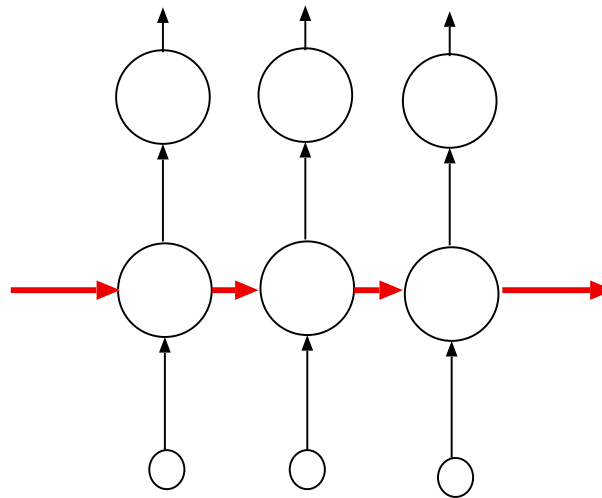
- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 *Bidirectional recurrent neural networks* ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Motivation (Schuster 1997)

- Networks with bounded-memory: dependencies with the last  $T$  inputs.
- Synchronous recurrent neural networks: dependencies with all past inputs.
- Dependencies with the future? i.e. off-line handwriting text recognition.

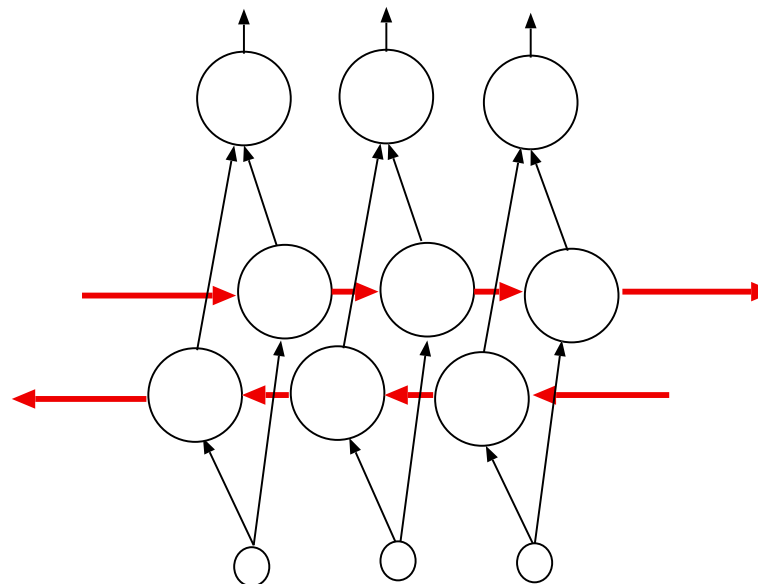


# Bidirectional recurrent neural networks



Standard dynamics

A recurrent  
neural network



Bidirectional dynamics

# Bidirectional recurrent neural networks

- **Forward** ( $1 \leq k \leq d_{Y1}$ )

$$y_{0,k}^{1f} = 0; \quad y_{n,k}^{1f} = f \left( \sum_{l=1}^{d_X} \omega_{k,l}^X x_{n,l} + \sum_{l=1}^{d_{Y1}} \omega_{k,l}^Y y_{n-1,l}^{1f} \right) \quad n = 1, \dots, N$$

- **Backward** ( $1 \leq k \leq d_{Y1}$ )

$$y_{N+1,k}^{1b} = 0; \quad y_{n,k}^{1b} = f \left( \sum_{l=1}^{d_X} \omega_{k,l}^X x_{n,l} + \sum_{l=1}^{d_{Y1}} \omega_{k,l}^Y y_{n+1,l}^{1b} \right) \quad n = N, \dots, 1$$

- The **output** ( $1 \leq p \leq d_{Y2}$ )

$$y_{n,p}^2 = f \left( \sum_{l=1}^{d_{Y1}} \omega_{p,l}^Z (y_{n,l}^{1f} + y_{n,l}^{1b}) \right) \quad n = 1, \dots, N$$

- The **output** ( $1 \leq p \leq d_{Y2}$ ) (an alternative)

$$y_{n,p}^2 = f \left( \sum_{l=1}^{d_{Y1}} \omega_{p,l}^Z [y_{n,l}^{1f}, y_{n,l}^{1b}] \right) \quad n = 1, \dots, N$$

# Bidirectional recurrent neural networks (Schuster 1997)

## Training BRNN

- BPTT

**Training forward pass:** Run all input data through the BRNN and determine all predicted outputs.

1. Do forward pass just for forward states ( $n = 1, \dots, N$ ) and backward states ( $m = N, \dots, 1$ )
2. Do forward pass for output.

**Training backward pass:** Calculate the part of the objective function derivative for the time slice used in the forward pass.

1. Do backward pass for output neurons.
2. Do backward pass just for forward states ( $n = N, \dots, 1$ ) and backward states ( $n = 1, \dots, N$ )

**Update weights**

# Index

- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 *Sequence-to-sequence network architectures* ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

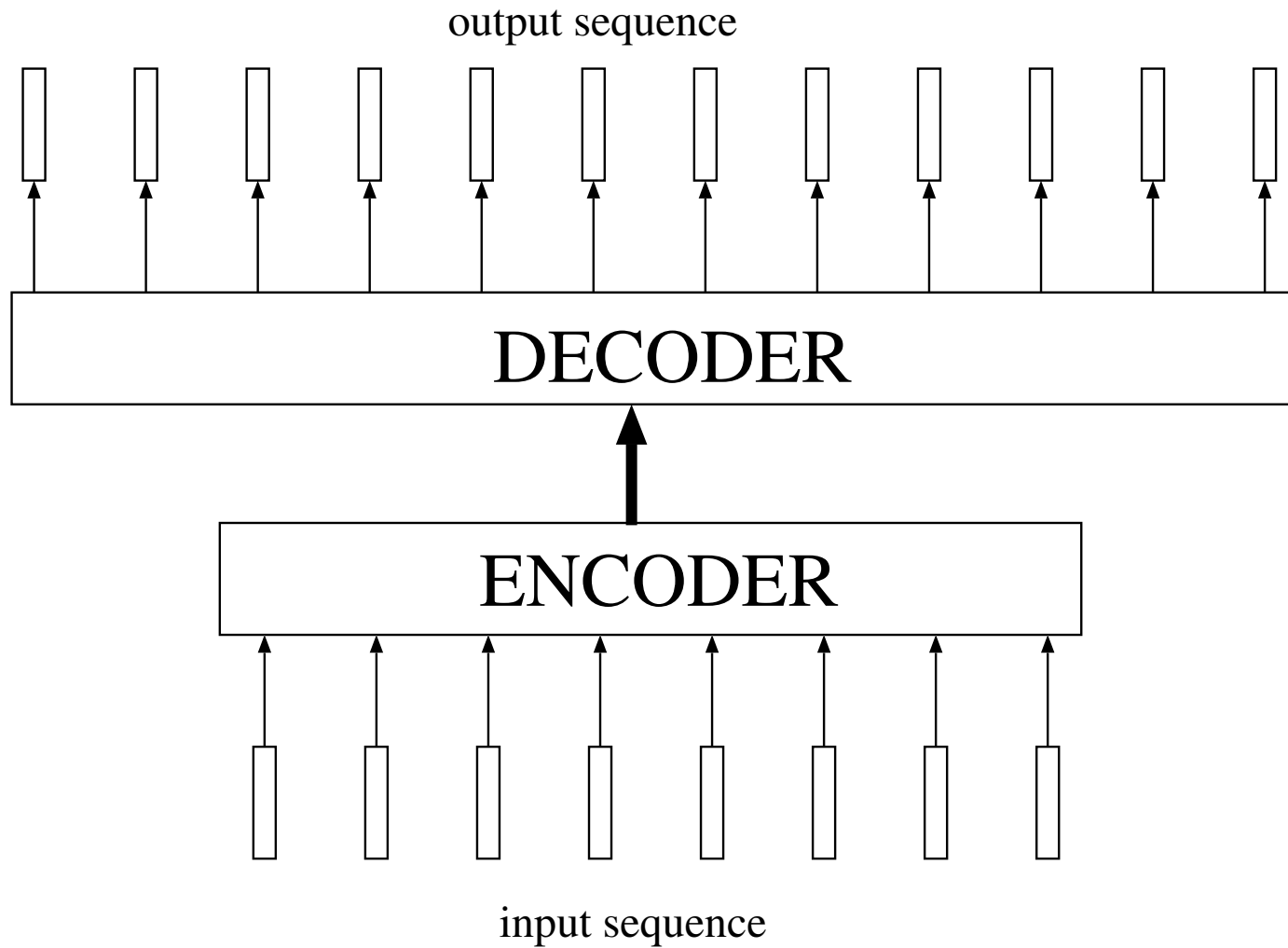
# Sequence-to-sequence problems

1. In many problems, the length of the output sequence is different from the length of the input sequence.
2. In recurrent neural and bounded-memory networks the length of the output sequence is equal to the length of the input sequence (or less in CTC)
3. The **encoder-decoder neural architectures** constitute a successful approach to deal with sequences of different lengths.

# Encoder-decoder architecture

1. Obtain a compact representation of the input sequence using an **encoder neural network**.
2. Generate an output sequence from the compact representation of the input sequence using a **decoder neural network**.

# Encoder-decoder architecture



# Encoder-decoder architecture

1. Project each input sequence into a sequence of continuous vectors → **input embedding**.
2. Obtain a contextual representation of the input sequence using an **encoder neural network**.
3. Generate an output sequence (typically as a-posteriori probabilistic distributions) from the contextual representation of the input sequence using a **decoder neural network**.
4. If the decoder is autoregressive, encode each output → **output embedding**.



# Encoder-decoder architecture

## LSTMs

# Encoder-decoder architecture using LSTM (Bahdanau et al. 2015)

- The RNN-based encoder ( $\mathbf{F}_e$ ) converts an input source token sequence  $x_1, \dots, x_J \equiv x_1^J$  into a state sequence  $\mathbf{h}_1^e, \dots, \mathbf{h}_J^e$ :

$$\mathbf{h}_0^e = \mathbf{0}; \quad \mathbf{h}_j^e = \mathbf{F}_e(\mathbf{W}_E(x_j), \mathbf{h}_{j-1}^e) = \mathbf{F}_e(\mathbf{x}_j, \mathbf{h}_{j-1}^e) \quad 1 \leq j \leq J$$

- The state sequence is converted into a vector  $\mathbf{u} = \mathbf{u}(x_1^J)$ :

$$\mathbf{u} = \mathbf{a}(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e) \triangleq \mathbf{h}_J^e$$

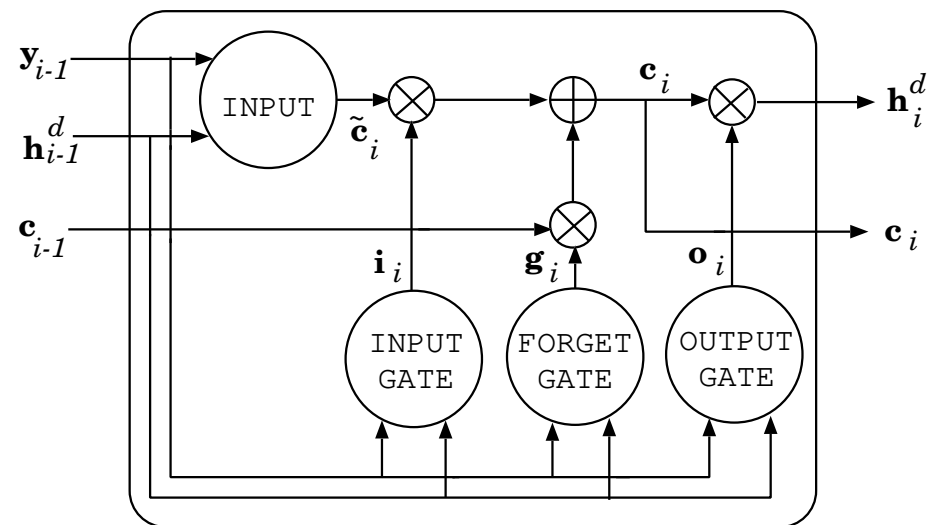
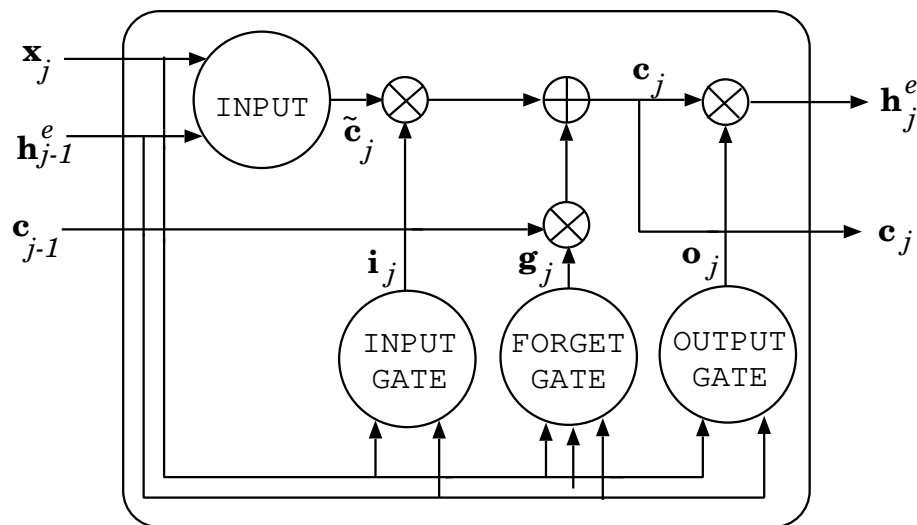
- The RNN-based decoder ( $\mathbf{F}_d$ ) is a kind of target language model

$$\mathbf{h}_0^d = \mathbf{u}; \quad \mathbf{h}_i^d = \mathbf{F}_d(\mathbf{W}_E(y_{j-1}^I), \mathbf{h}_{i-1}^d) = \mathbf{F}_d(\mathbf{y}_{j-1}^I, \mathbf{h}_{i-1}^d) \quad 1 \leq i \leq I$$

- The output token sequence is obtained as:

$$p(y_1^I | x_1^J) = \prod_{i=1}^I p(y_i | y_1^{i-1}, u(x_1^J)) = \prod_{i=1}^I \mathbf{f}_{sm}(\mathbf{W}_O \mathbf{h}_i^d)_{i(y_i)}$$

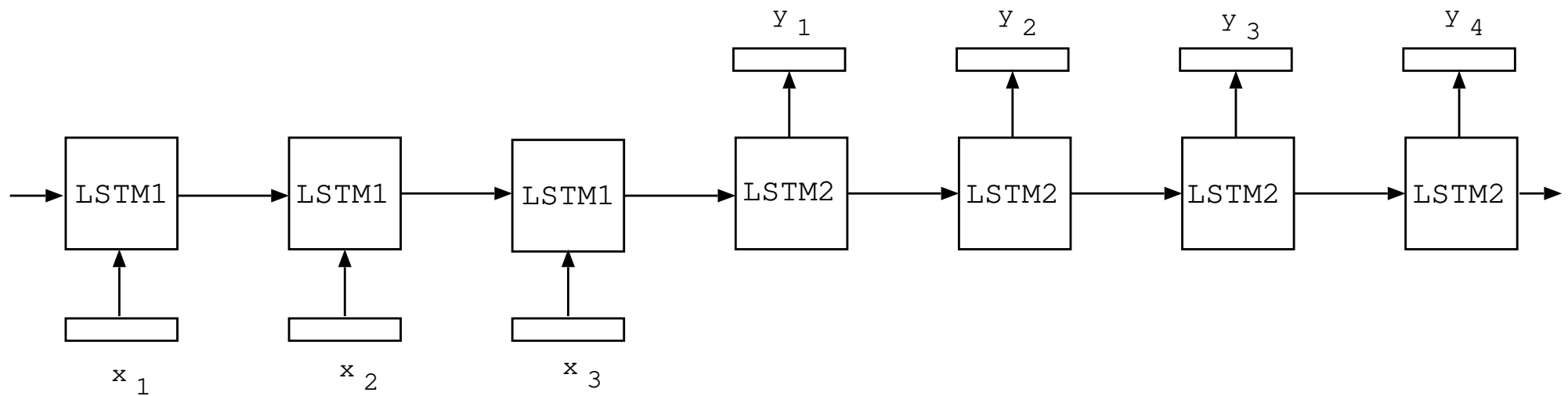
# Long Short-Term Memory (LSTM)



$$\mathbf{h}_j^e = \mathbf{F}(\mathbf{x}_j, \mathbf{h}_{j-1}^e) \quad 1 \leq j \leq J$$

$$\mathbf{h}_i^d = \mathbf{F}(\mathbf{y}_{i-1}, \mathbf{h}_{i-1}^e) \quad 1 \leq i \leq I \text{ with } \mathbf{h}_0^d = \mathbf{u}$$

# Encoder-decoder architecture using LSTM



# Attention models

Given a sequence of encoder states  $\mathbf{h}_j^e$  for  $1 \leq j \leq J$  and a decoder state  $\mathbf{h}_{i-1}^d$  for  $1 \leq i \leq I$  an **attention model** for the decoder state  $\mathbf{h}_i^d$  is a function:

$$\mathbf{u}_i = \mathbf{a}(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e, \mathbf{h}_{i-1}^d)$$

where  $\mathbf{a}$  is:

$$\mathbf{u}_i = \sum_{j=1}^J f_j(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e, \mathbf{h}_{i-1}^d) \mathbf{W}_V \mathbf{h}_j^e$$

$f_j$  is a softmax:

$$f_j(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e, \mathbf{h}_{i-1}^d) = \frac{\exp(\mathbf{W}_Q \mathbf{h}_{i-1}^d, \mathbf{W}_K \mathbf{h}_j^e))}{\sum_{j'=1}^J \exp(\mathbf{W}_Q \mathbf{h}_{i-1}^d, \mathbf{W}_K \mathbf{h}_{j'}^e))} \quad 1 \leq j \leq J$$

# Encoder-decoder architecture using LSTMs and attention

(Bahdanau et al. 2015)

- The RNN-based encoder ( $\mathbf{F}_e$ ) converts an input source token sequence  $x_1, \dots, x_J \equiv x_1^J$  into a state sequence  $\mathbf{h}_1^e, \dots, \mathbf{h}_J^e$ :

$$\mathbf{h}_0^e = \mathbf{0}; \quad \mathbf{h}_j^e = \mathbf{F}_e(\mathbf{W}_E(x_j), \mathbf{h}_{j-1}^e) = \mathbf{F}_e(\mathbf{x}_j, \mathbf{h}_{j-1}^e) \quad 1 \leq j \leq J$$

- The RNN-based decoder ( $\mathbf{F}_d$ ) is a kind of target language model

$$\mathbf{h}_0^d = \mathbf{0}$$

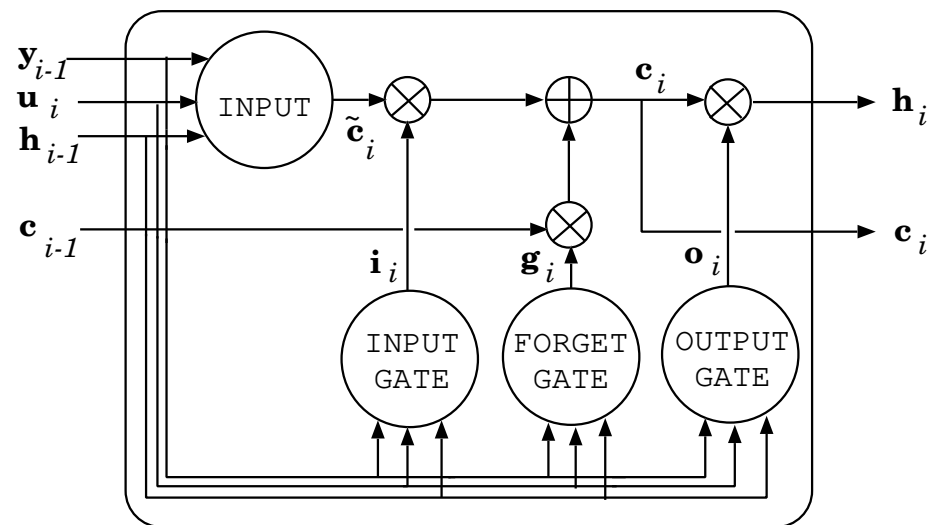
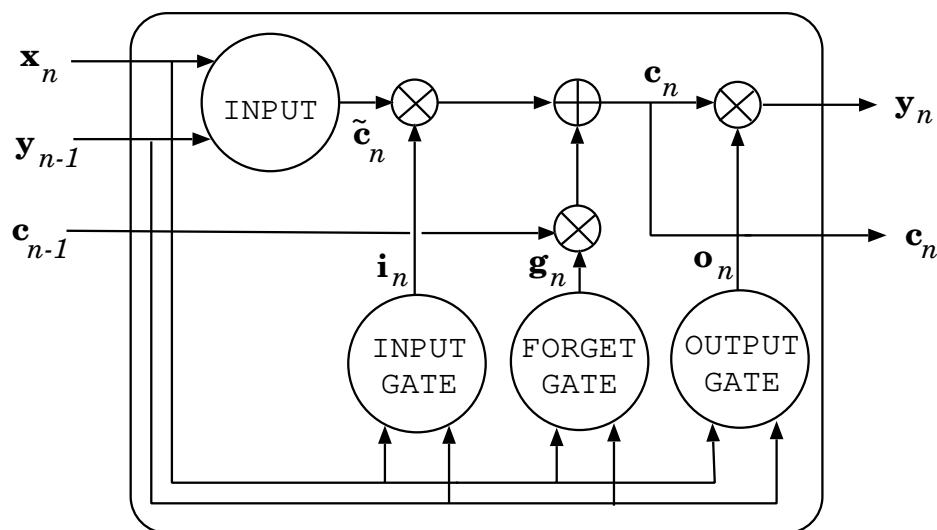
$$\mathbf{u}_i = \mathbf{a}(\mathbf{h}_1^e, \dots, \mathbf{h}_J^e, \mathbf{h}_{i-1}^d)$$

$$\mathbf{h}_i^d = \mathbf{F}_d(\mathbf{W}_E(y_{j-1}), \mathbf{h}_{i-1}^d, \mathbf{u}_i) = \mathbf{F}_d(\mathbf{y}_{j-1}, \mathbf{h}_{i-1}^d, \mathbf{u}_i) \quad 1 \leq i \leq I$$

- The output token sequence is obtained as:

$$p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J)) = \prod_{i=1}^I \mathbf{f}_{sm}(\mathbf{W}_O \mathbf{h}_i^d)_{i(y_i)}$$

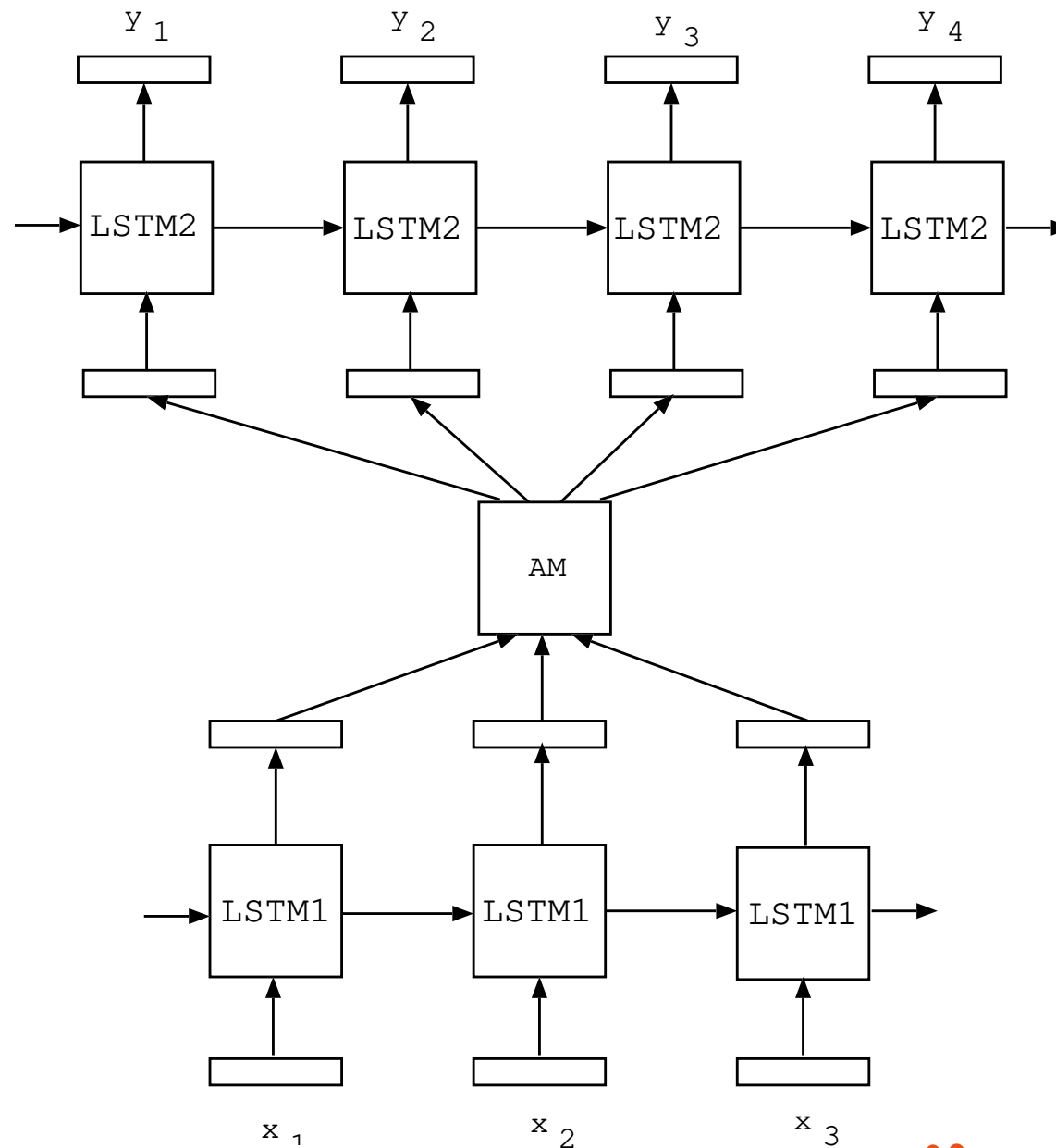
# Long Short-Term Memory (LSTM)



$$\mathbf{h}_j^e = \mathbf{F}(\mathbf{x}_j, \mathbf{h}_{j-1}^e) \quad 1 \leq j \leq J$$

$$\mathbf{h}_i^d = \mathbf{F}(\mathbf{y}_{i-1}, \mathbf{h}_{i-1}^e, \mathbf{u}_i) \quad 1 \leq i \leq I$$

# Encoder-decoder + attention model architecture





# Other encoder-decoder architectures

- Multilayer encoder / multilayer decoder
- Conditional recurrent units (RNN blocks with attention models in between)
- GRUs

# Encoder-decoder architecture

## Transformer

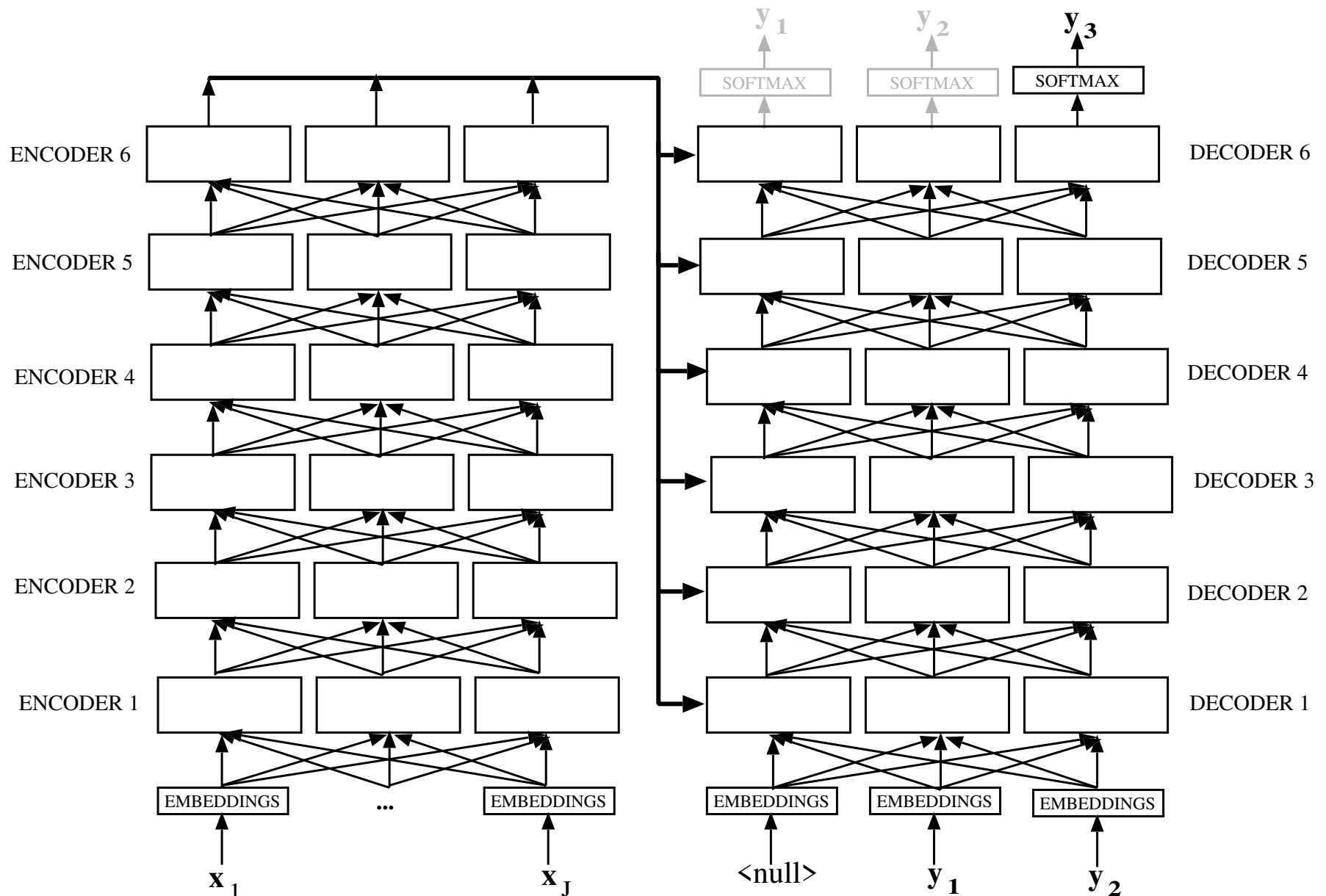
# Transformer (Vaswani 2017)

- Goal: Given an input token sequence  $x_1^J$  and an output token sequence  $y_1^I$ , compute:

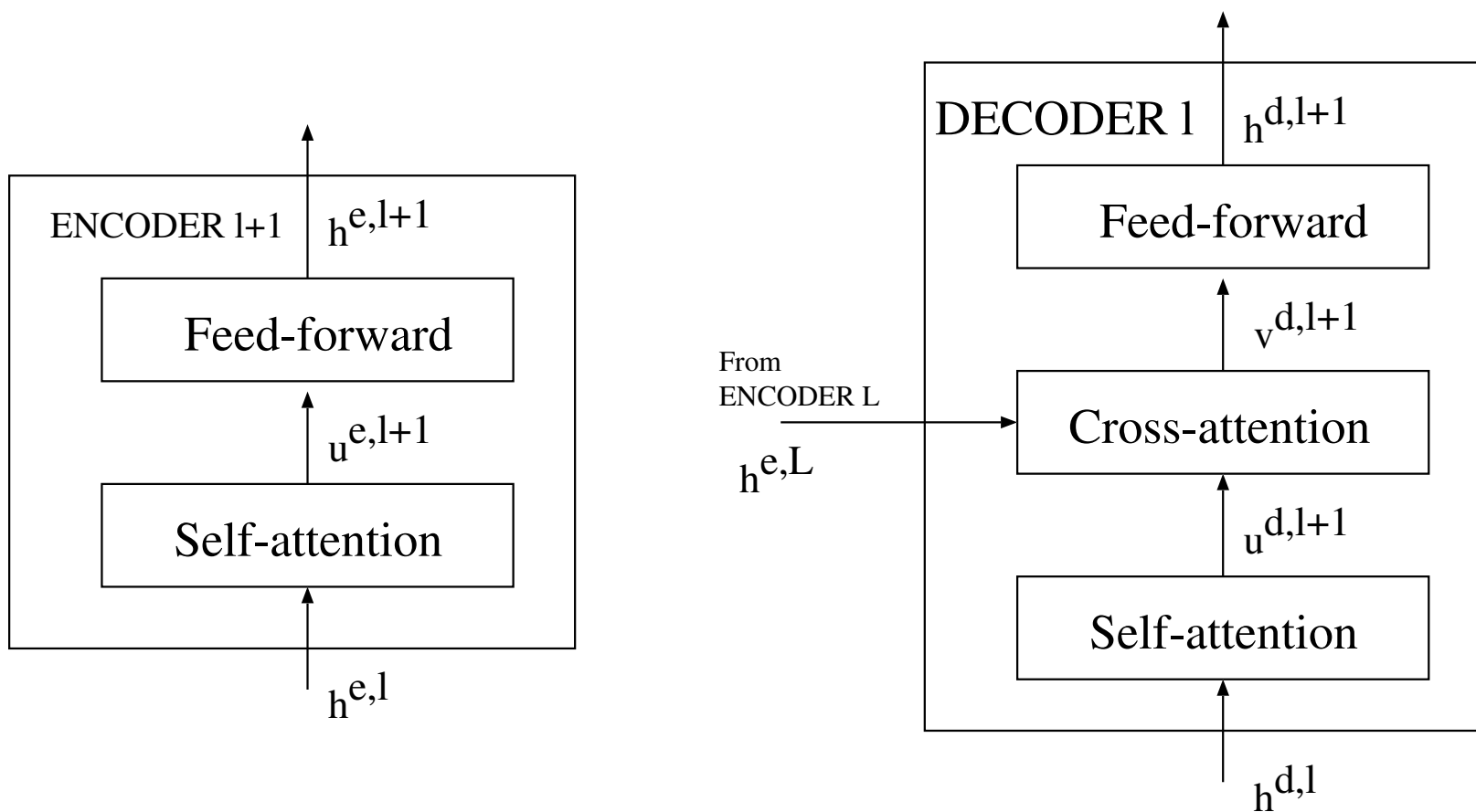
$$p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J))$$

- Feed-forward networks.
- Self-attention or intra-sentence attention:  $(j, j')$  &  $(i, i')$  in addition to the cross-attention  $(i, j)$ .
- Position encoding.
- Multi-head attention.
- Faster than RNN.

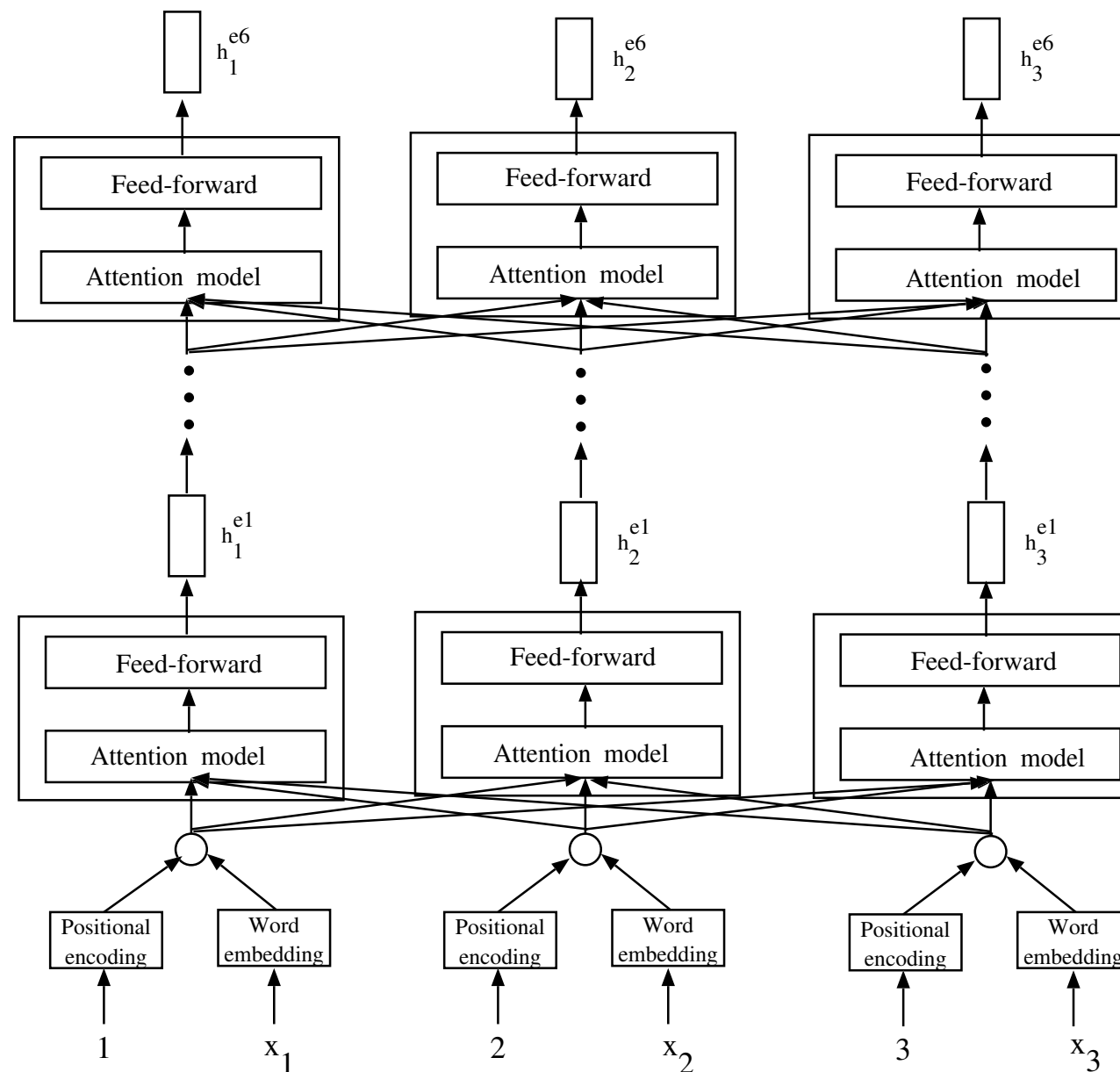
# Transformer



# Transformer



# A simplified view of the encoder in the Transformer



# Transformer

- A word  $x$  from a ordered vocabulary  $V_X$  with index  $i(x)$
- **Word embeddings:**  $\mathbf{W}_E(x) \equiv [\mathbf{W}_E]_{i(x)} = \mathbf{x} \in \mathbb{R}^{D_W}$ : a row of  $\mathbf{W}_E$  is the word embedding of  $x$
- **Positional embeddings:** Given a sentence  $x_1^J$  the positional embedding of position  $j$ ,  $1 \leq j \leq J$  is  $\mathbf{p}_j \in \mathbb{R}^{D_P}$ :

$$\begin{aligned} p_{j,2k} &= \sin(j/10000^{2k/D_P}) \\ p_{j,2k+1} &= \cos(j/10000^{2k/D_P}) \end{aligned}$$

- Relative positional embeddings.
- Learned positional embeddings.

# The encoder of Transformer

Given an discrete input sequence  $x_1^J$ ,

- Initialization:  $\mathbf{h}_j^{e,0} = \mathbf{x}_j = \mathcal{E}(x_j) \quad 1 \leq j \leq J$
- In layer  $l$  of the encoder ( $1 \leq l < L$ )

– Self-attention model:

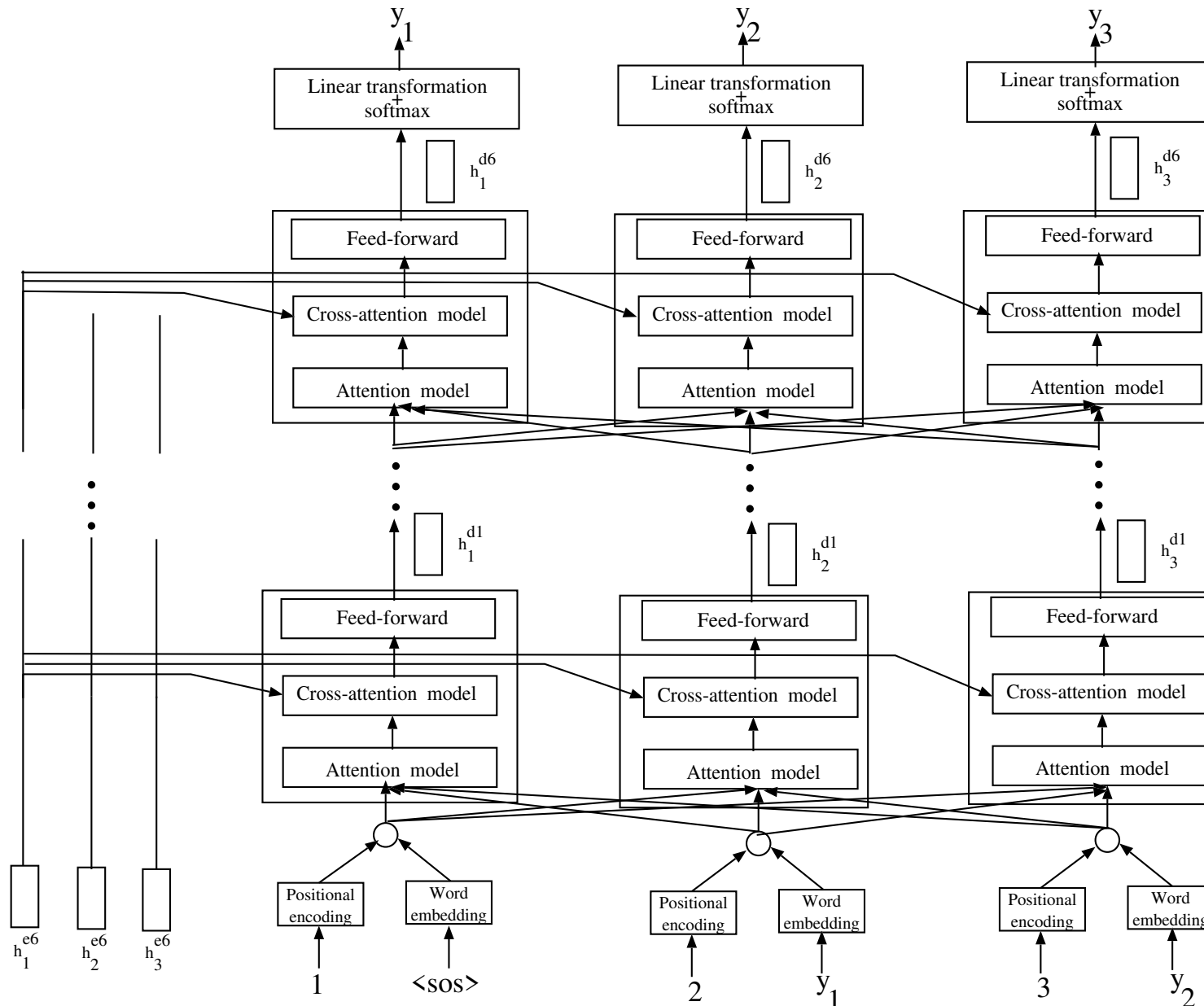
$$\mathbf{u}_j^{e,l+1} = \mathbf{a}(\mathbf{h}_1^{e,l}, \dots, \mathbf{h}_J^{e,l}, \mathbf{h}_j^{e,l}) \quad 1 \leq i \leq J$$

– Feed-forward network:

$$\mathbf{h}_j^{e,l+1} = \mathbf{F}_f(\mathbf{u}_j^{e,l+1}) \quad 1 \leq j \leq J$$



# A simplified view of the decoder in the Transformer



# The decoder of Transformer

- For  $1 \leq i \leq I$

- Initialization:  $\mathbf{u}_{i-1}^{d,1} = \mathbf{y}_{i-1} = \mathcal{E}(y_{i-1})$ , ( $y_0 = "$  < sos > ")

- In layer  $l$  of the decoder ( $1 \leq l < L$ ):

- \* Self-attention model:

$$\mathbf{u}_i^{d,l+1} = \mathbf{a}(\mathbf{h}_1^{d,l}, \dots, \mathbf{h}_i^{d,l}, \mathbf{h}_i^{d,l})$$

- \* Cross-attention model:

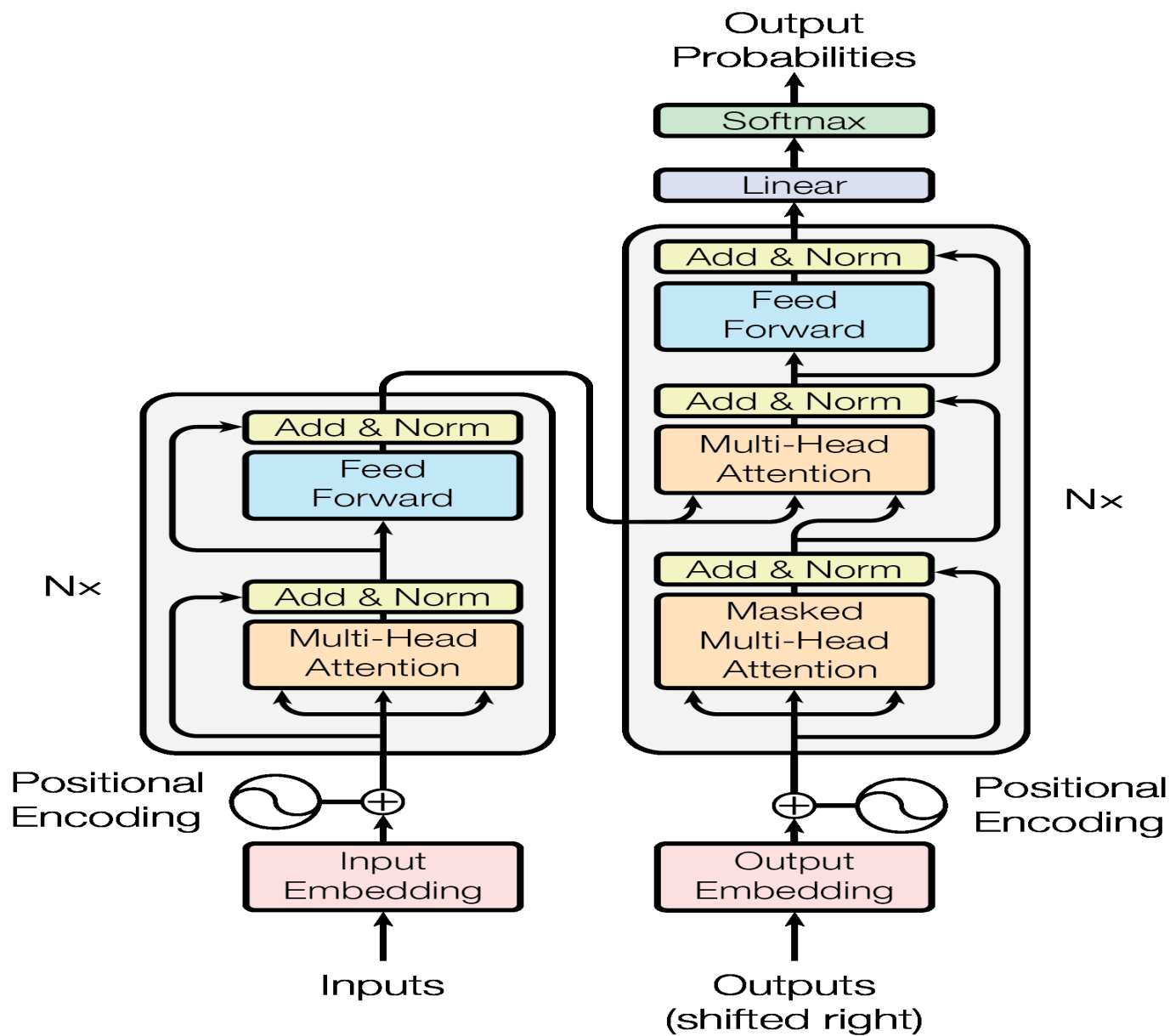
$$\mathbf{v}_i^{d,l+1} = \mathbf{a}(\mathbf{h}_1^{e,L}, \dots, \mathbf{h}_J^{e,L}, \mathbf{u}_i^{d,l+1})$$

- \* Feed-forward network:

$$\mathbf{h}_i^{d,l+1} = \mathbf{F}_f(\mathbf{v}_i^{d,l+1})$$

- Output generation:  $p(y_1^I \mid x_1^J) = \prod_{i=1}^I p(y_i \mid y_1^{i-1}, u(x_1^J)) = \prod_{i=1}^I \mathbf{f}_{sm}(\mathbf{W} \mathbf{h}_i^L)_{i(y_i)}$

# Transformer (Vaswani 2017)



# Transformer

- Residual networks
- Layer normalization
- Feed-forward networks: 1 hidden layer + ReLU function
- Multi-head attention

## Layer normalization (Xiong arXiv 2020)

- Given a sequence of vectors  $\mathbf{z}_1, \dots, \mathbf{z}_K$  from a layer with  $\mathbf{z}_k \in \mathbb{R}^d$  for  $1 \leq k \leq K$ , a **layer normalization**  $\mathbf{N}$  is  $\mathbf{N}(\mathbf{z}_1, \dots, \mathbf{z}_K) = (\bar{\mathbf{z}}_1, \dots, \bar{\mathbf{z}}_K)$  such that:

$$\bar{z}_{k,i} = \gamma \frac{z_{k,i} - \mu_i}{\sigma} + \beta \quad 1 \leq k \leq K, \quad 1 \leq i \leq d$$

where  $\gamma$  and  $\beta$  are hyper-parameters, and

$$\mu_i = \frac{\sum_{k=1}^K z_{k,i}}{K} \quad \sigma_i = \frac{\sum_{k=1}^K (z_{k,i} - \mu_i)^2}{K} \quad 1 \leq i \leq d$$

# Residual networks and feed-forward networks (Xiong arXiv 2020)

- Given  $\mathbf{z} \in \mathbb{R}^d$  and a function  $\mathbf{f} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ , a **residual function**  $\mathbf{R}$  is defined as:

$$\mathbf{R}(\mathbf{z}, \mathbf{f}(\mathbf{z})) = \mathbf{f}(\mathbf{z}) + \mathbf{z}$$

- Feed-forward networks,  $\mathbf{F}_f$ : Given a  $\mathbf{z} \in \mathbb{R}^d$ .

$$\mathbf{F}_f(\mathbf{z}) = \mathbf{W}_2 \mathbf{f}_{ReLU}(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$$

where  $\mathbf{W}_2$ ,  $\mathbf{W}_1$  are the weights of the second and first layer of the feed-forward networks, and  $\mathbf{b}_2$  and  $\mathbf{b}_1$  the corresponding bias.

# Multi-head attention in Transformer

For  $N$  heads and  $1 \leq l \leq L$ ,  $1 \leq j \leq J$  and  $1 \leq i \leq I$ :

- Encoder self-attention:

$$\begin{aligned} \mathbf{u}_{j,n}^{e,l+1} &= \mathbf{a}_n(\mathbf{h}_1^{e,l}, \dots, \mathbf{h}_J^{e,l}, \mathbf{h}_j^{e,l}) \quad 1 \leq j \leq J \text{ and } 1 \leq n \leq N \\ &= \sum_{j'=1}^J \frac{\exp(\mathbf{W}_Q^{e,n} \mathbf{h}_j^{e,l} \cdot \mathbf{W}_K^{e,n} \mathbf{h}_{j'}^{e,l})}{\sum_{j''=1}^J \exp(\mathbf{W}_Q^{e,n} \mathbf{h}_j^{e,l} \cdot \mathbf{W}_K^{e,n} \mathbf{h}_{j''}^{e,l})} \mathbf{W}_V^{e,n} \mathbf{h}_{j'}^{e,l} \quad 1 \leq j, j' \leq J \end{aligned}$$

$\mathbf{W}_Q^{e,n}, \mathbf{W}_K^{e,n}, \mathbf{W}_V^{e,n}$  are matrix of  $D_W/N \times D_W$

$$\mathbf{u}_j^{e,l+1} = [\mathbf{u}_{j,1}^{e,l+1}, \dots, \mathbf{u}_{j,N}^{e,l+1}]; \quad \mathbf{h}_j^{e,l+1} = \mathbf{F}_f(\mathbf{u}_j^{e,l+1}) \quad 1 \leq j \leq J$$

# Multi-head attention in Transformer

For  $N$  heads and  $1 \leq l \leq L$ ,  $1 \leq j \leq J$  and  $1 \leq i \leq I$ :

- Decoder self-attention:

$$\begin{aligned} \mathbf{u}_{i,n}^{d,l+1} &= \mathbf{a}_n(\mathbf{h}_1^{d,l}, \dots, \mathbf{h}_J^{d,l}, \mathbf{h}_i^{d,l}) \quad 1 \leq i \leq I \text{ and } 1 \leq n \leq N \\ &= \sum_{i'=1}^I \frac{\exp(\mathbf{W}_Q^{d,n} \mathbf{h}_i^{d,l} \cdot \mathbf{W}_K^{d,n} \mathbf{h}_{i'}^{d,l})}{\sum_{i''=1}^I \exp(\mathbf{W}_Q^{d,n} \mathbf{h}_i^{d,l} \cdot \mathbf{W}_K^{d,n} \mathbf{h}_{i''}^{d,l})} \mathbf{W}_V^{d,n} \mathbf{h}_{i'}^{d,l} \quad 1 \leq i, i' \leq I \end{aligned}$$

$\mathbf{W}_Q^{d,n}, \mathbf{W}_K^{d,n}, \mathbf{W}_V^{d,n}$  are matrix of  $D_W/N \times D_W$

$$\mathbf{u}_i^{d,l+1} = [\mathbf{u}_{i,1}^{d,l+1}, \dots, \mathbf{u}_{i,N}^{d,l+1}]; \quad \mathbf{h}_i^{d,l+1} = \mathbf{F}_f(\mathbf{u}_i^{d,l+1}) \quad 1 \leq i \leq I$$



# Multi-head attention in Transformer

For  $N$  heads and  $1 \leq l \leq L$ ,  $1 \leq j \leq J$  and  $1 \leq i \leq I$ :

- Decoder cross-attention:

$$\begin{aligned} \mathbf{v}_{i,n}^{d,l+1} &= \mathbf{a}_n(\mathbf{h}_1^{e,L}, \dots, \mathbf{h}_J^{e,L}, \mathbf{u}_i^{d,l+1}) \quad 1 \leq i \leq I \text{ and } 1 \leq n \leq N \\ &= \sum_{j=1}^J \frac{\exp(\mathbf{W}_Q^{c,n} \mathbf{u}_i^{d,l+1} \cdot \mathbf{W}_K^{c,n} \mathbf{h}_j^{e,L})}{\sum_{j=1}^J \exp(\mathbf{W}_Q^{c,n} \mathbf{u}_i^{d,l+1} \cdot \mathbf{W}_K^{c,n} \mathbf{h}_j^{e,L})} \mathbf{W}_V^{c,n} \mathbf{h}_j^{e,L} \quad 1 \leq i \leq I \end{aligned}$$

$\mathbf{W}_Q^{c,n}$ ,  $\mathbf{W}_K^{c,n}$ ,  $\mathbf{W}_V^{c,n}$  are matrix of  $D_W/N \times D_W$

$$\mathbf{v}_i^{d,l+1} = [\mathbf{v}_{i,1}^{d,l+1}, \dots, \mathbf{v}_{i,N}^{d,l+1}]; \quad \mathbf{h}_i^{d,l+1} = \mathbf{F}_f(\mathbf{v}_i^{d,l+1}) \quad 1 \leq i \leq I$$

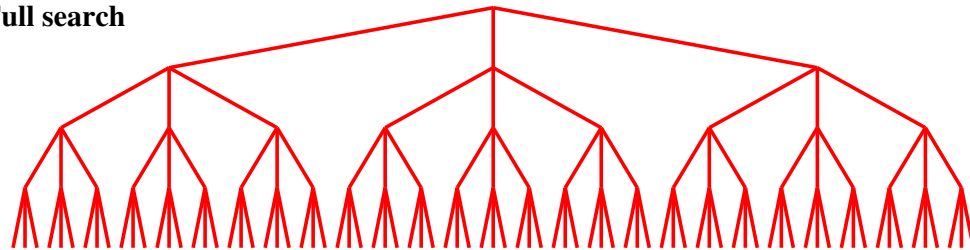
# Training (Koehn 2020)

- Given a training set of bilingual pairs  $T$ , maximize an objective function as the cross-entropy of the training set.
- Use of the computational graphs.
- Optimization algorithms: Based on backpropagation through time using stochastic gradient descent, Adagrad, Adadelata and Adam.
- Training issues
  - Early stopping.
  - Batch and layer normalization.
  - Data augmentation.
  - Hyperparameters
    - \* Dimension of source/target word embeddings.
    - \* Number of LSTM/GRU/Transformer in encoder/decoder.
    - \* Initial learning rate.
    - \* Patience.
    - \* Dropout parameter.
    - \* Weight decay.
    - \* Noisy injection (to inputs, outputs, ...)
    - \* ...

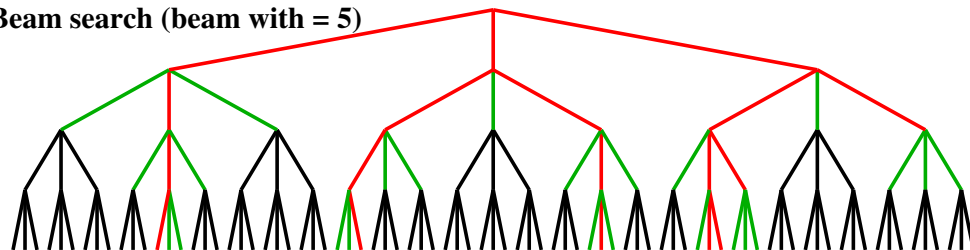
# Inference/decoding for discrete representations (Koehn 2020)

- Smart tree search by using beam search (suboptimal search)
  - At the beginning of the process there is a empty list of target hypotheses.
  - At each target position there is a bounded prefix tree of  $K$  target prefixes. For each prefix, a bounded set of extended prefixes is generated by appending the most promising words.
  - The process ends when a “eof” is generated.

Full search



Beam search (beam with = 5)



black=possible arcs

green = extended & discarded

red = extended & preserved

# Index

- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 *Pre-trained models* ▷ 113
- 11 Applications ▷ 129
- 12 Bibliography ▷ 153

# Why pre-trained models?

- Training neural machine translation models from scratch is expensive.
- Large corpora are necessary.
- For many task-specific, models there are trained with low resources.
- Pre-trained models have proved to be useful in many scenarios:
  - Text classification.
  - Text generation.
  - Image generation.
  - Sentiment analysis.
  - Image description/caption.
  - Video description.
  - ...

## Pre-trained models

- Bidirectional Encoder Representations from Transformers (BERT). Based on the Transformer encoder. [Devlin 2019] -Google AI Language-
- Generative Pre-trained Transformer (GPT, GPT-2, GPT-3, and GPT-4 coming soon GPT-J). Based on the Transformer decoder. [Radford 2018] -OpenAI-  
Demo: <https://gpt3demo.com/apps/openai-gpt-3-playground>  
The cost of a single training: \$5-12M. [Romero. Ultra-Large AI Models Are Over. 2022]  
Others: ImageGPT, chatGPT, ...
- Gopher. Based on the Transformer decoder. [Rae 2021] -DeepMind-
- Generalized Autoregressive Pretraining (XLNet). Decoder of Transformer without masks + permutation language modelling. [Yang 2020] -Google AI Brain-
- Cross-lingual Language Model Pretraining (XLM). Similar to BERT [Conneau 2019] -Facebook AI-
- BART and mBART. Full encoder-decoder (monolingual and multilingual) Transformer [Liu 2020] -Facebook AI-

# Pre-trained models

- Text-to-Text Transfer Transformer (T5. Trained with Colossal Clean Crawled Corpus (C4)). Complete Transformer. [Raffel 2020] -Google-
- Contrastive Language-Image Pre-training (CLIP). Multimodal encoder text+imatges [Radford 2021] -openAI-
- DALL-E 2. CLIP-based and GPT-3 image generation from text [Ramesh 2021] -openAI-  
<https://openai.com/dall-e-2/>
- DALL-E Mini. <https://github.com/borisdavyma/dalle-mini>
- Random Aligned Substitution Pre-training (mRASP). [Lin arXiv 2021]
- Embeddings from Language Models (ELMo). Birectional LSTM for pre-trained embeddings. [Peters 2018]
- Perceiver model [Hawthorne ICML 2022] -Deepmind-
- GATO model [Reed arXiv 2022] -Deepmind-

# Pre-trained models

- Chinchilla. Based on the Transformer decoder and uses less compute for fine-tuning and inference. [Hoffman 2022] -DeepMind-
- PaLM (Pathways Language Model). Based on the Transformer decoder and uses complex distributed computation for accelerators. [Chowdhery 2022] -Google-
- Flamingo: Visual Language Model [Alayrac 2022]. Input: Text and visual data interleaved as a prompt, and the output is a text. -DeepMind-
- Vision Transformer (ViT): BERT-like model for images. [Dosovitskiy 2020] -Google Research, Brain Team-
- Midjourney: Similar to DALL-E
- SetFit: based on sentence-BERT a BERT-like model. It outperforms GPT-3 being more than 1000 times smaller, [Tunstall 2022]
- ViT to GPT2 Encoder Decoder Model: A combination of ViT and GPT-2 for image captions

<https://aicomicrobooks.com/shop/zarya-of-the-dawn-download-now/>

<https://sachinruk.github.io/blog/pytorch/huggingface/2021/12/28/vit-to-gpt2-encoder-decoder-model.html/>



# Pre-trained models

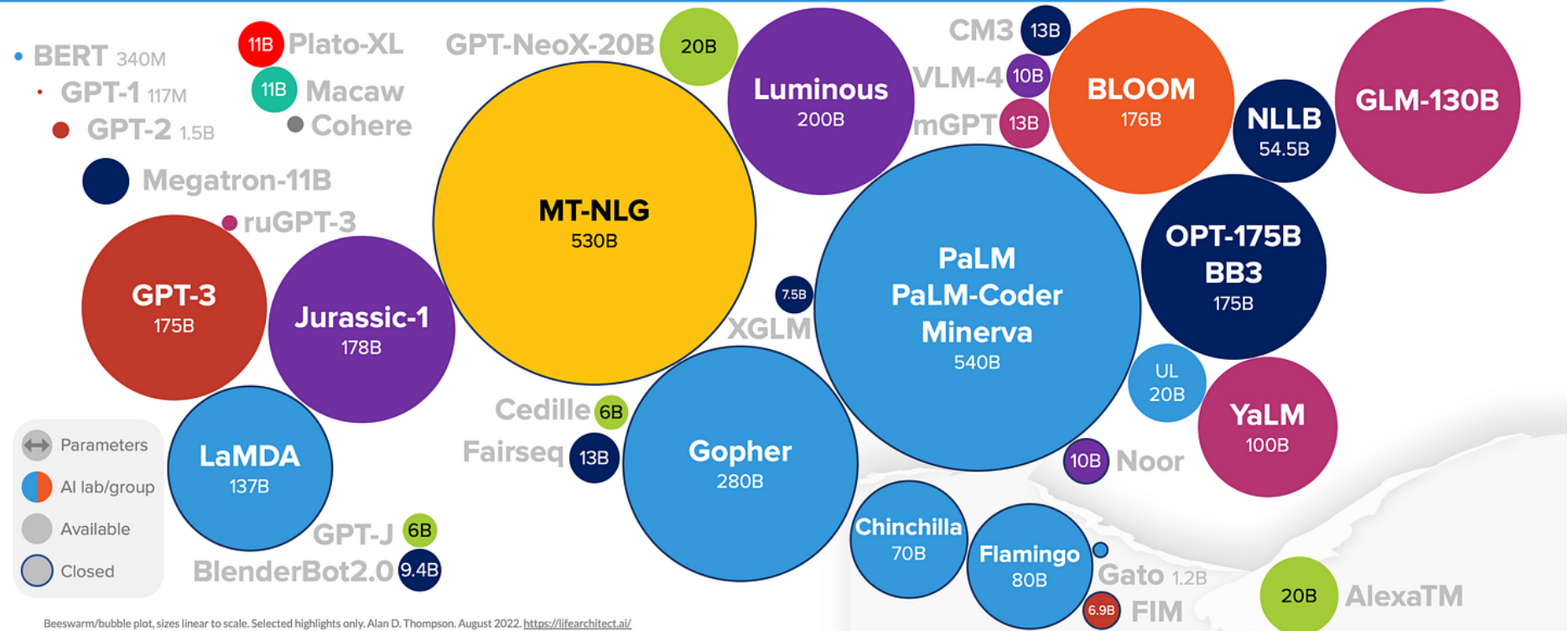
[Romero. Ultra-Large AI Models Are Over. Medium Daily Digest 2022]

- Google: LAMBDA (137B, May 2021), PaLM (540B, Apr 2022)
- Meta: OPT (175B, May 2022), BlenderBot3 (175B, Aug 2022)
- DeepMind: Gopher (280B, Dec 2021), Chinchilla (70B, Apr 2022)
- Microsoft-Nvidia: MT-NLG (530B, Oct 2021)
- BigScience: BLOOM (176B, June 2022)
- Baidu: PCL-BAIDU Wenxin (260B, Dec 2021), ERNIE-ViLG (24B, 2022)
- Yandex: YaLM (100B, June 2022)
- Tsinghua: GLM (130B, July 2022)
- AI21 labs: Jurassic-1 (178B, Aug 2021)
- Aleph Alpha: Luminous (200B, Nov 2021)

# Pre-trained models

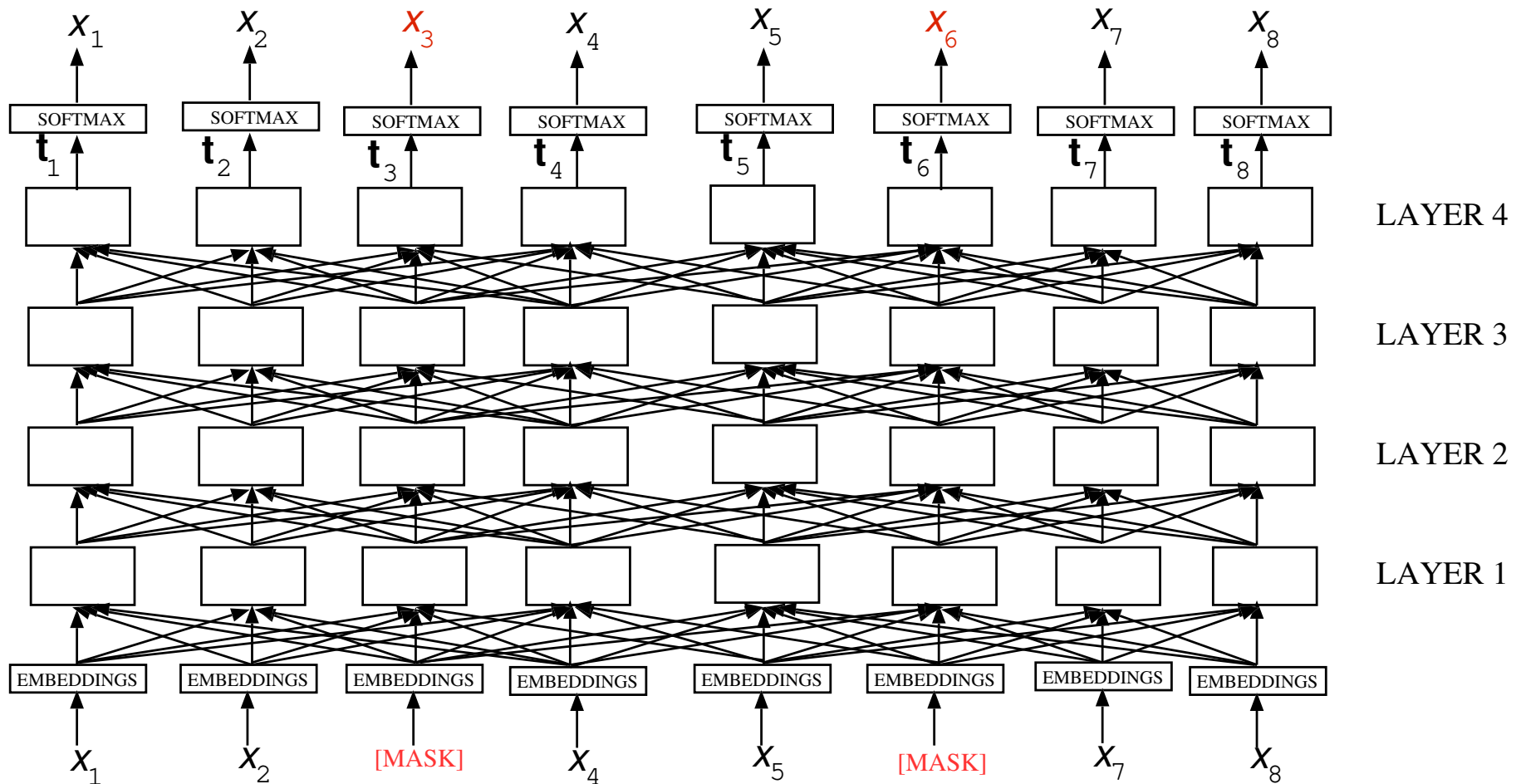
[Romero. Ultra-Large AI Models Are Over. Medium Daily Digest. 2022]

## LANGUAGE MODEL SIZES TO AUG/2022

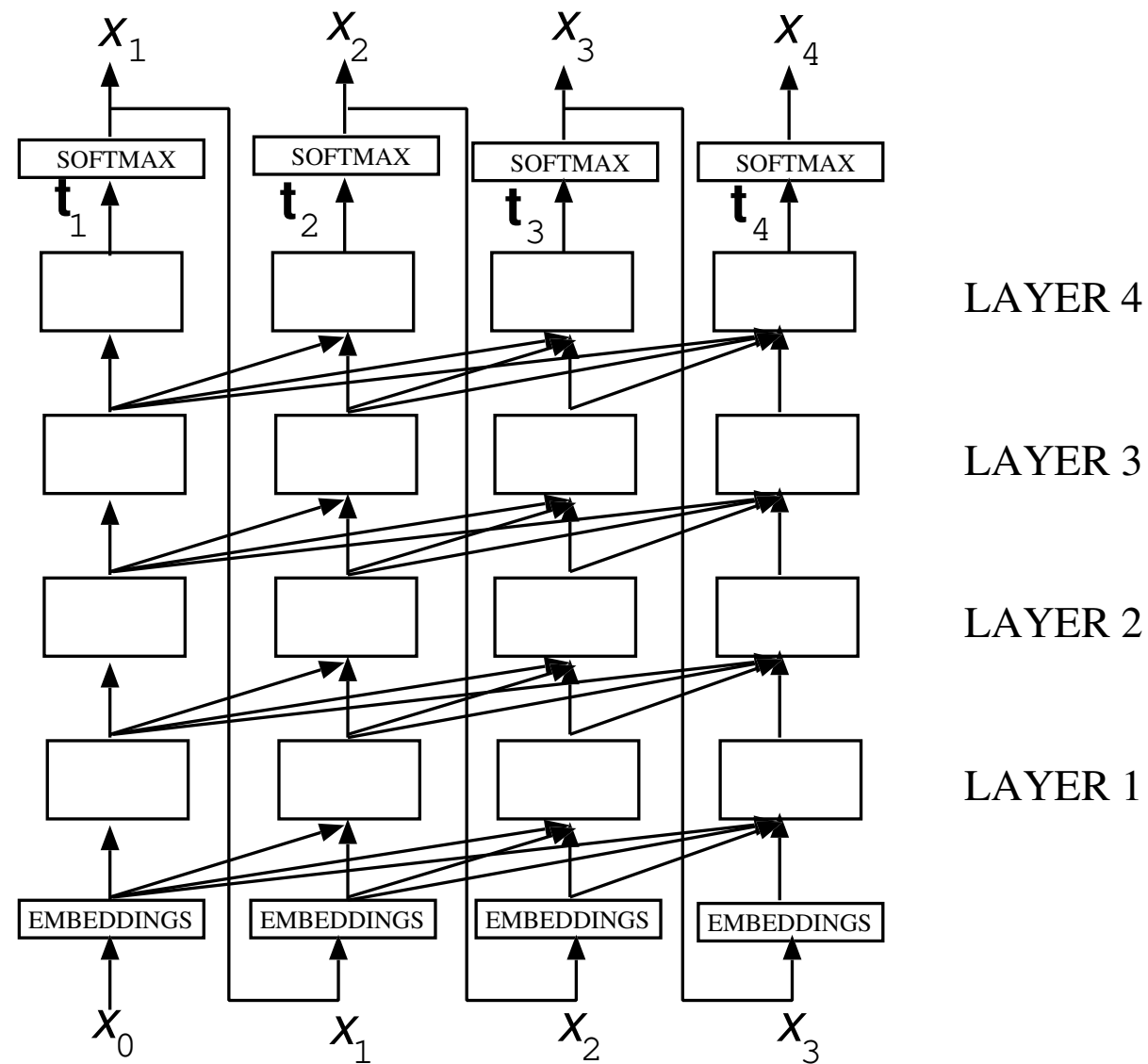


LifeArchitect.ai/models

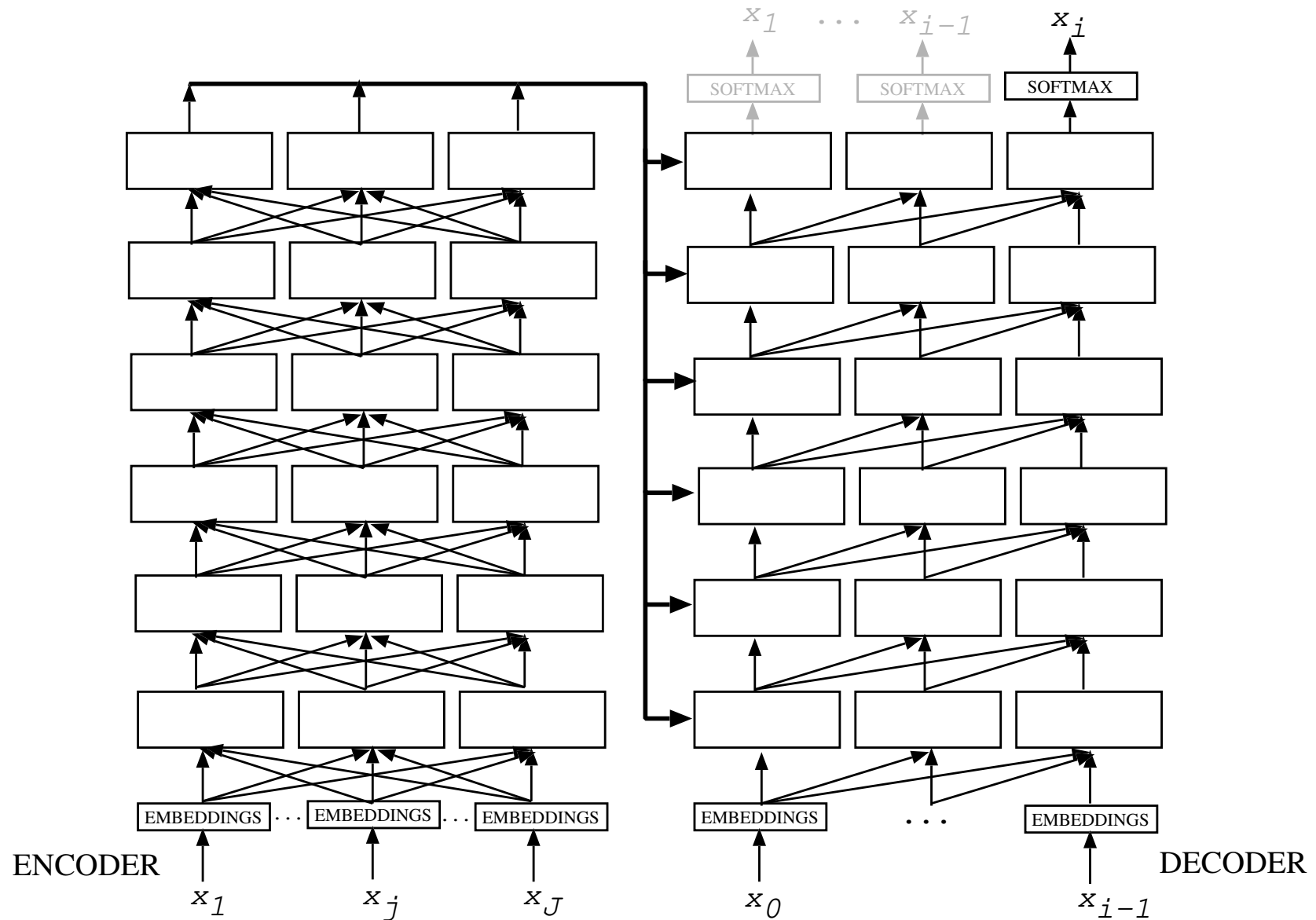
# BERT model [Devlin Google 2019]



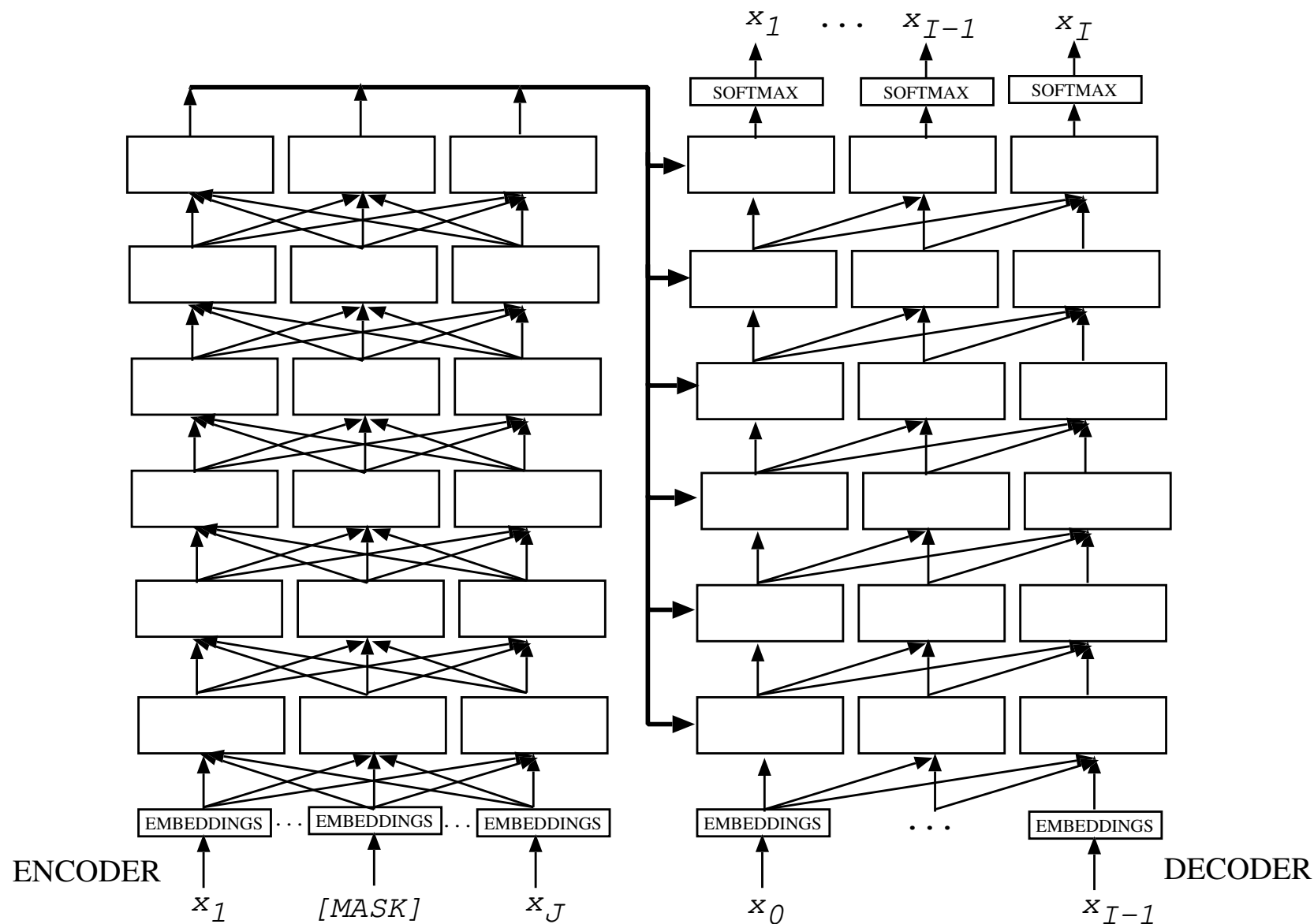
# GPT model [Radford OpenAI 2018]



# BART/T5 model [Lewis ACL 2020] [Raffel JMLR 2020]



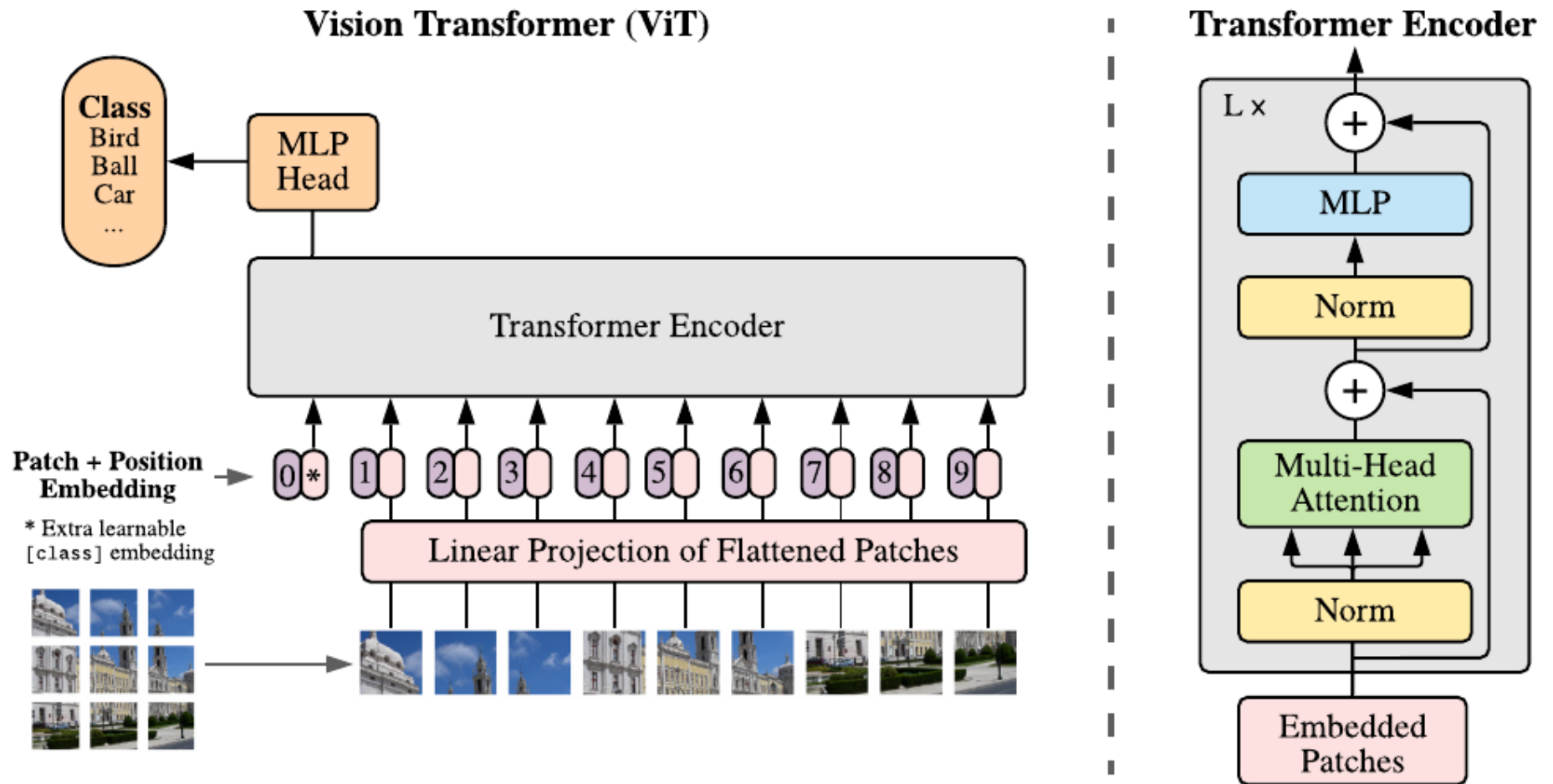
# Training BART/T5 model



# Training BART/T5 model

- BART (Amazon): masking input words & full input sentence.
- T5 (Google): masking input sub-sequences & input masked sub-sequences only.

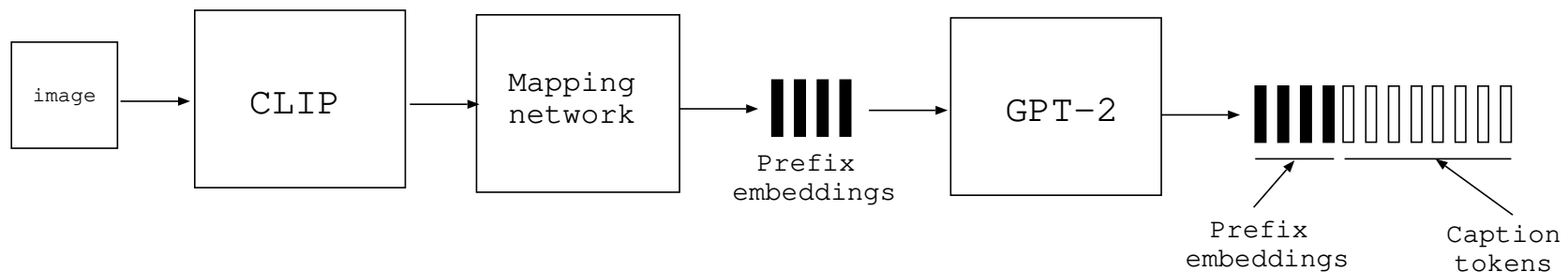
# Vision Tranformer [Dosovitskiy 2020]





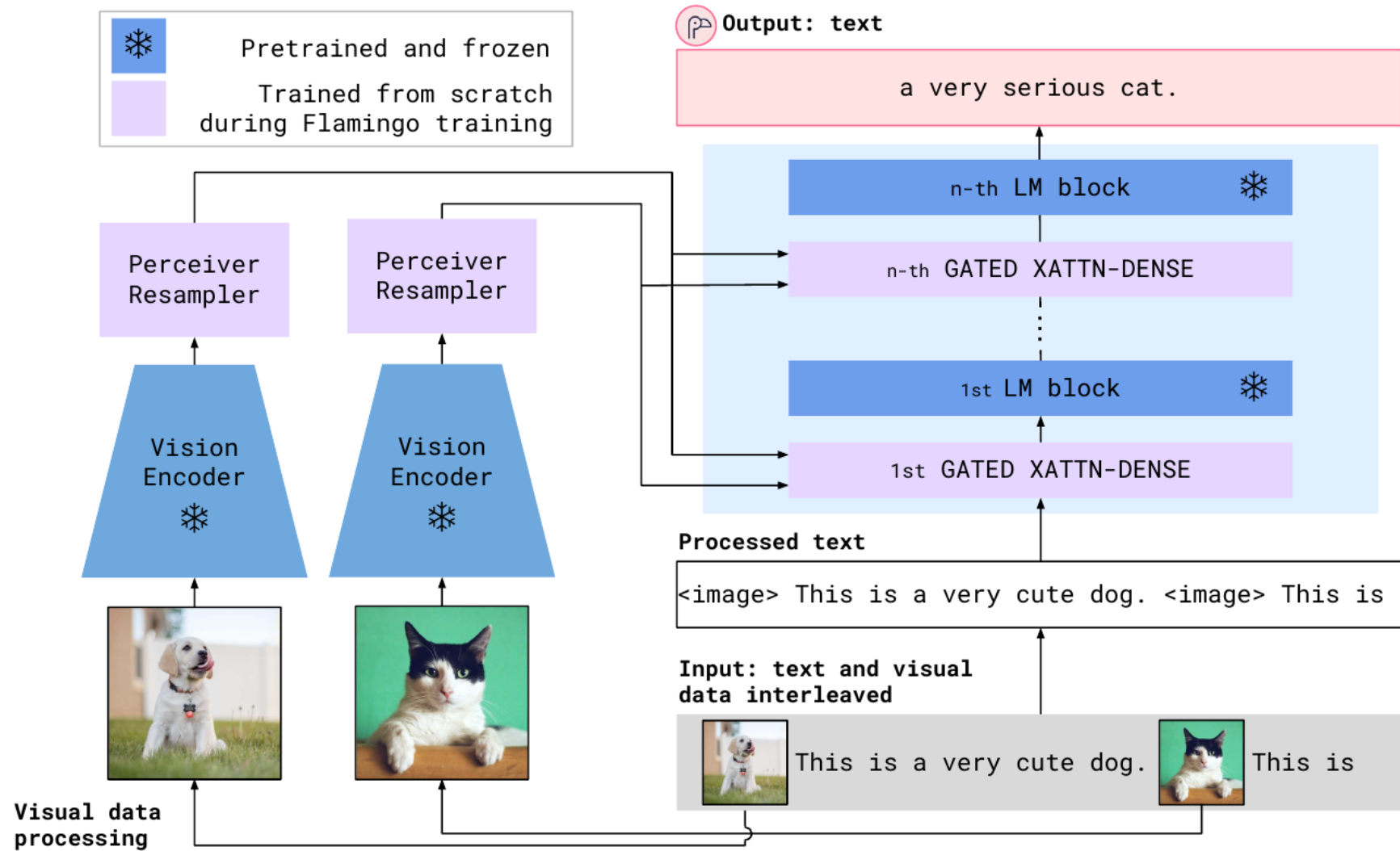
# CLIP model [Radford ICML 2021]

- Image encoder: ResNet-50 (CNN) or Vision Transformer (ViT).
- Text encoder: GPT architecture.
- Training: Optimize a symmetric cross entropy loss over the similarity scores between image and text embeddings.
- Application: ClipCap for image captioning [Mokady arXiv 2021]



- Another application: DALL-2 for image generation from text. [Ramesh OpenAI 2021]

# Flamingo model [Alyarac arXiv 2022]



# Prompts<sup>1</sup>

- Definition: “A prompt is a piece of text inserted in the input examples, so that the original task can be formulated as a (masked) language modeling problem.”<sup>1</sup>
- Discrete vs soft prompts.
  - Discrete prompts: manually-designed or automatically generated prompt.
- Frozen pre-trained models or fine-tuning models: Some models are so big that fine-tuning is not possible with regular resources.
- In-context learning: In addition of a piece of text, the models can see some few examples of the task (Few-shot). This is adequate when frozen models are used,

<sup>1</sup> <https://thegradient.pub/prompting/>

# Index

- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 *Applications* ▷ 129
- 12 Bibliography ▷ 153

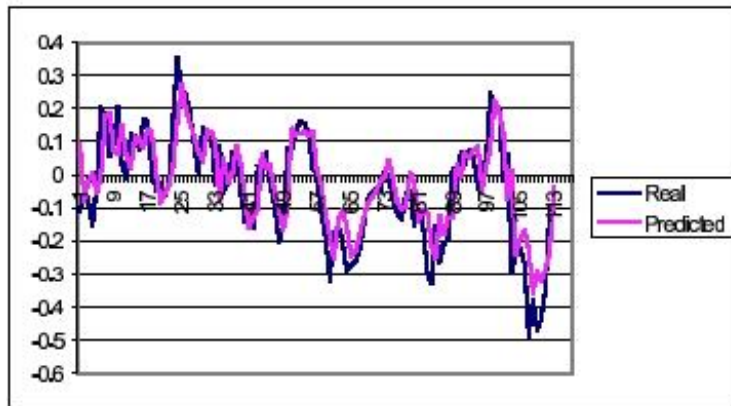
# Applications

## Applications with extended recurrent neural networks

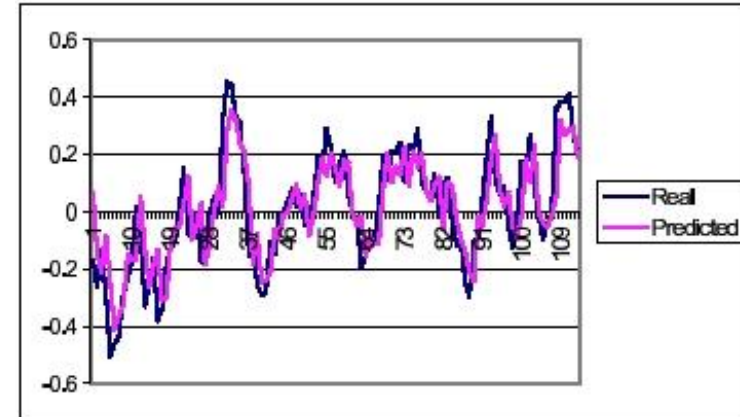
# Prediction of change of currency (Kondratenko & Kuperin, 2003)

(Euro, dollar, swiss franc, yen, pound (from 18/04/2001 to 1/10/2001))

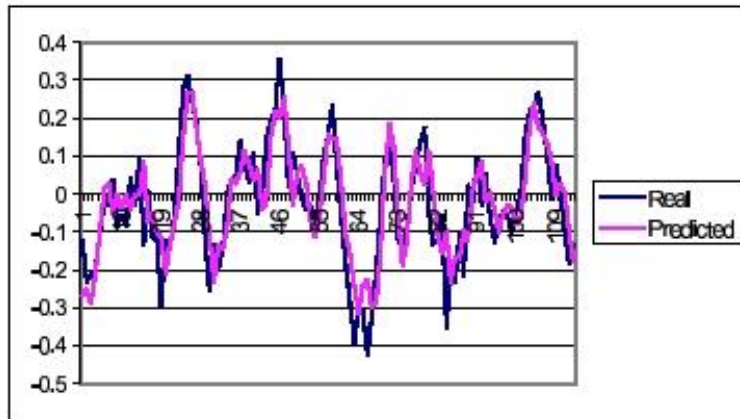
Hybrid Elman-Jordan network (100 hidden units, windows of five days, prediction to one day, 1,000 training samples)



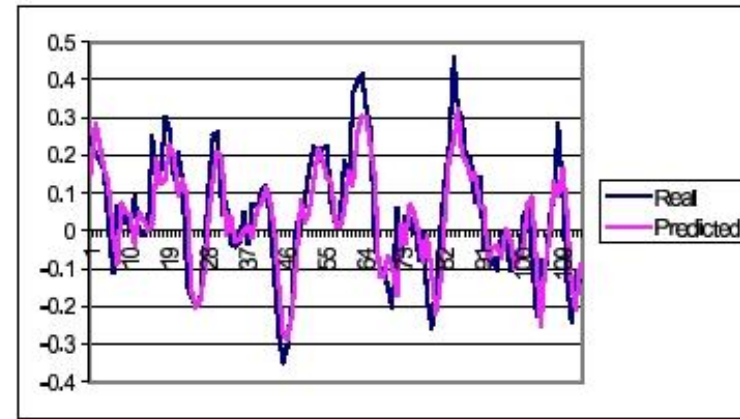
Swiss franc



Euro

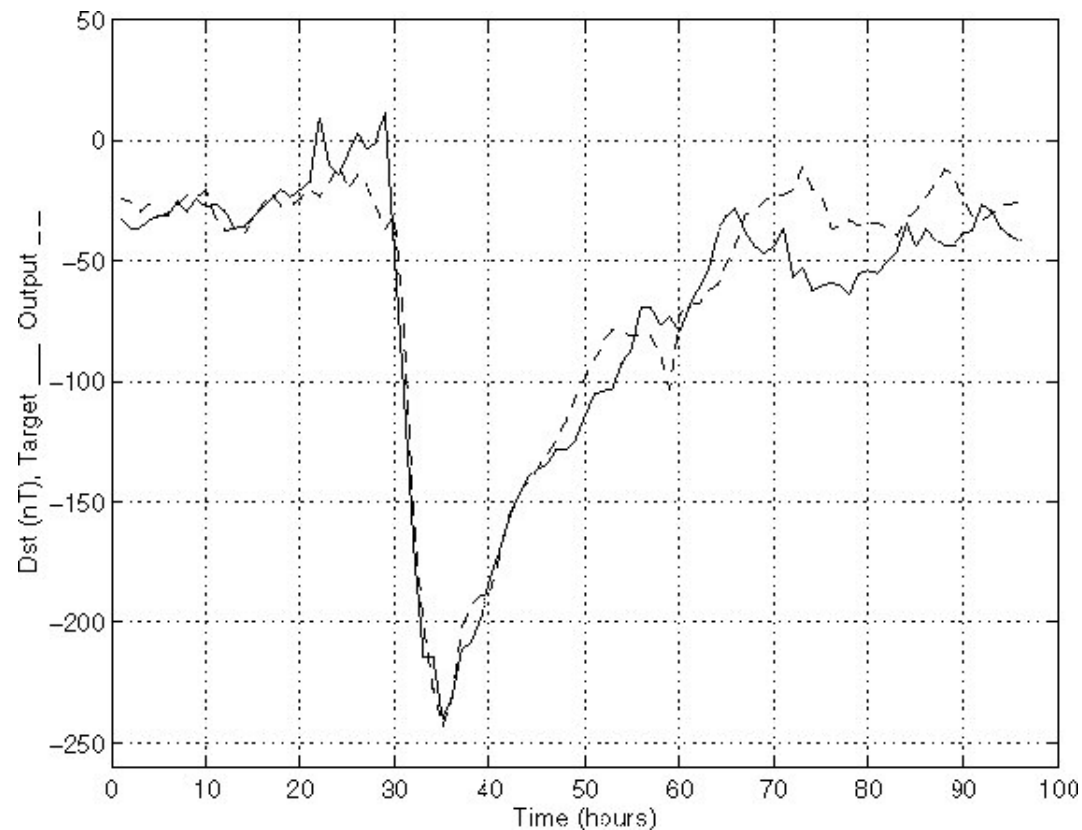


Pound



Yen

# Prediction of magnetic storms [Lundsteed 2001]



2h ahead based on solar wind data

DST= index that monitorizes worldwide magnetic storm level

# Classification of Chromosomes (Martínez 2007)

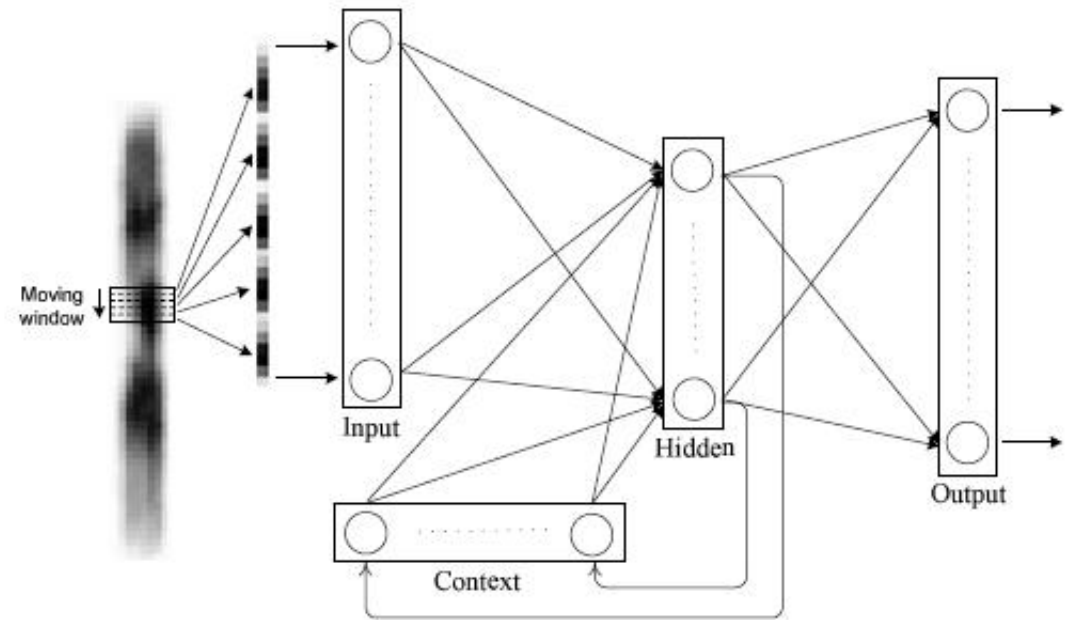
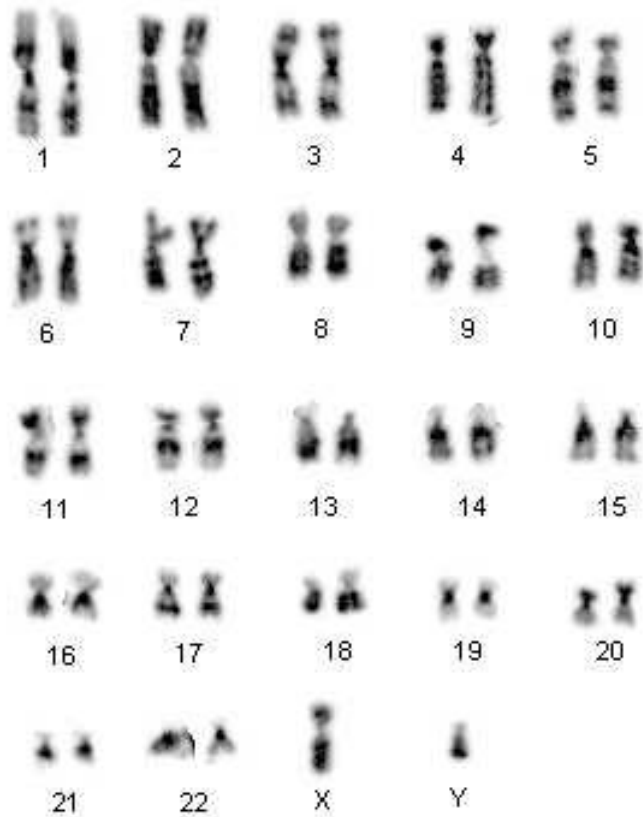


Figure 4. Illustration of the use of an Elman network for chromosome classification (chromosome image obtained from the Copenhagen data set).



# Classification of Chromosomes (Martínez 2007)

Copenhagen corpus: 2,804 cells with 48 chromosomes

1. Classification of each frame in one class.
2. Classification of each chromosome by voting

| System                          | Test-set error % |
|---------------------------------|------------------|
| Multilayer perceptron           | 6.5              |
| Hierarchical neural networks    | 5.6              |
| Continuous HMM (cell dependent) | 4.6              |
| Elman network (cell dependent)  | 3.9              |

# Text understanding using Elman networks (Castaño 1995)

- **Problem:** Translate an orthographic representation of a number (between 0 and  $10^6 - 1$ ) to an arithmetic representation:

$$\text{/docemilseis/} \Rightarrow (+2+10)\times 1000 + (+6)$$

- **Inputs:** a local representation

|   |   |   |   |   |
|---|---|---|---|---|
| a | e | i | o | u |
| c | r | d | s | v |
| t | q | n | m | l |
| z | h | y |   |   |

- **Outputs:** a local representation

|      |      |       |          |     |     |
|------|------|-------|----------|-----|-----|
| +0   | +1   | +2    | +3       | +4  | +5  |
| +6   | +7   | +8    | +9       | +10 | x10 |
| +100 | x100 | +1000 | )x1000+( |     |     |

# Text understanding using Elman networks

- *Topology*

- Elman network
- 19 input units, 16 output units and 40 hidden units
- windows of one input vector (without distortion) and windows of five input vectors (with distortion)

- *Training*

- Truncate gradient with momentum and pattern-based training
- Training corpus: 5000 pairs organized into 5 blocks
- Convergency: after the presentation of 10 random blocks

- *Test*

- Test corpus: 2000 (from another 5000) pairs.
- Percentage of right sentences: 99%
- Percentage of distorted sentences (10%) right recognized (distorted training): 42%

# Applications

## Applications with LSTMs or Transformer

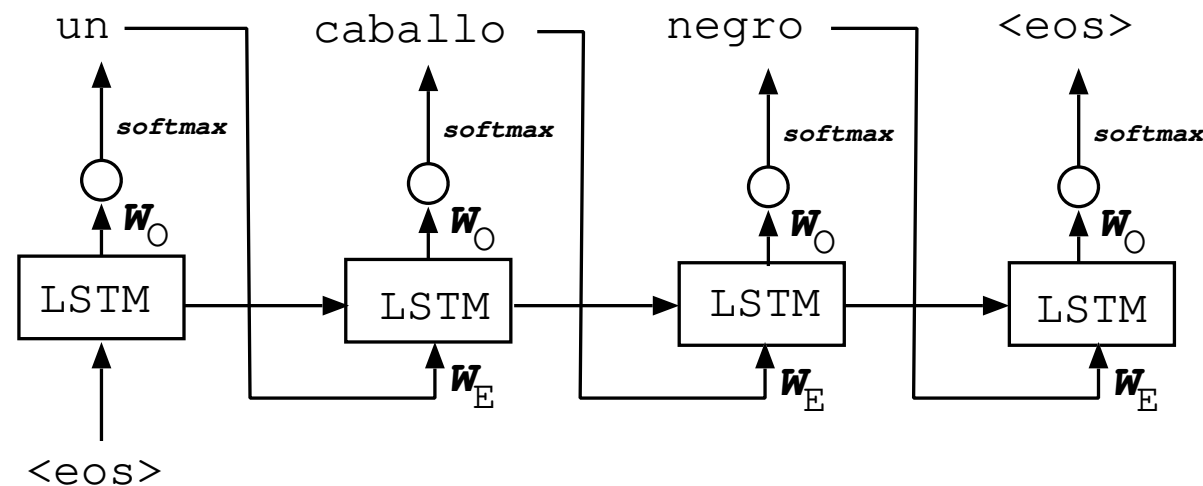
# Applications of LSTM o Transformer-based models

- Language modelling (Sundermeyer, 2015).
- Image description using LSTM and CNN (Vinyals 2015)
- Multilingual multimodal image description (Elliott 2016)
- Video description using LSTM and CNN (Venugopalan 2015)
- Visual question answering (Li 2016)
- Text summarization (Sinha 2018)
- Modeling human behaviour (Katzer 2020).
- Image Generation.
- ...

# A LSTM application: language modeling (Sundermeyer, 2015)

- For a word sequence  $x_1, \dots, x_N \equiv x_1^N \in \Sigma^N$

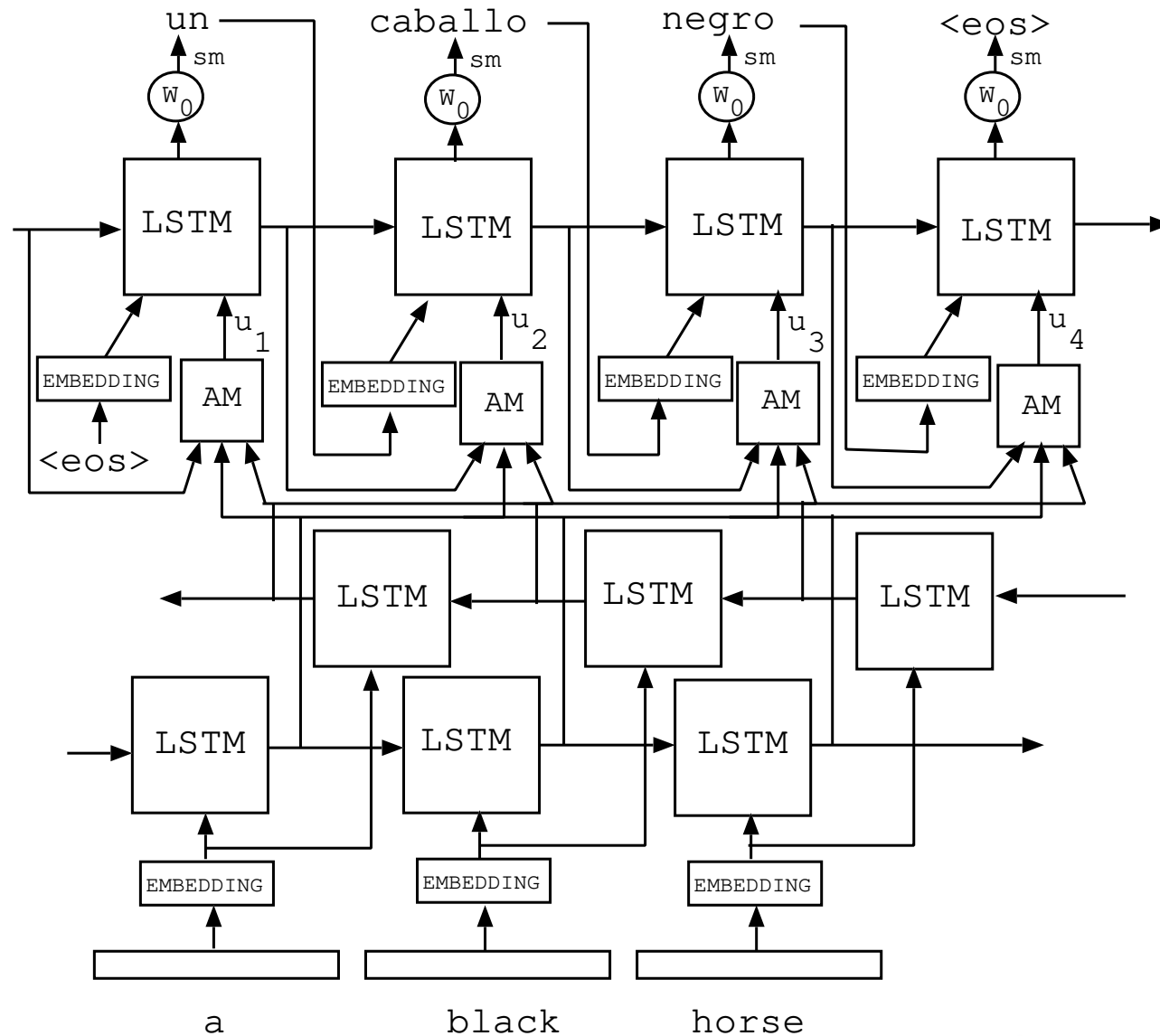
$$p(x_1^N) = \prod_{n=1}^N p(x_n | x_1^{n-1}) = \prod_{n=1}^N \mathbf{f}_{sm}(\mathbf{W}_O \mathbf{h}_n)_{i(x_n)}$$



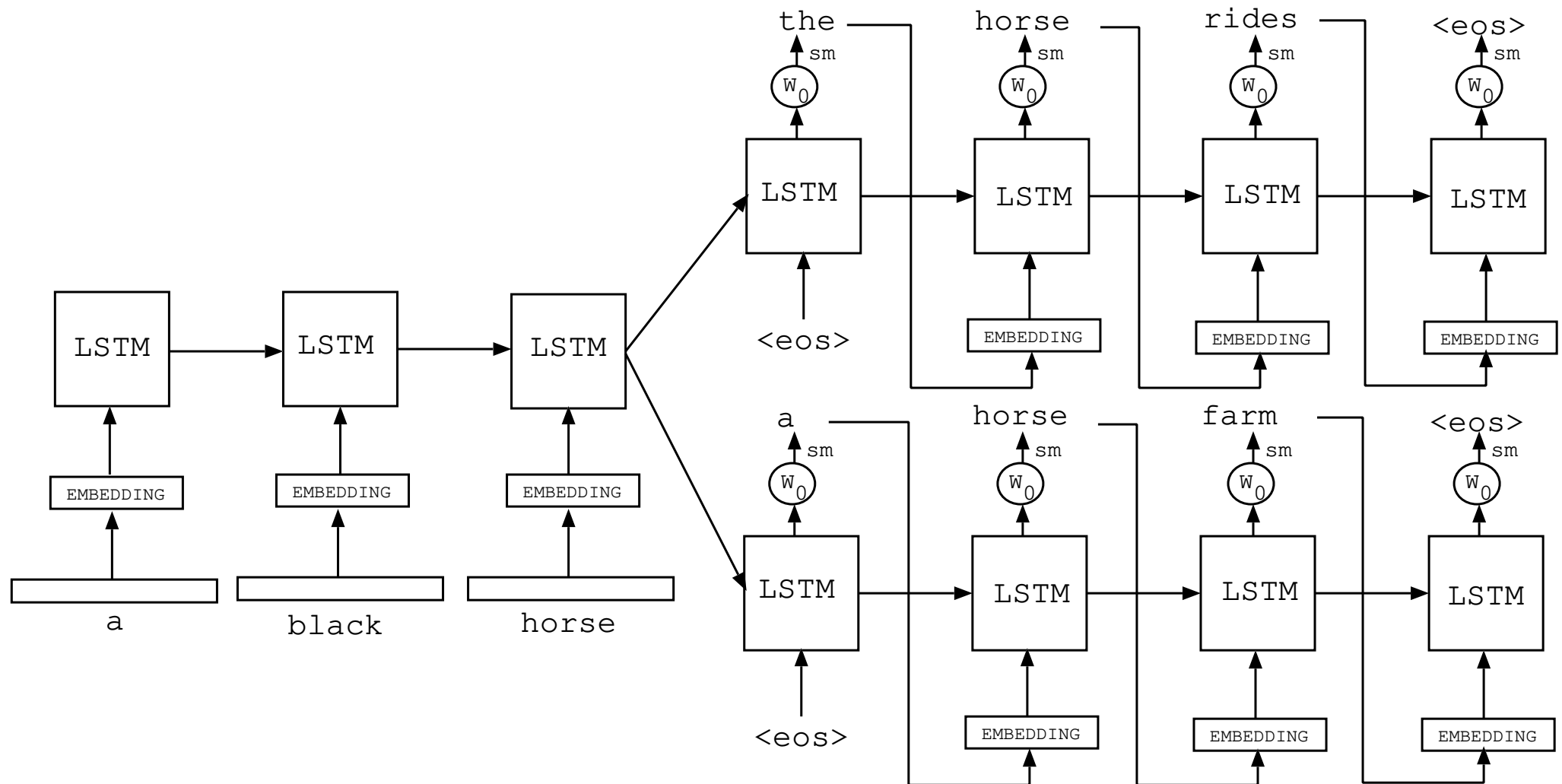
- The training is based on the optimization of the objective function can be the cross-entropy (equivalent to maximize the likelihood or minimizing the perplexity):

$$L(\Theta) = -\log \prod_{n=1}^N p(x_n | x_1^{n-1}; \Theta) = -\sum_{n=1}^N \log \mathbf{f}_{sm}(\mathbf{W}_O \mathbf{h}_n)_{i(x_n)}$$

# Machine Translation with bidirectional encoder-decoder with attention model (Luong 2015)



# Sentence embedding with encoder-decoder (Mishra, 2019)





# Online handwriting text recognition (Liwicki 2007)

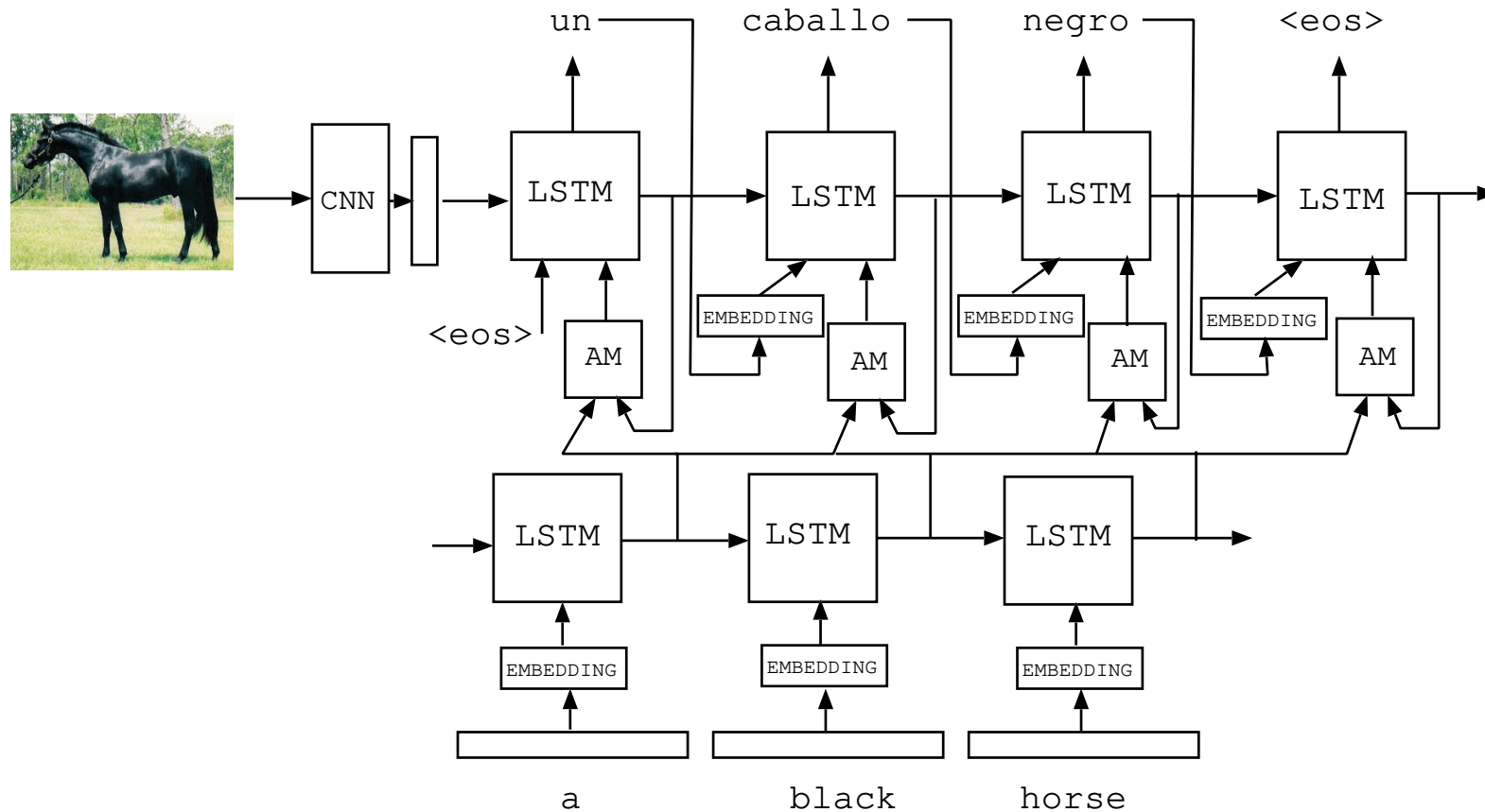
IAM-OnDB corpus: 86,272 word instances from a 11,050 word dictionary



never die. You with they did.  
never die. You with they did.

| System                   | Test-set error % |
|--------------------------|------------------|
| Continuous HMM           | 34.6             |
| Bidirectional RNN (LSTM) | 26.0             |

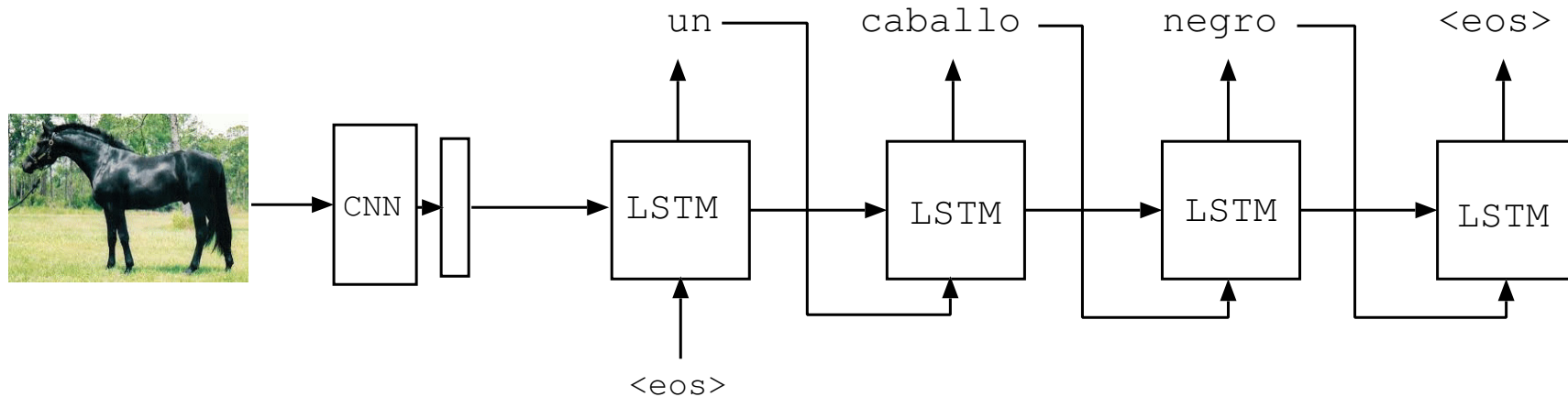
# Multimodal machine translation



Given an image  $x$  and a description of the image in a source language  $s$ , search for a translation of  $s$ :  $\hat{t} = \underset{t}{\operatorname{argmax}} p(t \mid x, s)$

Multimodal transformers. [Yao& Wan 2020]

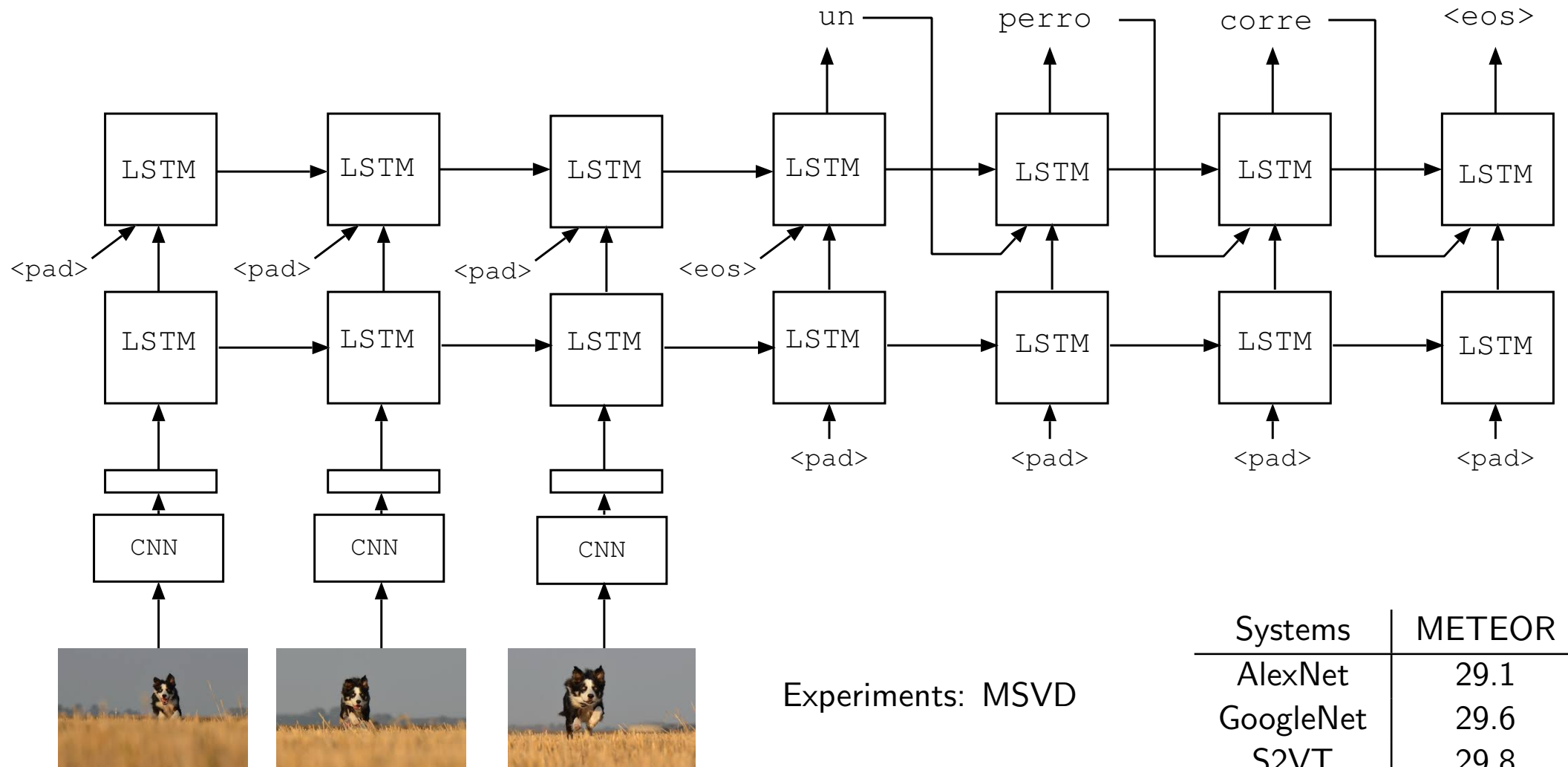
# Image description using LSTM and CNN (Vinyals 2015)



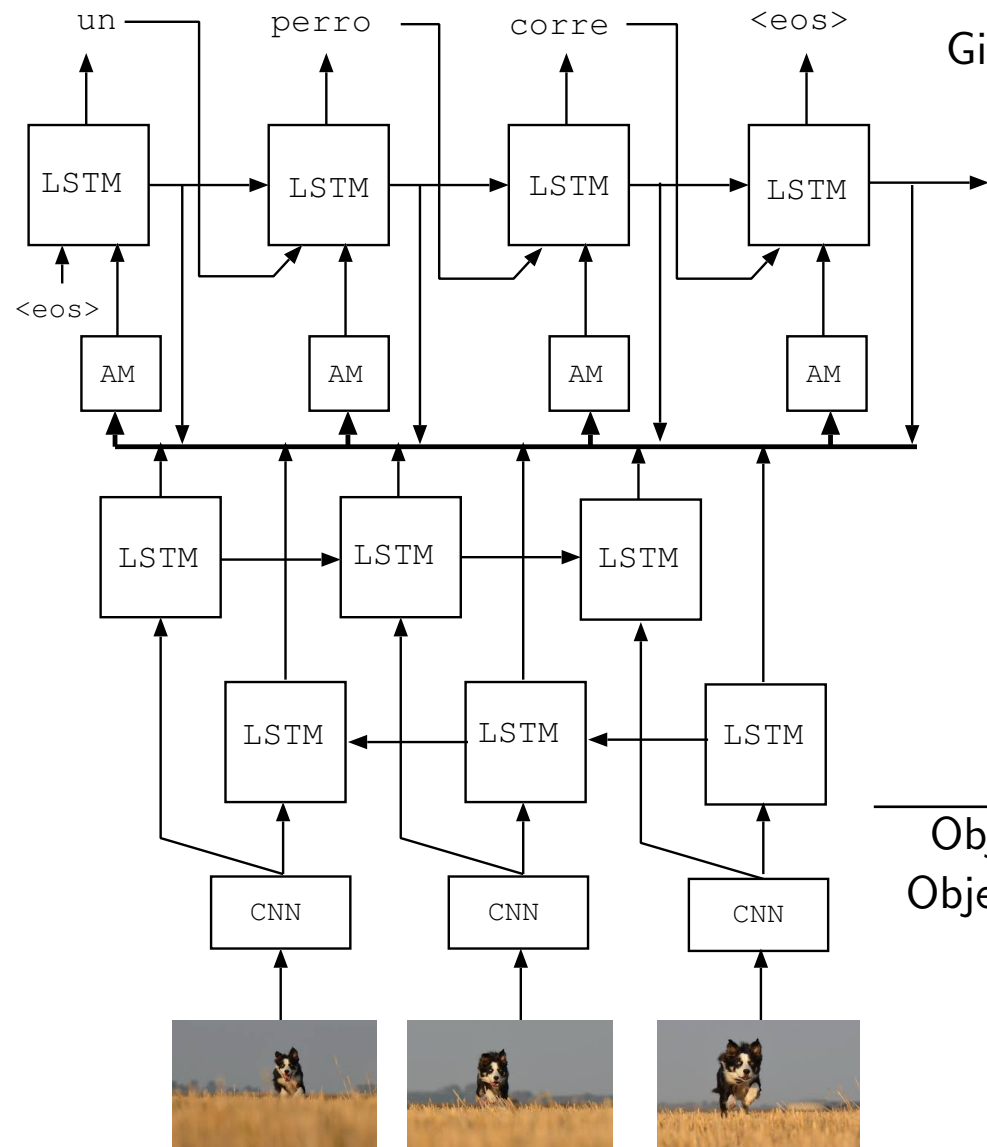
Experiments (BLEU-1):

| Systems          | Corpus |          |         |
|------------------|--------|----------|---------|
|                  | PASCAL | Flickr30 | Flickr5 |
| State-of-the-art | 25     | 56       | 58      |
| NIC              | 59     | 66       | 63      |
| Human            | 69     | 68       | 70      |

# Video description using LSTM and CNN (Venugopalan 2015)



# Video description using LSTM and CNN (Bolaños 2018)



Given a video  $v$ , search for a description of the video  $\hat{t}$ :

$$\begin{aligned}\hat{t}_1^I &= \operatorname{argmax}_{I,t} p(t_1^I | v) \\ &= \operatorname{argmax}_{I,t} \prod_{i=1}^I p(t_i | t_1^{i-1}, v)\end{aligned}$$

Microsoft Research Video Description Corpus

| Encoder                      | BLEU (%) | METEOR (%) |
|------------------------------|----------|------------|
| Objects + LSTM (Yao 2015)    | 51.5     | 32.5       |
| Objects + BLSTM (Peris 2016) | 53.6     | 32.6       |

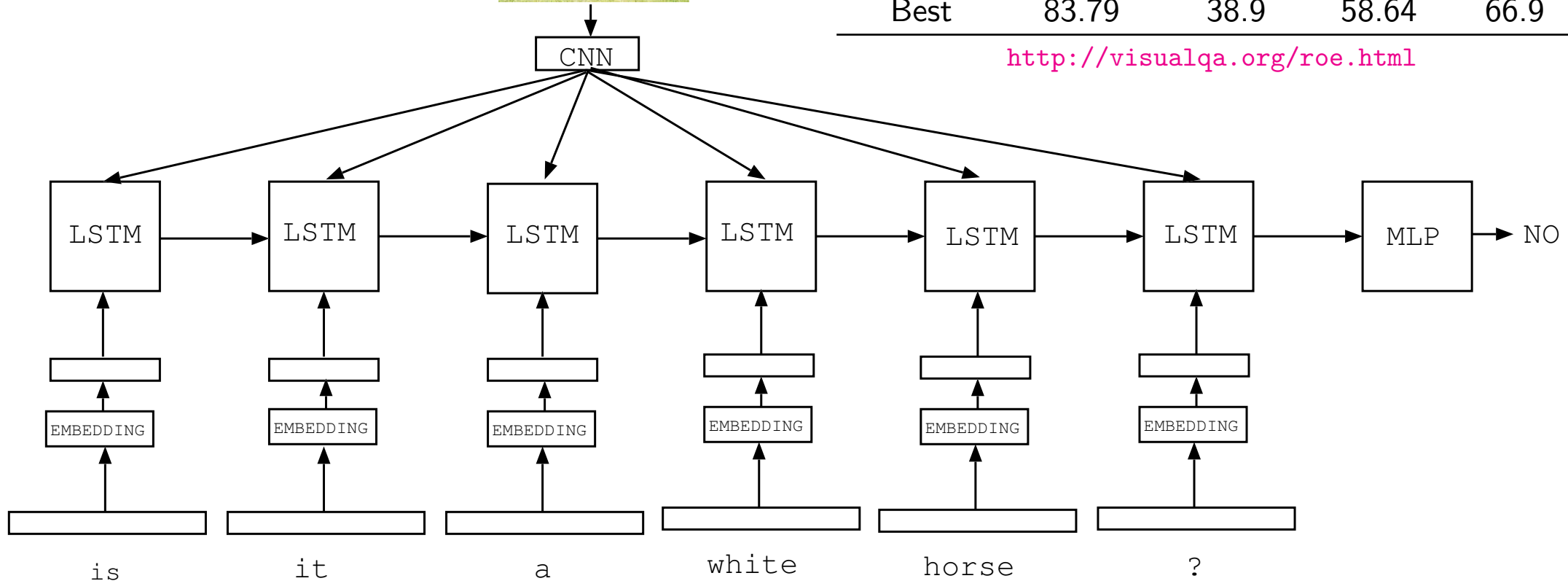
# Visual question answering



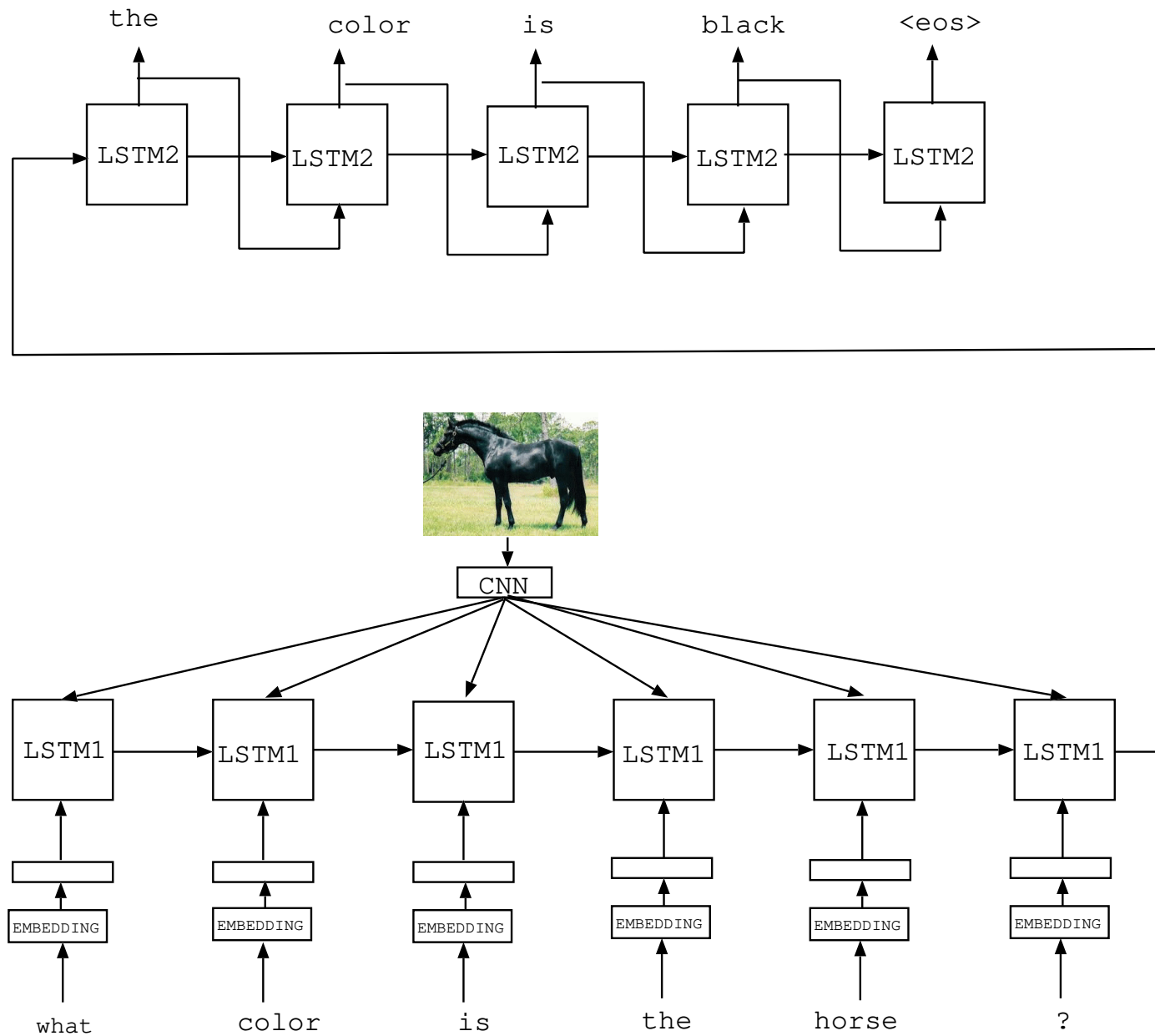
VQA Challenge 2016

| System | Yes/No | Number | Other | Overall |
|--------|--------|--------|-------|---------|
| UPV-UB | 79.24  | 35.72  | 40.79 | 56.1    |
| Best   | 83.79  | 38.9   | 58.64 | 66.9    |

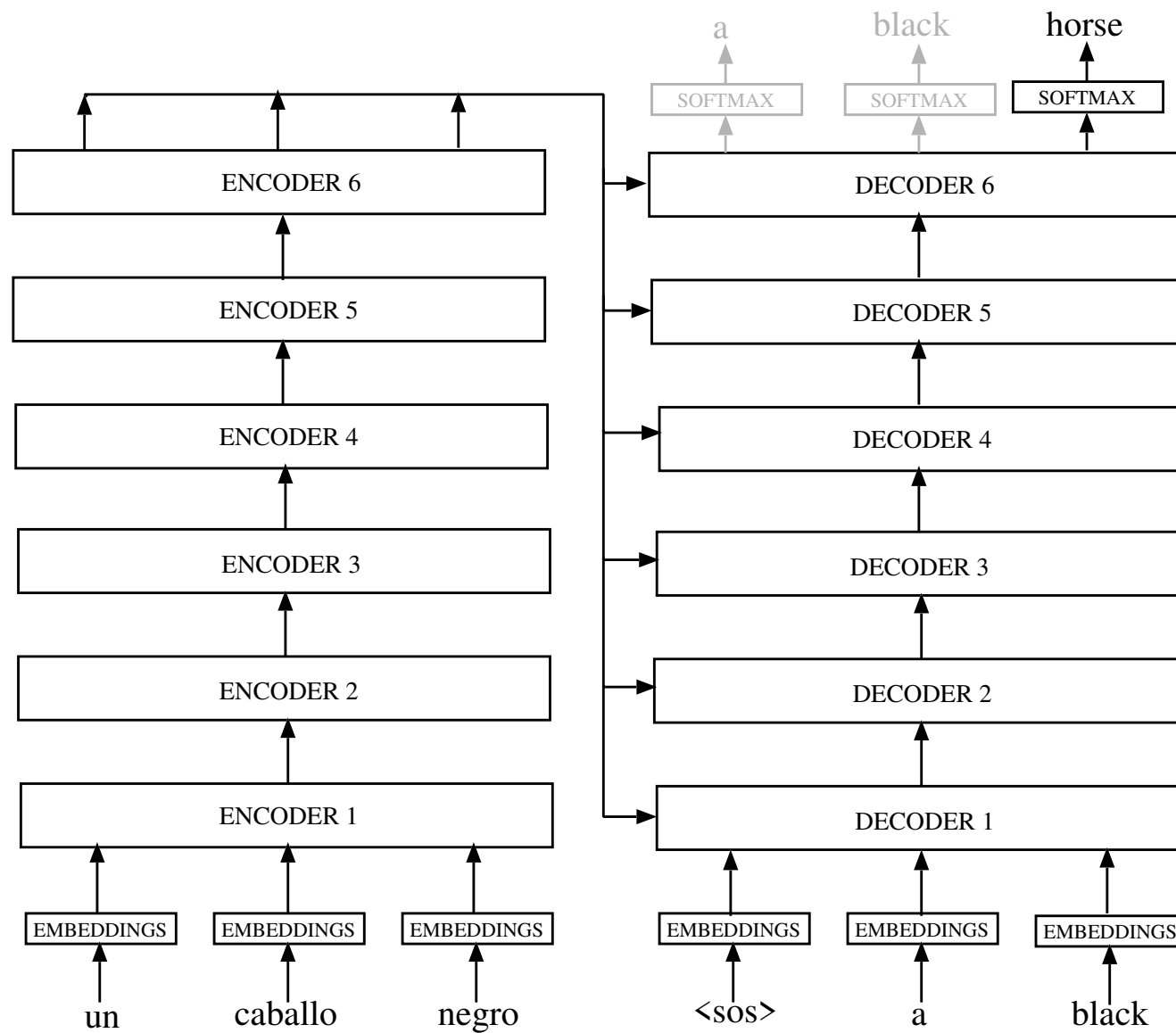
<http://visualqa.org/roe.html>



# Visual question answering

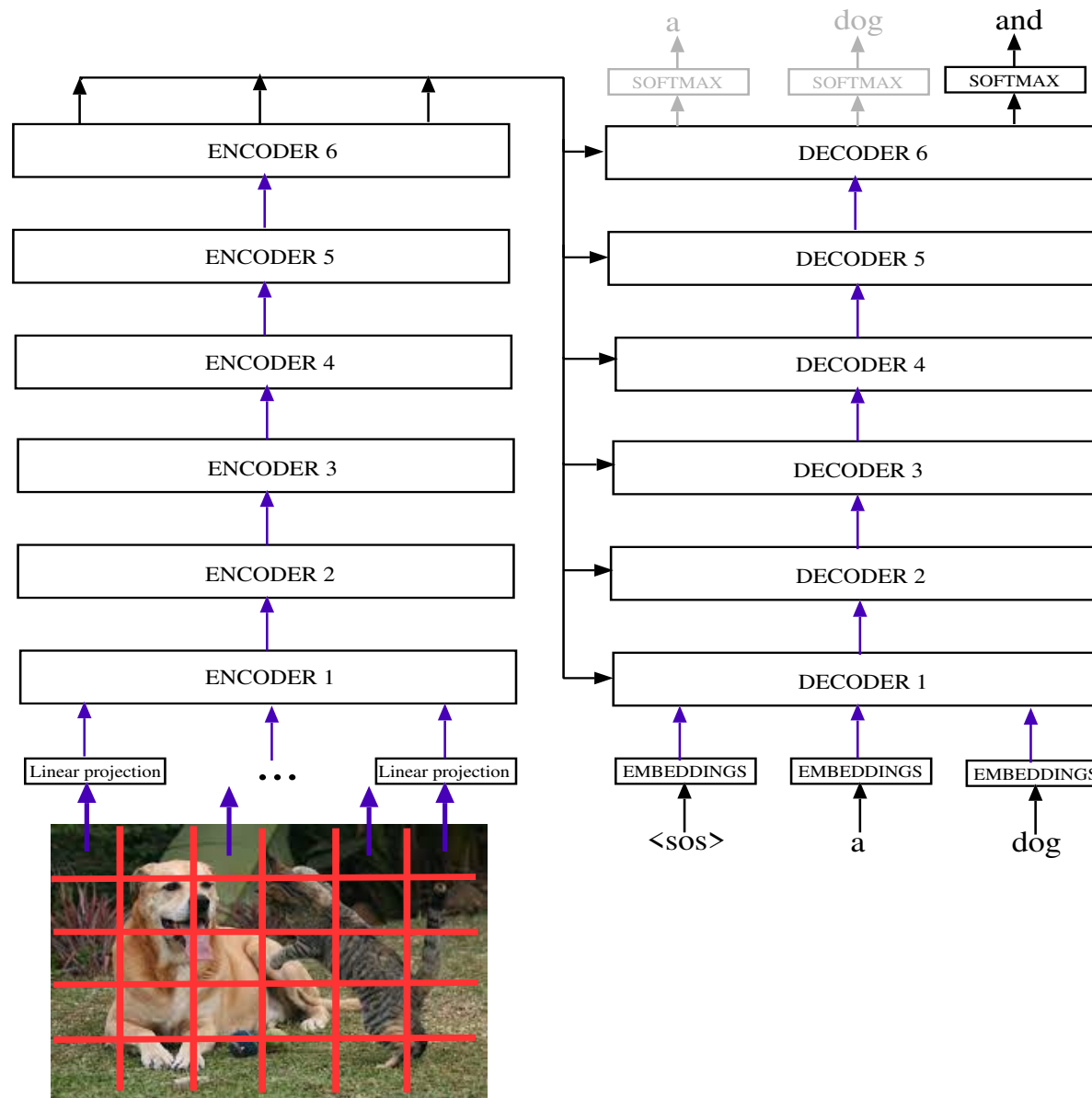


# Machine Translation with Transformer (Vaswani 2017)

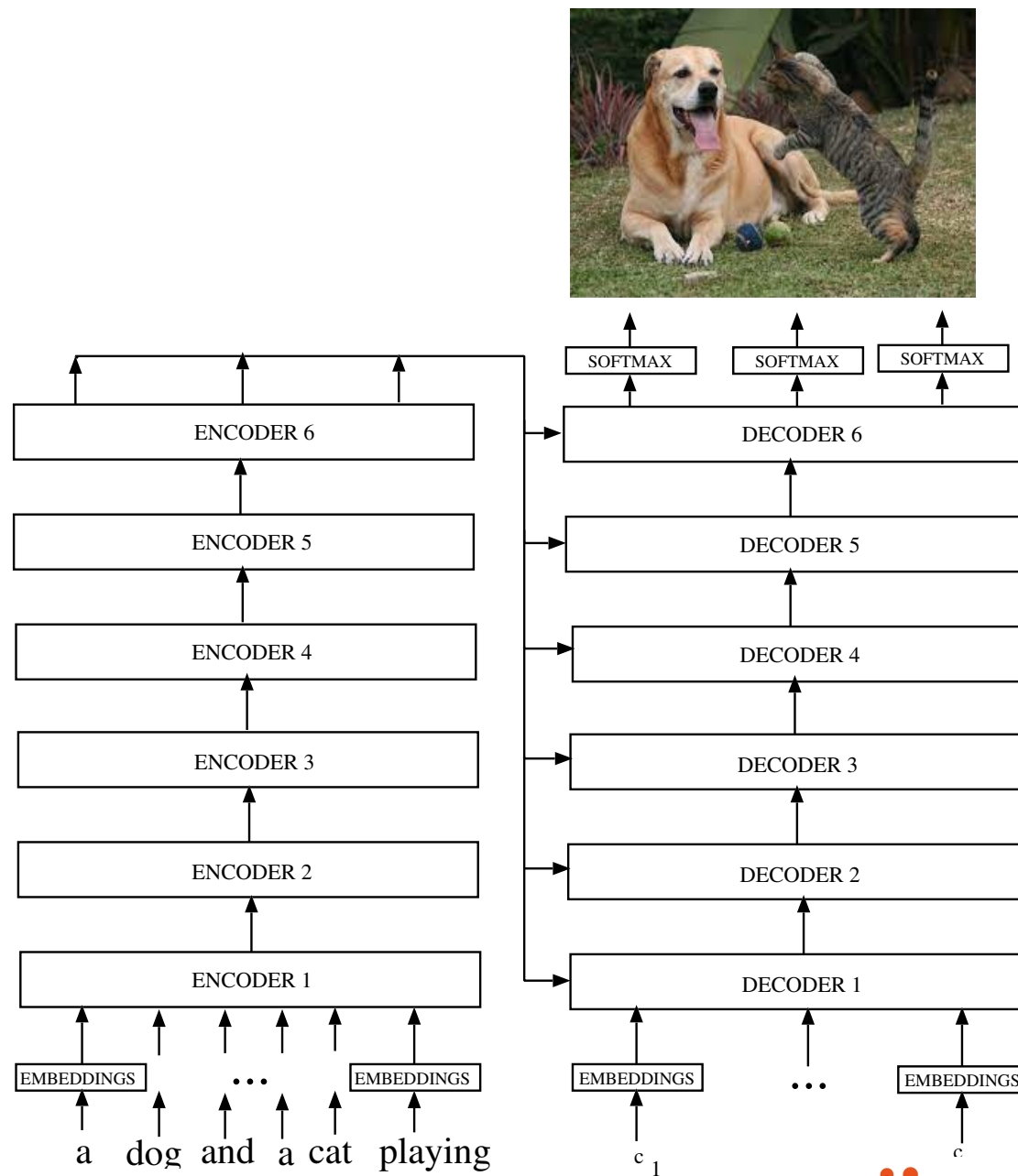




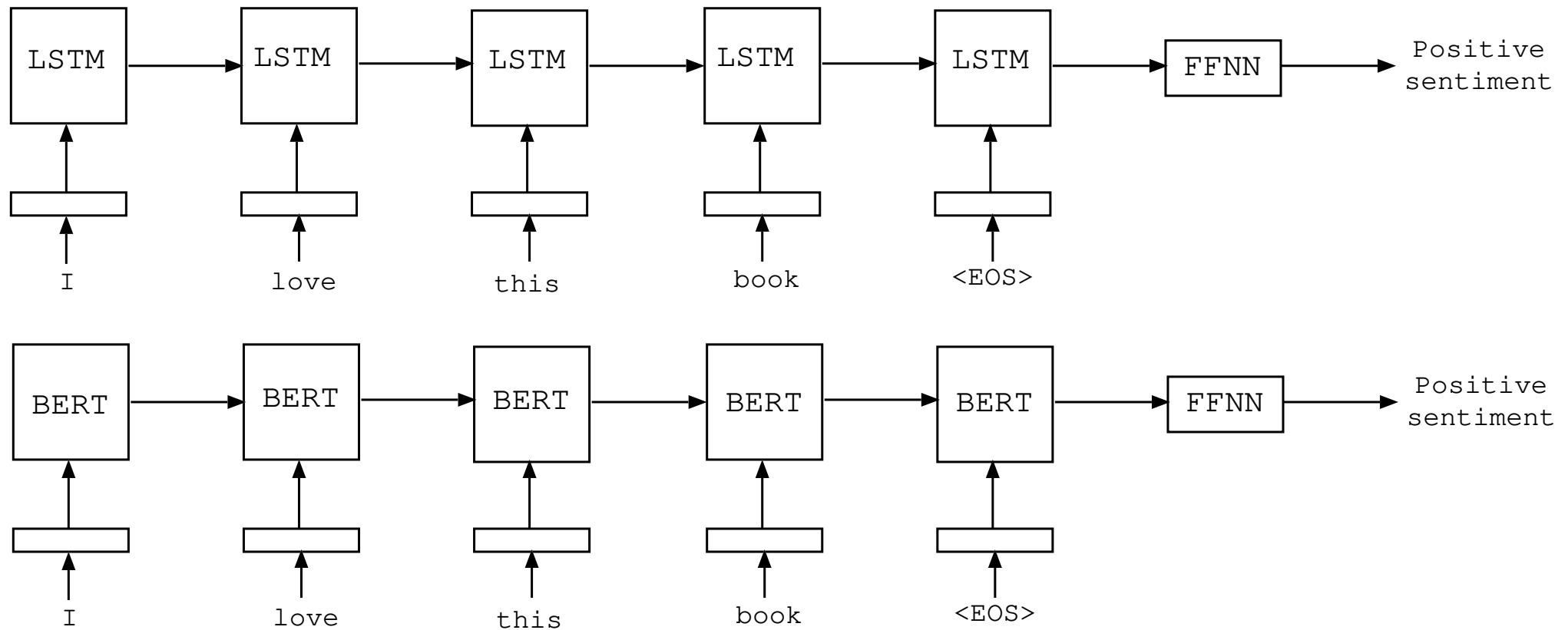
# Transformer for image caption [Wang 2022]



# Transformer for text-to-image generation



# Sentence-level classification [Aggarwal 2018]



# Index

- 1 Introduction ▷ 3
- 2 Sequence processing ▷ 6
- 3 Embeddings ▷ 17
- 4 Simple recurrent neural networks ▷ 25
- 5 Augmented recurrent neural networks ▷ 53
- 6 Long short-term memory ▷ 68
- 7 Connectionist temporal classification ▷ 72
- 8 Bidirectional recurrent neural networks ▷ 76
- 9 Sequence-to-sequence network architectures ▷ 81
- 10 Pre-trained models ▷ 113
- 11 Applications ▷ 129
- 12 *Bibliography* ▷ 153

# Bibliography (I)

- Aggarwal. Neural Networks and Deep Learning. Chap. 7. Springer. 2018.
- Alayrac et al. Flamingo: a Visual Language Model for Few-Shot Learning. NeuroNIPS. 2022.
- D. Bahdanau, K. Cho, Y. Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. ICLR. 2015.
- Bengio et al. A Neural Probabilistic Language Model. JMLR. 2003.
- Bolaños, A. Peris, F. Casacuberta, S. Soler, P. Radeva. Egocentric Video Description based on Temporally-Linked Sequences. Journal of Visual Communication and Image Representation, 2018.
- Castaño, F. Casacuberta. A Connectionist Approach to Machine Translation. Eurospeech. 1997.
- Cho et. al On the properties of neural machine translation: Encoder-decoder approaches. arXiv. 2014.
- Chowdhery et al. PaLM: Scaling Language Modeling with Pathways. arXiv. 2022.

## Bibliography (II)

- Conneau & Lample. Cross-lingual Language Model Pretraining. NeurNIPS
- Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL. 2019.
- Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2020.
- Elliott et. al. Multilingual Image Description with Neural Sequence Models. ICLR. 2016.
- Graves. Supervised Sequence Labelling with Recurrent Neural Networks. Springer. 2012.
- A. Graves, S. Fernández, F. Gómez. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. ICML 2006.
- Hawthorne et al. General-purpose, long-context autoregressive modeling with Perceiver AR. ICML 2022.
- Hochreiter & Schmidhuber. "Long short-term memory. Neural Computation. 1997.
- Hoffman et. al. Training Compute-Optimal Large Language Models. arXiv. 2022.

## Bibliography (III)

- Koehn. Neural Machine Translation. Cambridge University Press. 2020.
- Kondratenko & Kuperin. Using Recurrent Neural Networks To Forecasting of Forex. arXiv. 2003.
- Katzer et. al. Prediction of Human Full-Body Movements with Motion Optimization and Recurrent Neural Networks. IEEE ICRA. 2020.
- Lewis et al. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. ACL. 2020.
- Li & Jia. Visual Question Answering with Question Representation Update. NeurNIPS. 2016.
- Lin et. al. Pre-training Multilingual Neural Machine Translation by Leveraging Alignment Information. arXiv. 2021.
- Liu et al. Multilingual Denoising Pre-training for Neural Machine Translation. TACL. 2020.

## Bibliography (IV)

- Liwicki et al. A Novel Approach to On-Line Handwriting Recognition Based on Bidirectional Long Short-Term Memory Networks. ICDAR 2007.
- Lundstedt. A prototype real-time forecast service of space weather and effects using knowledge-based. Neurocomputing, 2001.
- Luong et al. Effective Approaches to Attention-based Neural Machine Translation. EMNLP. 2015.
- C. Martínez, A. Juan, F. Casacuberta. Iterative Contextual Recurrent Classification of Chromosomes. Neural Processing Letters. 2007.
- Mandic & Chambers. Recurrent Neural Networks for Prediction. John Wiley & Sons 2001.
- Medsker & Jain. Recurrent Neural Networks. CRC Press. 2001.
- Mikolov et al. Distributed Representations of Words and Phrases and their Compositionality. NIPS 2013.



## Bibliography (V)

- Mishra & Viradiya. Survey of Sentence Embedding Methods. JASC. 2019.
- Peters et. al. Deep contextualized word representations. NAACL. 2018.
- Qiu et. al. Pre-trained Models for Natural Language Processing: A Survey. SCTS. 2020.
- Radford et. al. Improving Language Understanding by Generative Pre-Training. OpenAI. 2018.
- Radford et. al. Learning Transferable Visual Models From Natural Language Supervision. ICML. 2021.
- Rae et. al. Scaling Language Models: Methods, Analysis & Insights from Training Gopher. DeepMind. 2021
- Raffel et. al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR. 2020.
- Reed et al. A Generalist Agent. MLR. 2022.

## Bibliography (VI)

- Reimers & Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
- Romero. Ultra-Large AI Models Are Over. <https://thealgorithmicbridge.substack.com/p/ultra-large-ai-models-are-over>. 2022.
- Schuster & Paliwal. Bidirectional Recurrent Neural Networks. IEEETSP. 1997.
- Sejnowski & Rosenberg. NETtalk: a parallel network that learns to read aloud. The Johns Hopkins University electrical engineering and computer science technical report. 1986.
- Schuster & Paliwal. Bidirectional recurrent neural networks. IEEE TSP. 1995.
- Sinha et. al. Extractive Text Summarization using Neural Networks. arXiv. 2018.
- Sundermeyer. Improvements in Language and Translation Modeling. Ph.D. RWTH Aachen. 2016.
- Tunstall et al. Efficient Few-Shot Learning Without Prompts. arXiv 2022.
- A. Vaswani et al. Attention Is All You Need. NIPS 2017.

## Bibliography (VII)

- Venugopalan et al. Sequence to Sequence - Video to Text. arXiv. 2015.
- Vinyals et al. A Neural Image Caption Generator (NIC). arXiv. 2015
- Wang et al. End-to-End Transformer Based Model for Image Captioning. AAAI 2022.
- Waibel et. al. Phoneme Recognition Using Time-Delay Neural Networks. IEEE TASSP 1989.
- Williams & Zipser. Gradient-based learning algorithms for recurrent networks and Their Computational Complexity. In "Back-propagation: Theory, Architectures and Applications". 1995.
- Xiong et al. On Layer Normalization in the Transformer Architecture. arXiv. 2020.
- Yang wt. al. XLNet: Generalized Autoregressive Pretraining for Language Understanding. NIPS. 2020.
- Yao & Wan. Multimodal Transformer for Multimodal Machine Translation. ACL. 2020.